

DOCUMENTACIÓN DE SISTEMA

Pañuelo al Viento

Documentación de sistema

Documentación completa

Versión del producto **0.1.0+1**

Commit **6efae74** · 2026-08-25

Rama **main**

Generado desde los Markdown de docs/system-documentation/ con tool/build_docs_pdf.py.
La versión y el commit se leen del repositorio; no están escritos a mano.

Contenido de este documento

Qué es este sistema

Propósito de esta documentación

Público

Índice

Documentación previa del repositorio

Datos del análisis

Convenciones

Qué no se pudo verificar, y por qué

Pendientes de validar

Regenerar los PDF

Documentación de sistema · Pañuelo al Viento

Portada e índice de la documentación técnica completa del repositorio [pañuelo-al-viento-cueca-app](#).

Qué es este sistema

Pañuelo al Viento es una aplicación educativa Flutter, local-first, que propone a niñas y niños de aproximadamente 10 años una ruta introductoria de 24 clases para acercarse a la cueca. Se distribuye como APK de Android y como ejecutable, MSI y ZIP portable de Windows. No tiene servidor, cuentas, publicidad ni analítica, y el APK publicado no declara ningún permiso del sistema.

Propósito de esta documentación

El repositorio ya tenía 18 documentos temáticos de producto, pedagogía, privacidad y operación. Esta carpeta **no los sustituye ni los repite**: añade la capa que faltaba —mapa del código, referencia símbolo a símbolo, explicación del flujo interno, trazabilidad y registro de hallazgos— y enlaza a los existentes cuando ya cubren un tema mejor de lo que lo haría una copia.

Cuando un tema ya está resuelto en [docs/](#), aquí encontrarás el enlace y el contexto técnico que lo conecta con el código, no una segunda versión del texto.

Público

Perfil	Por dónde empezar
Persona desarrolladora que llega hoy	18 · Guía de incorporación
Quien necesita decidir sin detalle técnico	17 · Resumen ejecutivo

Quien va a modificar el código	04 · Mapa del código y 06 · Explicación profunda
Quien audita privacidad o seguridad	11 · Seguridad y 15 · Riesgos
Quien opera la publicación	13 · Despliegue y operación
Quien no es técnico	01 · Visión general , sección final

Índice

#	Documento	Contenido	Estado
—	README	Portada, índice, convenciones y pendientes.	Completo
01	Visión general	Qué hace el sistema, para quién y qué no hace. Incluye explicación para persona no técnica.	Completo
02	Instalación y ejecución	Requisitos, arranque desde cero y las carpetas que no existen hasta que alguien las genera.	Completo
03	Arquitectura	Capas, decisiones y diagramas.	Completo
04	Mapa del código	Inventario jerárquico con responsabilidad, dependencias y estado aparente.	Completo
05	Referencia técnica	Catálogo de consulta: firma, efectos, llamantes y riesgo al modificar.	Completo
06	Explicación profunda	Flujo interno, módulo a módulo.	Completo
07	Persistencia	No hay base de datos: mecanismo real de persistencia, diccionario y ciclo de vida.	Completo
08	Flujo de datos	Recorrido del dato desde el JSON hasta la pantalla y de vuelta al disco.	Completo
09	APIs e integraciones	Demostración de que no hay red y contratos que sí existen.	Completo

10	Configuración	Manifiestos, variables de entorno, secrets y valores fijados.	Completo
11	Seguridad	Controles presentes y ausentes, superficie y hallazgos.	Completo
12	Pruebas y calidad	Las 25 pruebas, los validadores y lo que ninguno cubre.	Completo
13	Despliegue y operación	CI, release, landing y generación de estos PDF.	Completo
14	Troubleshooting	Síntoma, causa, diagnóstico, solución y riesgo de aplicarla.	Completo
15	Riesgos y deuda técnica	Hallazgos registrados, ninguno corregido .	Completo
16	Glosario	Términos técnicos, culturales y los que significan algo propio aquí.	Completo
17	Resumen ejecutivo	Para decisión, con esfuerzo cuantificado.	Completo
18	Guía de incorporación	Itinerario de los primeros días.	Completo
19	Matriz de trazabilidad	Funcionalidad → módulo → función → dato → prueba, en una fila.	Completo

Versiones PDF de todos ellos, más un consolidado, en [pdf/](#).

Documentación previa del repositorio

Estos documentos ya existían y **siguen siendo la fuente autorizada** de su tema. Esta carpeta los referencia; no los duplica.

Documento existente	Tema del que es fuente autorizada
../README.md	Presentación pública, estado verificable y descargas.
../ARCHITECTURE.md	Decisión arquitectónica y evolución futura sin romper privacidad.
../CURRICULUM.md	Programa pedagógico, mapa de ocho semanas y evaluación formativa.
../PEDAGOGY_AND_SAFETY.md	Enfoque, consentimiento corporal y reglas de seguridad.
../ACCESSIBILITY.md	Soporte de accesibilidad, equivalencias y límites conocidos.

../PRIVACY.md	Inventario de datos infantiles y condiciones de ampliación.
../PERMISSIONS.md	Matriz exacta de permisos y activaciones del dispositivo.
../BUILD_MOBILE.md · ../BUILD_WINDOWS.md	Procedimiento de compilación por plataforma.
../VALIDATION.md · ../VALIDATION_RESULT.md	Cómo validar y qué se midió en 0.1.0.
../RELEASE_CHECKLIST.md	Criterios de publicación.
../TEACHER_REVIEW.md	Protocolo de revisión docente y cultural.
../SOURCES.md · ../CONTENT_LICENSES.md · ../CONTENT_PRODUCTION.md	Procedencia cultural y licencias de contenido.
../PARENT_GUIDE.md · ../TEACHER_GUIDE.md · ../PRACTICE_PLAN_8_WEEKS.md	Uso en familia y en aula.
../SECURITY.md · ../CONTRIBUTING.md · ../ROADMAP.md · ../CHANGELOG.md	Reporte de seguridad, contribución, plan e historial.

Datos del análisis

Aspecto	Valor
Repositorio	pañuelo-al-viento-cueca-app
Rama analizada	main
Commit analizado	6efae74 · «Publica la landing en GitHub Pages» · 2026-08-25
Versión del producto	0.1.0+1, leída de pubspec.yaml
Versión del currículo	1.0.0, campo version de assets/content/curriculum.json
Fecha del análisis	2026-08-27
Toolchain usada para verificar	Flutter 3.44.6 · Dart 3.12.2 · Node v24.11.1
Estado del árbol al empezar	Limpio

La versión del producto y la del currículo son números distintos y no se mueven juntos. Confundirlos al leer el JSON es un error fácil de cometer.

Convenciones

Contenido y secciones en español. Los identificadores de código, claves de configuración y nombres de archivo se citan **en su forma original**, sin traducir, para que buscar el texto de un documento dentro del repositorio funcione.

Cada afirmación sin marcador está anclada a un archivo, un símbolo o un comando ejecutado. Lo demás lleva marcador:

Marcador	Significado
----------	-------------

(sin marcador)

Hecho verificado, con archivo y símbolo citados.

INFERENCIA

Conclusión razonada, no afirmación literal del repositorio.

REQUIERE VALIDACIÓN

Depende de ejecución real, de un servicio externo o de una decisión humana.

NO DOCUMENTADO EN EL REPOSITORIO

Existe en el código pero nadie lo explica.

NO IDENTIFICADO

Se buscó y no hay evidencia en ningún sentido.

Qué no se pudo verificar, y por qué

Una documentación que oculta sus límites es peor que una incompleta que los declara. Esto es lo que quedó fuera de comprobación directa:

Aspecto	Motivo
Compilación de Android y de Windows	Este análisis se hizo en Windows sin Android SDK ni Visual Studio con carga de C++. <code>flutter analyze</code> y <code>flutter test</code> sí se ejecutaron.
Contenido del APK publicado	<code>tool/verify_apk.mjs</code> necesita <code>aapt2</code> de las <code>build-tools</code> de Android, ausentes en la máquina de análisis. Los valores que cita <code>../VALIDATION_RESULT.md</code> se reproducen como afirmación del repositorio, no como medición propia.
Comportamiento en dispositivo real	No se instaló en un teléfono ni se ejecutó el escritorio. Vibración, clic del sistema y persistencia entre reinicios quedan REQUIERE VALIDACIÓN .
Ejecución de los workflows	No se dispararon. Su contenido se leyó línea a línea; su resultado no se observó.
Validez pedagógica y cultural	Fuera del alcance de una documentación técnica y expresamente pendiente según <code>../TEACHER_REVIEW.md</code> .
Accesibilidad con tecnologías de apoyo	No se probó TalkBack ni el Narrador de Windows. Lo que existe son etiquetas semánticas en el código, que no equivalen a una auditoría.

Pendientes de validar

Recogidos aquí para que no haya que buscarlos por los veinte documentos. El detalle está en [15 · Riesgos](#).

1. Que el pulso del laboratorio siga sonando al cambiar de pestaña es comportamiento **verificado en pruebas**, pero falta decidir si es un fallo o una función; `../PERMISSIONS.md` afirma lo contrario.
2. Si los siete valores de `diagram` del currículo deben tener siete dibujos distintos, o si `pair` y `free` comparten esquema a propósito.
3. Si `tool/validate_curriculum.dart` debe seguir existiendo junto a su gemelo en Node.
4. Los datos del APK publicado, hasta que alguien con las `build-tools` ejecute `node tool/verify_apk.mjs`.
5. Todo el bloque de validación humana que ya declara `../VALIDATION_RESULT.md`.

Regenerar los PDF

```
python tool/build_docs_pdf.py          # los veinte documentos y el consolidado
python tool/build_docs_pdf.py --only 03-architecture.md
```

El generador está descrito en [13 · Despliegue y operación](#).

Contenido de este documento

En una frase

Qué hace

Qué no hace

Cifras del sistema, contadas

Superficies del producto

Decisiones que explican el sistema

Estado actual, sin adornos

Para una persona no técnica

Continuar por

01 · Visión general del sistema

En una frase

Una aplicación Flutter de escritorio y móvil que lleva un currículo de 24 clases de cueca dentro de su propia instalación, guarda en el dispositivo qué clases se completaron y no habla con ningún servidor.

Qué hace

Función	Dónde vive	Qué produce
Mostrar la próxima clase pendiente	<code>lib/screens/home_tab.dart</code>	Una sola acción destacada, no un catálogo.
Recorrer los 8 niveles y sus 24 clases	<code>lib/screens/route_tab.dart</code>	Lista desplegable con avance por nivel.
Guiar una clase	<code>lib/screens/lesson_screen.dart</code>	Contexto, esquema de piso, tres actividades marcables, reto, seguridad, alternativa accesible y consejo adulto.
Entrenar el pulso	<code>lib/screens/rhythm_lab_screen.dart</code>	Metrónomo visual de seis pulsos con dos agrupaciones y salidas opcionales de sonido y vibración.
Mostrar el avance y la frontera de privacidad	<code>lib/screens/progress_tab.dart</code>	Porcentaje total, porcentaje por nivel, estado de cámara y micrófono, y reinicio confirmado.

Qué no hace

Esta lista es tan importante como la anterior, porque delimita la promesa del producto y está comprobada por herramientas del propio repositorio.

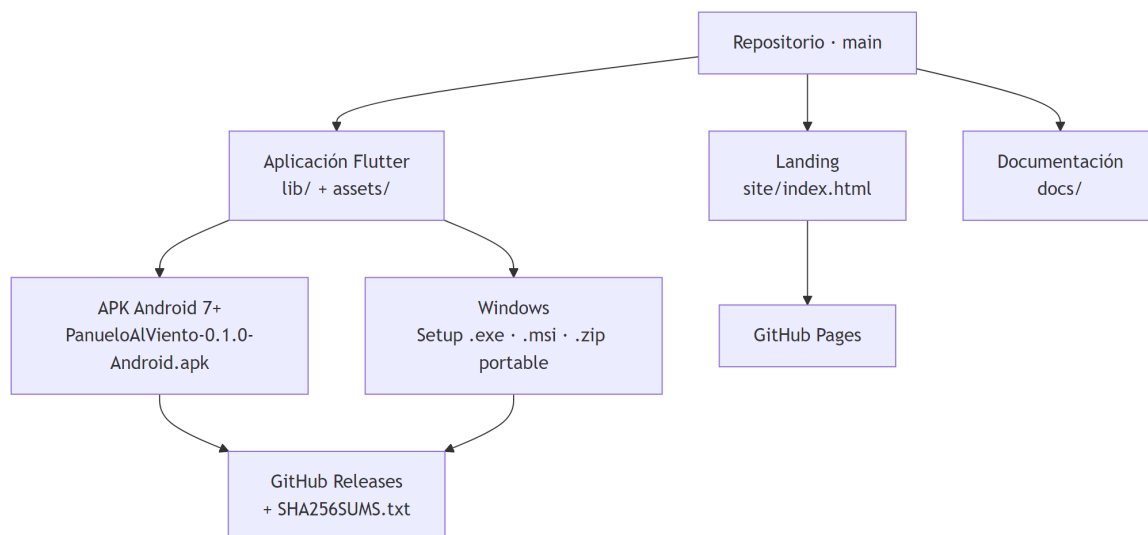
- No pide cuenta, nombre, edad, correo, escuela ni ubicación.
- No usa cámara ni micrófono: no hay botón, ni llamada, ni permiso.
- No hace llamadas de red. El APK publicado ni siquiera declara `android.permission.INTERNET`.
- No incluye música, video ni fotografía. La decisión es explícita: evita distribuir medios sin derechos verificables. Ver [../CONTENT_LICENSES.md](#).
- No puntúa, no compara cuerpos, no ordena en un ranking y no penaliza repetir.
- No evalúa la postura ni el movimiento con sensores.
- No exporta ni importa el avance. Desinstalar lo pierde para siempre.

Cifras del sistema, contadas

Todas obtenidas contando, no estimando.

Magnitud	Valor	Cómo se obtuvo
Archivos Dart de producto	15, en <code>lib/</code>	<code>wc -l lib/*.dart lib/**/*.dart</code>
Líneas Dart de producto	2 417 (2 123 antes de documentar el código)	ídem
Archivos de prueba	6, en <code>test/</code>	<code>wc -l test/*.dart</code>
Pruebas que ejecuta la suite	25	Salida de <code>flutter test</code> : <code>All tests passed!</code> con <code>+25</code>
Herramientas del repositorio	11, en <code>tool/</code>	Listado del directorio
Workflows	3	<code>.github/workflows/</code>
Dependencias directas de ejecución	1 (<code>shared_preferences: ^2.5.3</code>)	<code>pubspec.yaml</code>
Paquetes resueltos en total	52	<code>pubspec.lock</code>
Niveles / clases / actividades	8 / 24 / 72	<code>assets/content/curriculum.json</code>
Duración de cada clase	12 minutos, las 24	Campo <code>durationMinutes</code> de las 24 clases
Duración total del programa	288 minutos	Suma de las 24
Tamaño del currículo	43 165 bytes	<code>wc -c assets/content/curriculum.json</code>
Capturas versionadas	9	<code>docs/screenshots/</code>

Superficies del producto



El diagrama muestra qué se produce a partir del repositorio y hacia dónde va. No muestra el orden temporal ni las compuertas de calidad que hay entre medias: eso está en [13 · Despliegue y operación](#). Tampoco muestra las carpetas `android/` y `windows/`, porque no existen en el repositorio: se generan al compilar (ver [02 · Instalación](#)).

Decisiones que explican el sistema

El currículum es un dato, no código. Vive en un único JSON dentro de `assets/`. Eso permite editar clases sin recompilar lógica, validarlas con herramientas que no necesitan Flutter, y —según `../ARCHITECTURE.md`— traducirlas o generar guías impresas en el futuro. El precio es que no hay esquema JSON formal ni migraciones: hoy la disciplina la ponen dos validadores y tres pruebas.

El estado se pasa a mano. No hay contenedor de inyección ni gestor de estado global. `main` construye un `AppState` y lo entrega a la raíz. Con una sola fuente de estado y quince archivos, un gestor global no resolvería ningún problema existente y añadiría uno que entender.

La persistencia es transaccional. El estado en memoria solo cambia después de que el disco confirme. Es la regla más protegida del proyecto: tiene tres pruebas dedicadas en `test/app_state_test.dart` que existen específicamente para impedir que alguien invierta el orden.

El metrónomo no usa `Timer.periodic`. Usa un `Stopwatch` monotónico y temporizadores de un disparo calculados contra un instante absoluto, porque un temporizador periódico acumula el retraso de cada tick y el metrónomo se queda atrás sin recuperarse. Ver [06 · Explicación profunda](#).

La verificación mira el binario, no solo el repositorio. `tool/verify_apk.mjs` abre el APK compilado y cuenta las clases que trae dentro. Un repositorio impecable puede producir un artefacto vacío, y un build en verde no lo detecta.

Estado actual, sin adornos

0.1.0 es software verificado: las 25 pruebas pasan, el análisis estático está limpio, la coherencia entre manifiesto, historial, notas y README la comprueba un validador, y el artefacto publicado se abrió y se midió.

No es todavía un **método pedagógico validado**. Faltan la revisión por docentes de Educación Física, la de personas cultoras de las variantes representadas y la prueba presencial con niñas y niños. Tampoco hay clave de firma Android permanente, así que la primera actualización obligará a desinstalar y borrará el avance local. El propio repositorio dice esto sin maquillarlo en `../VALIDATION_RESULT.md` y en el README; es una de las propiedades que conviene proteger.

Para una persona no técnica

Imagina un cuaderno de 24 lecciones para aprender a bailar cueca, pensado para alguien de unos diez años. Cada lección cabe en doce minutos y siempre tiene la misma forma: primero se descubre algo, después se practica, y al final se piensa en lo que pasó. El cuaderno viene con un mapa dibujado del recorrido que harán los pies, un aviso de seguridad y —esto importa— una manera alternativa de hacerlo si alguien no puede moverse igual que los demás.

La aplicación es ese cuaderno metido dentro del teléfono o del computador. Y está metido **de verdad**: todas las lecciones viajan dentro del programa, así que funciona en un cerro sin señal, en una sala sin wifi o con el modo avión puesto. No hay que registrarse, no hay que dar un correo y no hay anuncios.

Trae además un juguete rítmico: seis luces que se encienden por turno, como un péndulo, para aprender a sentir el compás. Se le puede subir o bajar la velocidad, y opcionalmente hacer que suene un clic o que el teléfono vibre en los tiempos fuertes. Ese clic y esa vibración son cosas que la aplicación *pide* al teléfono, como quien pide encender una lámpara. No son un micrófono: no escuchan nada.

De hecho, la aplicación no tiene cámara ni micrófono en absoluto. Y no es una promesa: hay tres comprobaciones automáticas que revisan el código, el archivo de instalación a medio hacer y el archivo terminado, y bloquean la publicación si alguna de las tres los encuentra. Lo único que la aplicación guarda es una lista como «lección 1, lección 2, lección 5», y esa lista vive únicamente en el aparato de quien la usa. Nadie más la ve.

Lo que todavía no es. El programa está bien construido, pero *bien construido* y *pedagógicamente correcto* no son lo mismo. Ninguna profesora de Educación Física ni ninguna persona cultora de cueca ha revisado todavía estas 24 lecciones y firmado que estén bien planteadas. Tampoco se ha probado con niños de verdad en una sala. La aplicación lo dice de frente en su propia documentación en vez de dar a entender lo contrario, y esa honestidad es una de sus mejores propiedades.

Hay además un detalle práctico que conviene saber: por ahora, cuando salga una versión nueva, probablemente habrá que borrar la anterior antes de instalarla, y al borrarla se pierde la lista de lecciones completadas. Conviene anotarlas en un papel antes. Es un problema conocido y tiene solución; simplemente todavía no se ha aplicado.

Continuar por

- [02 · Instalación y ejecución](#) para poner el proyecto en marcha.
- [03 · Arquitectura](#) para entender cómo encajan las piezas.
- [17 · Resumen ejecutivo](#) si hay que decidir algo.

Contenido de este documento

La sorpresa número uno al clonar

Requisitos

Puesta en marcha

Ejecutar sin instalar Flutter

Comandos de uso diario

Dónde queda cada cosa al compilar

Verificación del entorno usado en este análisis

Continuar por

02 · Instalación y ejecución

La sorpresa número uno al clonar

`android/` y `windows/` no existen en el repositorio. Están excluidas por `.gitignore`:

```
# Generated platform files are created by tool/bootstrap.* and CI.  
/android/  
/windows/  
/release-artifacts/
```

Quien clone y ejecute `flutter run` sin más se encontrará con que no hay ninguna plataforma disponible. No es un error: es una decisión declarada en `./ARCHITECTURE.md` —«evitar almacenar plantillas antiguas reduce fallos al abrir el repositorio años después»—. Las plantillas nativas se regeneran con la versión de Flutter que tengas instalada, y después `tool/configure_platforms.mjs` les aplica la identidad del producto.

El paso obligatorio es, por tanto, ejecutar el bootstrap antes de cualquier otra cosa.

Requisitos

Requisito	Versión	Para qué
Flutter estable	3.44.6 fijada en CI; <code>pubspec.yaml</code> admite desde 3.29.0	Todo
Dart	3.12.2 (viene con Flutter 3.44.6); <code>pubspec.yaml</code> admite desde 3.7.0	Todo
Node.js	Cualquiera con soporte de módulos ES. Verificado con v24.11.1	Los 11 scripts de <code>tool/</code>
JDK 17	—	Compilar Android

Android SDK con build-tools	—	Compilar y verificar el APK
Visual Studio 2022 con <i>Desktop development with C++</i>	—	Compilar Windows
Google Chrome	—	Solo para regenerar capturas
Python 3 con markdown, xhtml2pdf, PyMuPDF	Verificado con 3.12.9	Solo para regenerar los PDF de esta carpeta

La versión de Flutter no es una recomendación blanda: CI ejecuta `dart format --set-exit-if-changed`, y el formateador cambia entre versiones. Con otra versión el formato puede fallar aunque el código sea correcto. Está avisado en [../CONTRIBUTING.md](#).

Puesta en marcha

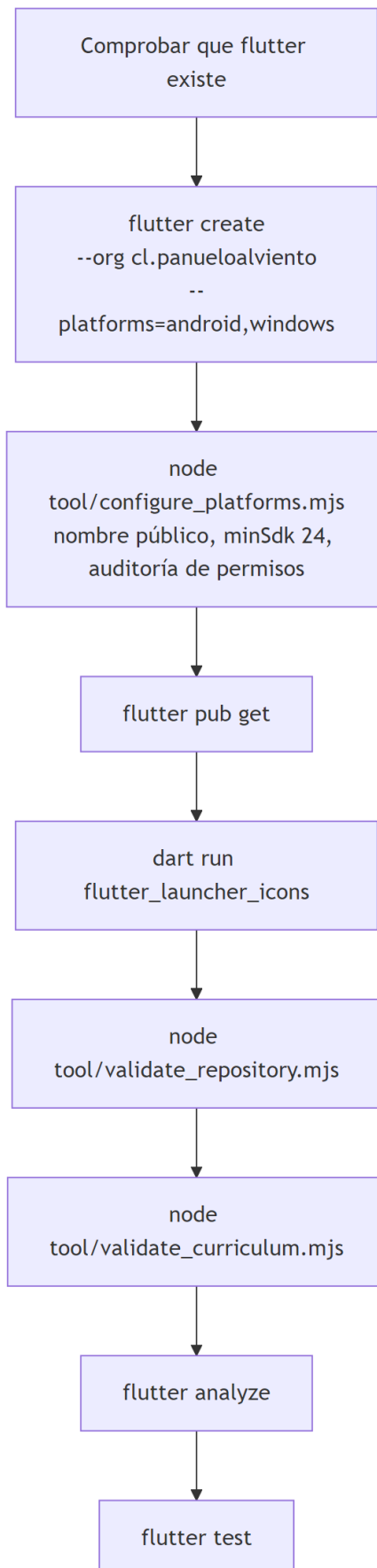
Windows

```
.\tool\bootstrap.ps1
flutter run -d windows
```

Linux y macOS, para Android

```
./tool/bootstrap.sh
flutter devices
flutter run -d <id-del-dispositivo>
```

Ambos scripts hacen lo mismo salvo que la versión PowerShell añade `flutter config --enable-windows-desktop` antes de empezar:



El diagrama muestra la secuencia completa de `tool/bootstrap.ps1` y `tool/bootstrap.sh`. Lo que no muestra: cada paso aborta el script si falla —`$ErrorActionPreference = 'Stop'` en PowerShell, `set -euo pipefail` en Bash—, de modo que el bootstrap es también una compuerta de calidad, no solo una preparación.

`tool/configure_platforms.mjs` lanza una excepción si no encontró ninguna plataforma que configurar (No se encontró ninguna plataforma generada), lo que convierte el olvido de `flutter create` en un error explícito en vez de en una compilación con el nombre genérico `panuelo_al_viento`.

Ejecutar sin instalar Flutter

Buena parte de la verificación no necesita el toolchain. Es una propiedad deliberada del proyecto:

```
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
node tool/build_site.mjs
```

Salida real de las tres, sobre el commit `6efae74`:

```
Repositorio válido: 32 archivos esenciales, 9 capturas y versión 0.1.0+1 coherente en pubspec, CHANGELOG, notas y README.
Currículo válido: 8 niveles, 24 clases y 72 actividades.
Landing ensamblada en C:\dev\panuelo-al-viento-cueca-app\build\site: versión 0.1.0 en 5 lugares, 9 capturas y 9 imágenes comprobadas.
```

Comandos de uso diario

Comando	Para qué	Nota
<code>flutter pub get</code>	Resolver dependencias	<code>pubspec.lock</code> está versionado, así que resuelve igual en toda máquina
<code>flutter run -d windows</code>	Ejecutar en escritorio	Necesita <code>windows/</code> generado
<code>flutter test</code>	Las 25 pruebas	Unos 2 s
<code>flutter analyze</code>	Análisis estático	Unos 3 s en caliente, ~57 s en frío
<code>dart format --output=none --set-exit-if-changed lib test tool</code>	Compuerta de formato	Sin <code>--set-exit-if-changed</code> el paso pasa siempre
<code>dart run tool/validate_curriculum.dart</code>	Validador de currículo en Dart	Gemelo del <code>.mjs</code> ; CI no lo ejecuta
<code>node tool/build_site.mjs --serve</code>	Landing en <code>http://127.0.0.1:8080</code>	Puerto configurable con <code>SITE_PORT</code>
<code>node tool/verify_apk.mjs <apk> [versión]</code>	Abrir y medir un APK	Necesita <code>aapt2</code> y <code>unzip</code>
<code>node tool/capture_screenshots.mjs</code>	Rehacer <code>docs/screenshots/</code>	Compila para web y conduce Chrome sin ventana
<code>python tool/build_docs_pdf.py</code>	Regenerar los PDF de esta carpeta	Ver 13

Dónde queda cada cosa al compilar

Ruta	Contenido	¿Versionada?
<code>build/app/outputs/flutter-apk/app-release.apk</code>	APK de Android	No, <code>build/</code> está ignorada
<code>build/windows/x64/runner/Release/</code>	Ejecutable, DLL y carpeta <code>data</code>	No
<code>build/site/</code>	Landing ensamblada	No
<code>docs/system-documentation/pdf/</code>	PDF de esta documentación	Sí
<code>release-artifacts/</code>	Artefactos con nombre final	No, ignorada
<code>.mermaid-cache/</code>	Diagramas rasterizados por el generador de PDF	No, ignorada

El ZIP portable de Windows contiene la carpeta `Release` completa. Extraer solo el `.exe` no funciona: necesita `flutter_windows.dll` y la carpeta `data`. Está advertido en el README y en [../BUILD_WINDOWS.md](#).

Verificación del entorno usado en este análisis

```
Flutter 3.44.6 • channel stable
Framework • revision ee80f08bbf • 2026-07-08
Engine • hash d3a3293399556a85388faf8c6f0723a7a5597aa8
Tools • Dart 3.12.2 • DevTools 2.57.0

node v24.11.1
Python 3.12.9
```

La ruta del SDK en la máquina de análisis fue `C:\dev\tools\flutter`, que no estaba en el `PATH` de la sesión. Si `flutter` no responde, la ruta real puede deducirse de `.dart_tool/package_config.json`, donde el paquete `flutter` apunta al SDK que resolvió las dependencias.

Continuar por

- [10 · Configuración](#) para las variables de entorno y los valores fijados.
- [14 · Troubleshooting](#) si algo de lo anterior falla.
- [../BUILD_MOBILE.md](#) y [../BUILD_WINDOWS.md](#) para el detalle por plataforma: siguen siendo la fuente autorizada de ese tema.

Contenido de este documento

La forma del sistema

Las capas, con su regla

Las cinco decisiones que hay que entender

Ciclo de vida de la aplicación

Navegación

Adaptación a la pantalla

Lo que la arquitectura hace bien y conviene no romper

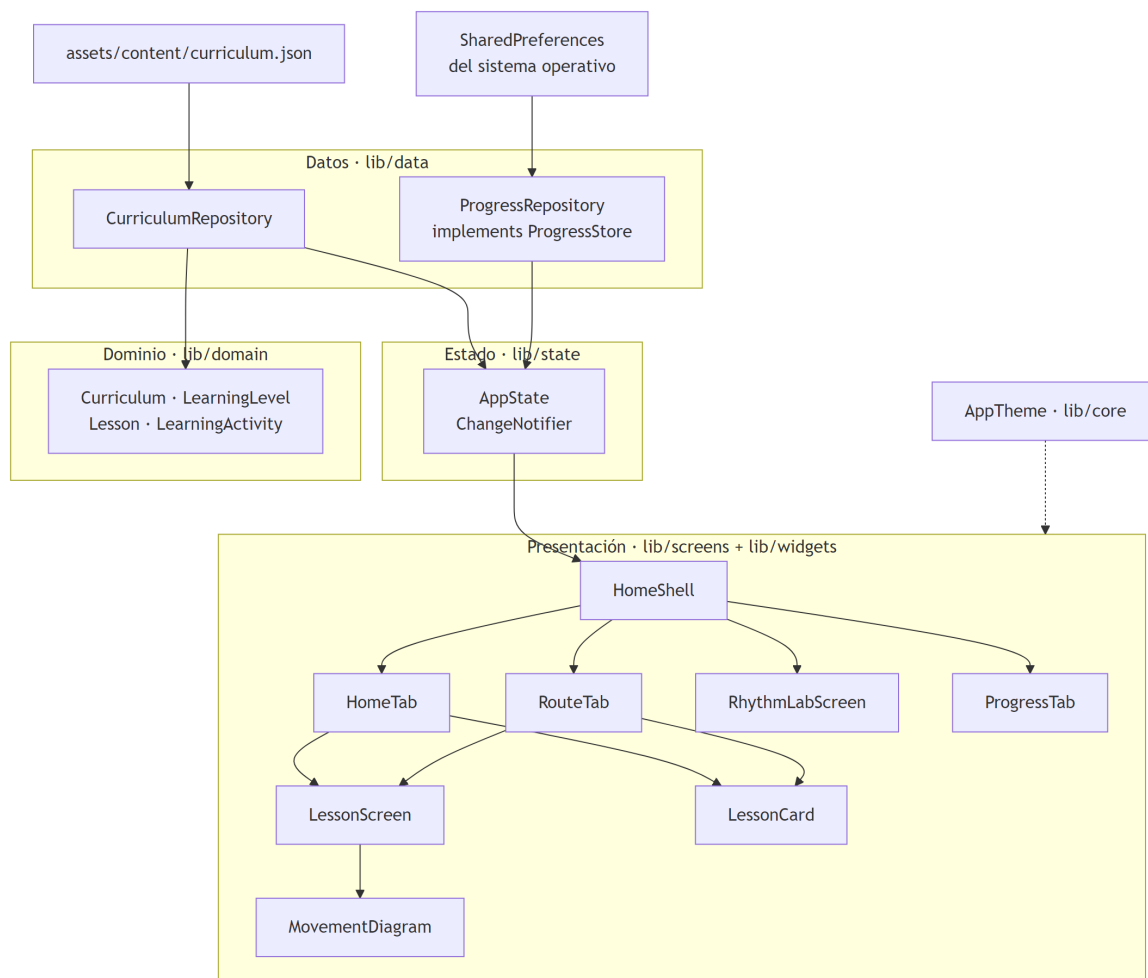
Continuar por

03 · Arquitectura

[../ARCHITECTURE.md](#) es la fuente autorizada de la **decisión** arquitectónica y de la evolución prevista. Este documento describe la **estructura tal como está construida hoy**, con los símbolos concretos, y nombra lo que ese documento no llega a cubrir.

La forma del sistema

Cuatro capas, dependencia en un solo sentido, sin ciclos.



Lo que el diagrama muestra: las dependencias reales, comprobables leyendo los `import` de cada archivo. `domain` no importa nada del proyecto ni de Flutter; `data` importa `domain`; `state` importa `data` y `domain`; `screens` importan `state`, `domain` y `widgets`.

Lo que **no** muestra: `RhythmLabScreen` no aparece conectada a `AppState` porque no lo está —es la única pantalla sin estado compartido, y por eso su constructor es `const`—; y `AppTheme` llega a todas las pantallas por el `ColorScheme` de Material, no por importación directa salvo en dos casos (`home_tab.dart` y `movement_diagram.dart` importan `AppColors` para colores fijos).

Las capas, con su regla

Capa	Archivos	Regla que la define
<code>lib/domain</code>	1	Modelos inmutables. Cero dependencias , ni siquiera de Flutter. Eso permite que <code>test/curriculum_integrity_test.dart</code> construya el currículo leyendo el archivo sin levantar un entorno de widgets.
<code>lib/data</code>	2	Acceso a fuentes externas. <code>CurriculumRepository</code> solo lee; <code>ProgressRepository</code> lee y escribe. Ninguno decide nada.

<code>lib/state</code>	1	Única fuente de verdad en memoria. Contiene todas las reglas de negocio: qué clase es la próxima, qué avance es válido, en qué orden se persiste.
<code>lib/screens</code> · <code>lib/widgets</code>	6 + 2	Dibujan y disparan. No calculan progreso ni deciden validez.
<code>lib/core</code>	1	Paleta y tipografía. Sin lógica.

Las cinco decisiones que hay que entender

1. El contenido es dato, no código

Las 24 clases viven en un JSON de 43 165 bytes dentro de `assets/`. La consecuencia práctica: cambiar una clase no recompila lógica y puede validarse con Node, sin Flutter. La consecuencia menos evidente: **no hay esquema JSON formal ni mecanismo de migración**. Si el formato cambia, lo único que avisa son los dos validadores y las tres pruebas de integridad. [../ARCHITECTURE.md](#) reconoce ese hueco y lo deja como trabajo futuro.

2. El estado se inyecta a mano, desde `main`

```
final state = AppState(
  curriculumRepository: const CurriculumRepository(),
  progressRepository: ProgressRepository(preferences),
);
await state.load();
runApp(PañueloAlVientoApp(state: state));
```

No hay provider, riverpod, `get_it` ni `InheritedWidget` propio. `HomeShell` se suscribe con un `AnimatedBuilder` sobre el `ChangeNotifier` y reconstruye las cuatro pestañas juntas.

Coste declarado: `AppState` viaja como parámetro por cinco constructores. Con quince archivos eso es legible; a partir de cierto tamaño dejaría de serlo. INFERENCIA: es el punto donde la arquitectura tendría que cambiar primero si el producto crece.

Beneficio: las pruebas de widget construyen la aplicación con un currículum y un avance conocidos sin tocar ningún contenedor global, y no hay estado residual entre pruebas.

3. La persistencia se confirma antes de publicarse

El orden es siempre el mismo y está en `AppState.completeLesson`:

```
final updated = {..._completedLessonIds, lessonId};
await _progressRepository.writeCompletedLessonIds(updated); // 1. confirmar
_completedLessonIds = updated; // 2. publicar
notifyListeners(); // 3. avisar
```

`ProgressRepository` lanza `StateError` si `SharedPreferences` devuelve `false`. Si lanzara, el `await` propaga y las líneas 2 y 3 no se ejecutan: la interfaz nunca muestra una clase completada que el disco no guardó.

Esta es la regla más protegida del proyecto. La interfaz separada `ProgressStore` existe únicamente para poder inyectar un almacén que falle a voluntad y probarlo (`test/app_state_test.dart`).

4. El pulso se agenda contra un reloj monotónico

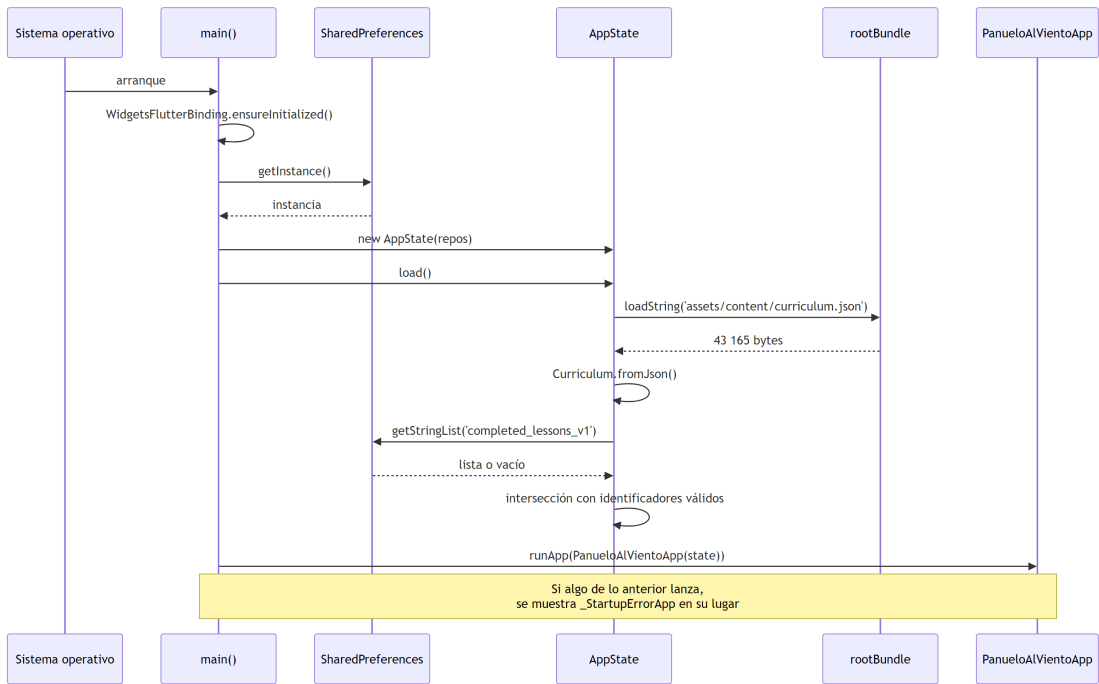
RhythmLabScreen no usa `Timer.periodic`. Calcula el instante objetivo de cada pulso como `microsPerPulse * (tick + 1)` sobre un `Stopwatch` y programa un temporizador de un disparo con el tiempo que falte. Un retraso puntual acorta el intervalo siguiente en vez de sumarse. El detalle línea a línea está en [06 · Explicación profunda](#).

5. La verificación se separa en dos planos

Plano	Herramienta	Qué puede detectar que el otro no
Repositorio	<code>tool/validate_repository.mjs</code>	Que el README prometa una versión que el manifiesto no declara.
Binario	<code>tool/verify_apk.mjs</code>	Que el APK se instale sin currículo dentro, o con permisos que una dependencia añadió al fusionar manifiestos.

`../ARCHITECTURE.md` lo resume mejor de lo que lo haría una paráfrasis: «un repositorio impecable puede producir un artefacto equivocado, y esa es exactamente la clase de fallo que un build en verde no detecta».

Ciclo de vida de la aplicación



El diagrama muestra la única secuencia asíncrona del arranque. No muestra el caso de error con detalle: cualquier excepción entre `getInstance()` y `load()` se captura en `main`, se reporta con `FlutterError.reportError` y se sustituye la aplicación completa por `_StartupErrorApp`, una pantalla sin reintento.

Navegación

Dos mecanismos distintos, y la diferencia importa:

- **Entre las cuatro secciones:** no hay navegación. Un `IndexedStack` mantiene las cuatro vivas y cambia cuál se pinta. Conserva el desplazamiento de cada una y evita reconstruir listas.
- **Hacia una clase:** `Navigator.push` con `MaterialPageRoute`, desde `HomeTab` y desde `RouteTab`. `LessonScreen` es la única pantalla apilada.

La elección del `IndexedStack` tiene una consecuencia que ningún documento del repositorio recogía y que aquí se documenta por primera vez: **el estado de las pestañas no visibles sigue vivo**. Un pulso iniciado en el laboratorio continúa sonando mientras se navega por las otras tres secciones. Está registrado como hallazgo en [15 · Riesgos](#) porque contradice el paso 6 de [../PERMISSIONS.md](#).

Adaptación a la pantalla

Un solo umbral, en HomeShell: `constraints.maxWidth >= 840`. Por encima, `NavigationRail` lateral; por debajo, `NavigationBar` inferior. Se mide el ancho disponible, no la plataforma, así que una ventana de escritorio estrecha se comporta como un teléfono. Cada pantalla añade además su propio `ConstrainedBox`: 1040 px en Inicio, 920 en Ruta, 860 en Clase y 820 en Ritmo y Avance, para que el texto no se estire en un monitor ancho.

Lo que la arquitectura hace bien y conviene no romper

- **La dirección de las dependencias.** `domain` sin importaciones es lo que permite validar el currículum sin Flutter. Meter un `import 'package:flutter/...'` ahí rompería tres pruebas y un validador a la vez.
- **El orden confirmar-luego-publicar.** Invertirlo produce el fallo más caro posible en este producto: avance que se ve y desaparece.
- **La única fuente de verdad de la versión.** `tool/app_version.mjs` lee `pubspec.yaml` y todo lo demás lo consume. El CHANGELOG documenta que antes no era así y que un tag `v0.2.0` habría publicado archivos llamados `0.1.0`.
- **La ausencia de dependencias.** Una sola dependencia de ejecución (`shared_preferences`) y 52 paquetes resueltos en total. Cada plugin nuevo es una vía por la que un permiso puede entrar en el manifiesto fusionado sin aparecer en el código.

Continuar por

- [04 · Mapa del código](#) para el inventario archivo a archivo.
- [07 · Persistencia](#) para el mecanismo de guardado.
- [08 · Flujo de datos](#) para el recorrido completo de un dato.

Contenido de este documento

Árbol versionado

lib/ — el producto

assets/

test/ — 25 pruebas en 6 archivos

tool/ — 11 herramientas

.github/workflows/

packaging/windows/

site/

Continuar por

04 · Mapa del código

Inventario jerárquico de todo lo versionado. Cada elemento lleva ubicación, responsabilidad, dependencias, quién lo usa y **estado aparente**.

Estados usados: **activo** (participa en el producto o en sus compuertas), **duplicado** (existe otro que hace lo mismo), **auxiliar** (herramienta que no entra en el binario), **no determinado**.

Árbol versionado

```
panuelo-al-viento-cueca-app/
■■■■ .github/workflows/  3 workflows
■■■■ assets/
■ ■■■■ branding/        logo.svg · app-icon.png
■ ■■■■ content/         curriculum.json (43 165 bytes)
■■■■ docs/              18 documentos + releases/ + screenshots/ + system-documentation/
■■■■ lib/                15 archivos Dart · 2 417 líneas
■■■■ packaging/windows/  installer.iss · product.wxs
■■■■ site/               index.html (317 líneas)
■■■■ test/               6 archivos · 25 pruebas
■■■■ tool/               11 herramientas
■■■■ pubspec.yaml · pubspec.lock · analysis_options.yaml
■■■■ .gitignore · .gitattributes · .editorconfig
■■■■ README · CHANGELOG · ROADMAP · SECURITY · CONTRIBUTING · CODE_OF_CONDUCT · LICENSE
```

MASTER_PROMPT.md existe en el directorio de trabajo pero **no está versionado**: .gitignore lo excluye con el comentario «Documento de trabajo local; no forma parte del producto publicado». No se le hace referencia desde ningún archivo que sí lo esté, salvo CONTRIBUTING.md, que no lo menciona. **NO DOCUMENTADO EN EL REPOSITORIO**: su existencia no se explica en ninguna parte pública.

lib/ — el producto

lib/main.dart · 89 líneas · activo

Aspecto	Detalle
Responsabilidad	Punto de entrada. Resuelve dependencias en orden y decide si arranca la aplicación o la pantalla de recuperación.
Depende de	shared_preferences, app.dart, ambos repositorios, AppState.
Lo usa	El motor de Flutter.
Símbolos	main(), _StartupErrorApp (privado).
No obvio	El try envuelve todo el arranque asíncrono. Un activo corrompido o un almacenamiento inaccesible producen una pantalla explicativa en vez de una en blanco. _StartupErrorApp no ofrece reintento a propósito.

lib/app.dart · 32 líneas · activo

Aspecto	Detalle
Responsabilidad	MaterialApp raíz: título, temas y pantalla inicial.
Depende de	AppTheme, HomeShell, AppState.
Lo usa	main() y test/app_smoke_test.dart.
No obvio	themeMode: ThemeMode.system sin selector propio. La preferencia de contraste ya está en el dispositivo; añadir un control sería una decisión más que explicar a una niña o un niño.

lib/core/app_theme.dart · 138 líneas · activo

Aspecto	Detalle
Responsabilidad	AppColors (8 constantes) y AppTheme (light, dark, _base, _textTheme).
Depende de	Solo Material.
Lo usa	app.dart, home_tab.dart y movement_diagram.dart (estos dos por AppColors).
No obvio	Los roles *Container se declaran a mano en los dos modos. El propio archivo explica por qué: ColorScheme.fromSeed no regenera los contenedores cuando se sobrescribe primary, secondary o tertiary, y el resultado eran tarjetas ocre sobre fondo azulado en modo oscuro. AppColors.redDeep existe porque el rojo de marca sobre blanco solo alcanza 4,2:1.

Este archivo ya tenía comentarios de calidad antes de este trabajo y **no se tocó**: añadir explicación a un código bien explicado lo empeora.

[lib/domain/curriculum.dart](#) · 193 líneas · activo

Aspecto	Detalle
Responsabilidad	Los cuatro modelos inmutables y la conversión desde JSON.
Depende de	Nada. Ni del proyecto ni de Flutter.
Lo usa	<code>data</code> , <code>state</code> , cuatro pantallas, dos widgets y tres archivos de prueba.
Símbolos	<code>ActivityType</code> (enum, 6 valores), <code>activityTypeFromJson</code> , <code>LearningActivity</code> , <code>Lesson</code> , <code>LearningLevel</code> , <code>Curriculum</code> .
No obvio	<code>activityTypeFromJson</code> degrada a <code>discover</code> ante un valor desconocido en vez de lanzar; el resto de las fábricas no toleran campos ausentes. La asimetría es deliberada: <code>type</code> no cambia lo que se muestra.

[lib/data/curriculum_repository.dart](#) · 27 líneas · activo

Lee `assets/content/curriculum.json` con `rootBundle` y devuelve un `Curriculum`. Constructor `const`, sin estado. No captura errores: la excepción sube hasta `main`. Lo usan `AppState` y tres archivos de prueba.

[lib/data/progress_repository.dart](#) · 68 líneas · activo

Aspecto	Detalle
Responsabilidad	<code>ProgressStore</code> (interfaz) y <code>ProgressRepository</code> (implementación sobre <code>SharedPreferences</code>).
Depende de	<code>shared_preferences</code> .
Lo usa	<code>main.dart</code> , <code>AppState</code> , <code>test/progress_repository_test.dart</code> , <code>test/app_smoke_test.dart</code> .
No obvio	La interfaz existe solo para poder inyectar un almacén que falle en las pruebas. Ambos métodos de escritura lanzan <code>StateError</code> si la operación no se confirma: devolver sin más sería indistinguible del éxito. Escribe siempre ordenado, para que dos ejecuciones con el mismo avance produzcan el mismo valor.

[lib/state/app_state.dart](#) · 126 líneas · activo

Aspecto	Detalle
Responsabilidad	Única fuente de verdad en memoria y todas las reglas de negocio.
Depende de	Ambos repositorios y el dominio.

Lo usa	<code>main.dart</code> , <code>app.dart</code> , <code>home_shell.dart</code> y las cuatro pantallas principales, más <code>lesson_screen.dart</code> .
Símbolos	<code>curriculum</code> , <code>completedLessonIds</code> , <code>completedCount</code> , <code>totalCount</code> , <code>progress</code> , <code>nextLesson</code> , <code>load</code> , <code>isCompleted</code> , <code>levelProgress</code> , <code>completeLesson</code> , <code>resetProgress</code> .
No obvio	<code>completedLessonIds</code> devuelve <code>Set.unmodifiable</code> : nadie fuera puede alterar el avance sin pasar por los métodos que persisten. <code>load()</code> cruza lo guardado con los identificadores válidos y descarta el resto sin reescribir el almacén .

lib/screens/ — cinco pantallas

Archivo	Líneas	Estado	Responsabilidad	Símbolos privados
<code>home_shell.dart</code>	143	activo	Contenedor, navegación adaptable y <code>IndexedStack</code> .	<code>_HomeShellState</code>
<code>home_tab.dart</code>	299	activo	Bienvenida, progreso, próxima clase y dos accesos.	<code>_ProgressCard</code> , <code>_ActionCard</code> , <code>_FinishedCard</code>
<code>route_tab.dart</code>	121	activo	Los ocho niveles desplegables con sus clases.	<code>_LevelProgress</code>
<code>lesson_screen.dart</code>	299	activo	Detalle de una clase y su cierre.	<code>_LessonScreenState</code> , <code>_ActivityTile</code> , <code>_InfoCard</code>
<code>rhythm_lab_screen.dart</code>	356	activo	Metrónomo de seis pulsos.	<code>_PulsePattern</code> , <code>_RhythmLabScreenState</code>
<code>progress_tab.dart</code>	256	activo	Avance, privacidad visible y reinicio.	<code>_CapabilityLine</code> , <code>_HeroProgress</code>

`rhythm_lab_screen.dart` es la única pantalla que **no recibe `AppState`**: su constructor es `const` y no comparte nada. Es también la única con temporizador.

lib/widgets/ — dos componentes

Archivo	Líneas	Estado	Nota
<code>lesson_card.dart</code>	78	activo	Fila de clase, reutilizada por Inicio y Ruta. Comunica el estado completado con icono y color, nunca solo con color.
<code>movement_diagram.dart</code>	192	activo	Esquema de piso dibujado con <code>CustomPaint</code> . <code>MovementDiagram</code> público, <code>_MovementPainter</code> privado.

`MovementPainter` tiene ramas para `circle`, `eight`, `semicircle`, `steps` y `wave`, más una rama `default`. El currículo usa **siete** valores de `diagram`: esos cinco más `pair` y `free`, que caen ambos en `default`. La descripción semántica de `MovementDiagram._description` sí distingue `pair`, pero no `free`. Registrado en [15 · Riesgos](#).

assets/

Ruta	Contenido	Estado
<code>assets/content/curriculum.json</code>	8 niveles, 24 clases, 72 actividades. Campo <code>version</code> : "1.0.0", independiente de la versión del producto.	activo
<code>assets/branding/logo.svg</code>	776 bytes. Se copia a la landing como <code>logo.svg</code> .	activo
<code>assets/branding/app-icon.png</code>	41 700 bytes. Fuente de <code>flutter_launcher_icons</code> para Android y Windows.	activo

`pubspec.yaml` declara `assets/branding/` como directorio completo, así que cualquier archivo que se añada ahí entra en el binario aunque nadie lo use.

test/ — 25 pruebas en 6 archivos

Archivo	Pruebas	Qué protege
<code>app_smoke_test.dart</code>	1	Que las cuatro secciones abran y que los textos de privacidad estén presentes.
<code>app_state_test.dart</code>	3	Las tres reglas transaccionales. Es la prueba más valiosa del repositorio.
<code>curriculum_integrity_test.dart</code>	3	8 niveles, 24 clases, órdenes continuos, tres actividades que suman la duración, identificadores únicos.
<code>curriculum_model_test.dart</code>	1	Que <code>Lesson.fromJson</code> construya un modelo correcto.
<code>progress_repository_test.dart</code>	1	Escritura, lectura y borrado sobre <code>SharedPreferences</code> simulado.
<code>theme_contrast_test.dart</code>	16	Contraste WCAG de cada par de color realmente dibujado, en los dos modos.

`curriculum_integrity_test.dart` importa `dart:io` para leer el archivo del repositorio en vez del activo empaquetado. Es legítimo en una prueba y `tool/validate_repository.mjs` no lo alcanza porque solo inspecciona archivos de `lib/`.

tool/ — 11 herramientas

Archivo	Líneas	Estado	Qué hace	Lo invoca
---------	--------	--------	----------	-----------

app_version.mjs	32	activo	Única fuente de verdad de la versión. Exporta <code>readAppVersion()</code> y funciona también como CLI.	validate_repository, verify_apk, build_site, release.yml
validate_repository.mjs	167	activo	32 archivos esenciales, coherencia de versión, capturas, enlaces de las notas, versión en la landing y cadenas prohibidas en el código.	CI, release, bootstrap
validate_curriculum.mjs	62	activo	Estructura del currículo.	CI, release, bootstrap
validate_curriculum.dart	106	duplicado	Lo mismo que el anterior, en Dart. No lo ejecuta ningún workflow ni bootstrap; solo lo mencionan CONTRIBUTING.md y el MASTER_PROMPT.md no versionado.	Manual
verify_apk.mjs	195	activo	Abre un APK con aapt2, comprueba identidad, versión, API, permisos, ABIs y compara el currículo empaquetado byte a byte.	release.yml, manual
build_site.mjs	103	activo	Ensambla la landing sustituyendo <code>__APP_VERSION__</code> . Puede servirla con <code>--serve</code> .	CI, release, pages.yml
capture_screenshots.mjs	312	auxiliar	Compila para web, sirve en localhost y conduce Chrome sin ventana por el protocolo DevTools para rehacer las 9 capturas.	Manual
configure_platforms.mjs	71	activo	Nombre público, <code>minSdk = 24</code> , metadatos de Windows y rechazo de permisos de cámara y micrófono.	Bootstrap y ambos jobs de compilación

<code>configure_android_signing.mjs</code>	42	activo	Inserta <code>signingConfigs</code> en el Gradle generado leyendo <code>key.properties</code> .	<code>release.yml</code>
<code>bootstrap.ps1</code>	18	activo	Preparación completa en Windows. Aborta al primer fallo.	Manual
<code>bootstrap.sh</code>	18	activo	Igual, para Linux y macOS.	Manual
<code>build_docs_pdf.py</code>	—	auxiliar	Genera los PDF de esta carpeta. Añadido por este trabajo.	Manual

[.github/workflows/](#)

Workflow	Disparo	Jobs	Estado
<code>ci.yml</code>	Push y PR a <code>main</code> , manual	1 (<code>quality</code>)	activo
<code>pages.yml</code>	Push a <code>main</code> que toque <code>site/</code> , capturas, marca, <code>pubspec.yaml</code> o los dos scripts implicados; manual	1 (<code>deploy</code>)	activo
<code>release.yml</code>	Tag <code>v*</code> , manual con tag	5 (<code>version</code> → <code>quality</code> → <code>android+windows</code> → <code>publish</code>)	activo

[packaging/windows/](#)

Archivo	Herramienta	Estado
<code>installer.iss</code>	Inno Setup 6. AppId fijo, español, <code>x64compatible</code> , requiere administrador.	activo
<code>product.wxs</code>	WiX 3. Mismo GUID como <code>UpgradeCode</code> , idioma 1034, <code>perMachine</code> , acceso directo en el menú de inicio.	activo

Ambos comparten el identificador `7C81FC5C-67A6-4F3A-A519-78019866B239`, lo que **INFERENCIA** sugiere que EXE y MSI se consideran el mismo producto para efectos de actualización. Ningún documento lo declara.

[site/](#)

`index.html`, 317 líneas, autocontenido salvo por `logo.svg` y las capturas, que `build_site.mjs` copia al ensamblar. Usa `__APP_VERSION__` en 5 lugares. La galería referencia 8 de las 9 capturas: `09-escritorio.png` se copia pero nunca se muestra.

Continuar por

- 05 · Referencia técnica para las firmas.
- 06 · Explicación profunda para el flujo interno.

Contenido de este documento

lib/domain/curriculum.dart

lib/data/curriculum_repository.dart

lib/data/progress_repository.dart

lib/state/app_state.dart

lib/screens/rhythm_lab_screen.dart

lib/screens/lesson_screen.dart

lib/widgets/movement_diagram.dart

Herramientas · tool/app_version.mjs

Continuar por

05 · Referencia técnica

Catálogo de consulta. Para cada símbolo relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién lo llama, a quién llama y **riesgo al modificarlo**.

Los identificadores se citan en su forma original.

lib/domain/curriculum.dart

enum ActivityType

```
enum ActivityType { discover, observe, move, listen, create, reflect }
```

Naturaleza pedagógica de una actividad. **La interfaz no ramifica por este valor**: hoy solo documenta la intención con la que el currículo equilibra la secuencia. Reparto real en las 72 actividades: `move` 22, `reflect` 18, `observe` 16, `create` 7, `discover` 5, `listen` 4.

Riesgo al modificar: bajo hoy. Añadir un valor no rompe nada porque no hay switch exhaustivo sobre él. Pasaría a ser alto en cuanto alguien lo use para decidir un icono o un color.

ActivityType activityTypeFromJson(String value)

Aspecto	Detalle
Parámetro	<code>value</code> — el campo <code>type</code> del JSON.
Retorno	El valor coincidente, o <code>ActivityType.discover</code> si no hay ninguno.
Excepciones	Ninguna.

Llamado por	<code>LearningActivity.fromJson</code> .
-------------	--

Degrada en silencio. Es la única tolerancia del dominio, y es deliberada: un `type` mal escrito no debe impedir que una clase se abra. El control estricto vive en los validadores.

Riesgo: hacerlo lanzar convertiría una errata de contenido en un fallo de arranque de toda la aplicación.

`class LearningActivity`

Campos final: `type`, `title`, `instruction`, `minutes`. Constructor `const`.

factory `LearningActivity.fromJson(Map<String, dynamic> json)` — lanza `TypeError` si falta un campo o el tipo no coincide. Los validadores comprueban antes que los minutos de las tres actividades sumen el `durationMinutes` de su clase.

`class Lesson`

Campos final: `id`, `order`, `title`, `durationMinutes`, `objective`, `why`, `diagram`, `activities`, `challenge`, `safety`, `accessibility`, `teacherTip`.

`order` es la posición **global** entre las 24, no dentro del nivel: es el número de la tarjeta y el que decide cuál es la próxima clase.

`diagram` es una clave que interpreta `MovementDiagram`. Valores presentes en 0.1.0 y su frecuencia: `circle 4`, `eight 4`, `pair 4`, `steps 4`, `free 3`, `semicircle 3`, `wave 2`.

`activities` se construye con `.toList(growable: false)`: no se puede añadir ni quitar después.

Riesgo al añadir un campo: alto en cadena. Hay que tocar `fromJson`, el JSON de las 24 clases, los dos validadores y probablemente `curriculum_model_test.dart`. Un campo nuevo obligatorio hace fallar el arranque hasta que las 24 lo tengan.

`class LearningLevel`

Campos: `id`, `order`, `title`, `subtitle`, `emoji`, `lessons`. Los validadores exigen exactamente 3 clases por nivel.

`class Curriculum`

Campos: `version`, `title`, `audience`, `levels`.

`List<Lesson> get lessons`

```
List<Lesson> get lessons =>
  levels.expand((level) => level.lessons).toList(growable: false)
  ..sort((a, b) => a.order.compareTo(b.order));
```

Aplana y ordena por `order`. **Construye y ordena una lista nueva en cada llamada**. Con 24 elementos es irrelevante, pero conviene saberlo: lo invocan `AppState.totalCount`, `AppState.nextLesson` y `AppState.completeLesson`, y este último recorre además el resultado para construir un `Set`.

Riesgo: usarlo dentro de un `build` que se repinte a 60 fps. Hoy no ocurre.

`lib/data/curriculum_repository.dart`

`Future<Curriculum> load()`

Aspecto	Detalle
---------	---------

Retorno	El currículo completo.
Excepciones	<code>FlutterError</code> si el activo no está en el bundle; <code>FormatException</code> si el JSON es inválido; <code>TypeError</code> si falta un campo. Ninguna se captura aquí.
Efectos	Lectura del bundle de activos. Ninguna escritura.
Llamado por	<code>AppState.load()</code> y tres archivos de prueba.
Llama a	<code>rootBundle.loadString</code> , <code>jsonDecode</code> , <code>Curriculum.fromJson</code> .

Usa `rootBundle` y no el sistema de archivos deliberadamente: el contenido viaja dentro de la instalación y se lee igual en Android, en Windows y en las pruebas. Esa elección es también lo que mantiene al producto sin acceso general a archivos del dispositivo.

Riesgo: capturar la excepción aquí destruiría la pantalla de recuperación de `main`, que es lo que evita que una niña o un niño vea una pantalla en blanco.

`lib/data/progress_repository.dart`

abstract interface class ProgressStore

```
Set<String> readCompletedLessonIds();
Future<void> writeCompletedLessonIds(Set<String> lessonIds);
Future<void> clear();
```

Contrato del almacenamiento. **Regla:** los dos métodos asíncronos *lanzan* cuando la operación no se confirma. Devolver sin más sería indistinguible del éxito.

Existe únicamente para poder inyectar un almacén que falle a voluntad. Sin esta interfaz, las tres pruebas de `test/app_state_test.dart` serían imposibles de escribir, y con ellas se perdería la única red de seguridad de la regla transaccional.

`ProgressRepository(SharedPreferences preferences)`

Clave: `static const _completedKey = 'completed_lessons_v1'`. El sufijo `_v1` reserva la posibilidad de migrar el formato. Cambiarla deja huérfano el avance guardado en los dispositivos ya instalados.

`Set<String> readCompletedLessonIds()`

Síncrono. Devuelve conjunto vacío si la clave no existe. No lanza.

`Future<void> writeCompletedLessonIds(Set<String> lessonIds)`

Ordena, escribe y lanza `StateError('No fue posible guardar el avance en este dispositivo.')` si `setStringList` devuelve `false`.

El ordenamiento hace comparables dos ejecuciones con el mismo avance, lo que `test/progress_repository_test.dart` aprovecha.

Riesgo: eliminar el `throw` rompe silenciosamente toda la garantía transaccional. `AppState` no tiene forma de saber que la escritura falló.

`Future<void> clear()`

Lanza `StateError`('No fue posible borrar el avance de este dispositivo.') si `remove` devuelve `false`.

REQUIERE VALIDACIÓN: `SharedPreferences.remove` sobre una clave inexistente devuelve `true` en las implementaciones habituales, pero el comportamiento no está documentado en el repositorio ni se probó en un dispositivo real. Si devolviera `false`, reiniciar un avance ya vacío lanzaría.

`lib/state/app_state.dart`

`AppState({required CurriculumRepository curriculumRepository, required ProgressStore progressRepository})`

Extiende `ChangeNotifier`. Nótese la asimetría de los tipos: el repositorio de currículo se pide **concreto** y el de progreso se pide por **interfaz**. Solo el segundo necesita ser sustituible, porque solo el segundo puede fallar de una forma que haya que probar.

Curriculum `get curriculum`

Lanza `StateError`('El currículo todavía no está cargado.') si se consulta antes de `load()`. Intencionado: un `null` silencioso se propagaría a toda la interfaz.

`Set<String> get completedLessonIds`

Devuelve `Set.unmodifiable(...)`. Nadie fuera de la clase puede alterar el avance sin pasar por los métodos que persisten. Propiedad pequeña y valiosa.

`int get completedCount` · `int get totalCount` · `double get progress`

`progress` protege la división: `totalCount == 0 ? 0 : completedCount / totalCount`. `totalCount` invoca `curriculum.lessons`, que ordena.

Lesson? `get nextLesson`

Primera clase no completada en orden global, o `null`. Es un avance **por hueco**, no por posición: completar la 5 antes que la 3 deja la 3 como próxima. La ruta no bloquea clases, así que este valor es una sugerencia, no una puerta.

Llamado por `HomeTab.build`.

`Future<void> load()`

Aspecto	Detalle
Efectos	Asigna <code>_curriculum</code> , filtra <code>_completedLessonIds</code> , <code>notifyListeners()</code> .
Excepciones	Propaga las de <code>CurriculumRepository.load()</code> .
Llamado por	<code>main()</code> y cinco pruebas.

La línea que hay que entender:

```
_completedLessonIds =  
  _progressRepository.readCompletedLessonIds().intersection(validIds);
```

Cruza lo guardado con los identificadores que el currículo actual declara válidos. Una clase retirada o renombrada en una versión posterior deja de contar como completada y no infla el porcentaje. El descarte es **silencioso y no**

reescribe el almacén: los identificadores desconocidos siguen en disco hasta la próxima escritura, momento en el que desaparecen. NO DOCUMENTADO EN EL REPOSITORIO fuera de una frase en `../ARCHITECTURE.md`.

`bool isCompleted(String lessonId)` · `double levelProgress(LearningLevel level)`

Consultas puras. `levelProgress` devuelve 0 para un nivel sin clases, caso que el currículo actual no produce porque los validadores exigen 3.

`Future<void> completeLesson(String lessonId)`

Aspecto	Detalle
Excepciones	<code>ArgumentError</code> si el identificador no existe en el currículo. <code>StateError</code> propagado si la escritura no se confirma.
Efectos	Escribe en <code>SharedPreferences</code> , actualiza el conjunto, notifica. En ese orden.
Llamado por	<code>LessonScreen._complete()</code> y dos pruebas.

Si la clase ya estaba completada, retorna sin escribir: repetir es parte del diseño pedagógico y no debe generar tráfico ni notificaciones.

Riesgo al modificar: **el más alto del repositorio.** Publicar `_completedLessonIds` antes del `await` deja la interfaz mintiendo cuando el disco falla. Las tres pruebas de `test/app_state_test.dart` existen exactamente para impedir ese cambio.

`Future<void> resetProgress()`

Mismo orden: borrar, luego vaciar en memoria, luego notificar. Si el borrado falla, el avance visible se mantiene. Es preferible a mostrar cero clases y que reaparezcan al reiniciar. Llamado por `ProgressTab._confirmReset` tras confirmación.

`lib/screens/rhythm_lab_screen.dart`

Estado interno de `_RhythmLabScreenState`

Campo	Tipo	Inicial	Papel
<code>_timer</code>	<code>Timer?</code>	<code>null</code>	Temporizador de un disparo del próximo pulso.
<code>_clock</code>	<code>Stopwatch</code>	detenido	Referencia monotónica.
<code>_scheduledTick</code>	<code>int</code>	0	Cuántos pulsos se han programado desde el último reinicio.
<code>_bpm</code>	<code>double</code>	84	Velocidad. El deslizador va de 60 a 120 con 12 divisiones, es decir pasos de 5; 84 no cae en ninguna división.
<code>_pulse</code>	<code>int</code>	0	Pulso actual, 0–5.

<code>_playing</code>	<code>bool</code>	<code>false</code>	Si el metrónomo corre.
<code>_sound</code> · <code>_vibration</code>	<code>bool</code>	<code>true</code>	Interruptores de salida. Encendidos por defecto, pero inertes hasta pulsar Comenzar .
<code>_pattern</code>	<code>_PulsePattern</code>	<code>sixEight</code>	Agrupación.

`List<int> get _accents`

`sixEight` → [0, 3] (3+3), `threeFour` → [0, 2, 4] (2+2+2). Un `switch` de expresión exhaustivo sobre el enum: añadir un valor a `_PulsePattern` **rompe la compilación aquí**, que es el comportamiento deseado.

`void _toggle()`

Si suena: cancela, para y reinicia el reloj, `_playing` = `false`, `_pulse` = 0. Si no: `_playing` = `true`, emite el pulso 0 inmediatamente y arranca la planificación. El primer pulso suena antes de esperar un intervalo, que es lo que hace que Comenzar se sienta inmediato.

`void _startSchedule()`

Cancela el temporizador vigente, pone `_scheduledTick` a 0, reinicia el `Stopwatch` y programa. Se llama también al mover el deslizador mientras suena, para que el cambio de tempo se note de inmediato.

`void _scheduleNextPulse()`

El corazón del laboratorio:

```
final microsPerPulse = 60000000 / _bpm;
final targetMicros = (microsPerPulse * (_scheduledTick + 1)).round();
final remainingMicros = targetMicros - _clock.elapsedMicroseconds;
_timer = Timer(Duration(microseconds: remainingMicros > 0 ? remainingMicros : 0), ...);
```

Cada pulso se agenda contra un **instante absoluto** de la línea temporal, no contra el momento en que se ejecutó el anterior. Un retraso acorta el intervalo siguiente —o lo dispara de inmediato con espera cero— y la serie vuelve a su sitio.

Guardas: retorna si `!_playing` antes de programar; dentro de la devolución de llamada comprueba `!mounted || !_playing` antes de continuar.

Riesgo: sustituirlo por `Timer.periodic` reintroduce una deriva que solo se nota tras varios minutos, que es exactamente cuando importa.

`void _emitPulse()`

Clic en cada pulso si `_sound`; háptica **solo en accents** si `_vibration`, para que la vibración marque la agrupación y no sea un zumbido continuo. Ambas pueden ser ignoradas por el equipo y esa ausencia no se trata como error.

`void dispose()`

Cancela el temporizador y para el reloj. **Solo se ejecuta al destruirse el widget**, y el `IndexedStack` de `HomeShell` no lo destruye al cambiar de pestaña. Consecuencia verificada en pruebas y registrada en [15 · Riesgos](#).

`LessonScreenState._activityChecks`

`List<bool>` de longitud fija igual al número de actividades. **Estado local, no persistente.** Salir de la clase sin completarla lo descarta. Es coherente con el inventario de `../PRIVACY.md`: no se guarda el detalle de la práctica.

`Future<void> _complete()`

`await completeLesson` primero, comprobación de `mounted`, diálogo de celebración, `setState` final. El orden importa: si la escritura falla, la excepción sube y no se llega a felicitar por algo que no quedó guardado.

`void _finishRepeat()`

No toca la persistencia. Limpia las casillas y muestra un `SnackBar` avisando de que el avance anterior sigue guardado, porque desmarcar tres casillas parece un retroceso si nadie lo dice.

`lib/widgets/movement_diagram.dart`

`MovementDiagram({required String pattern})`

Envuelve un `CustomPaint` de proporción 16:9 en un `Semantics` con `image: true` y una etiqueta descriptiva.

`String _description(String value)`

`switch` de expresión con ramas para `circle`, `eight`, `semicircle`, `steps`, `pair`, `wave` y un comodín. **No tiene rama para `free`**, que cae en el comodín: «Espacio personal para practicar un movimiento libre», descripción que resulta correcta por casualidad.

`MovementPainter.paint(Canvas, Size)`

Ramas de dibujo: `circle`, `eight` (paramétrica, 210 segmentos), `semicircle`, `steps`, `wave`, y `default` (una línea entre las dos figuras). `pair` y `free` comparten esa rama por defecto.

Todas las medidas son fracciones de `size`. Los colores oscuros se eligen a mano en lugar de tomarse del `ColorScheme` porque el lienzo tiene fondo propio y debe contrastar con él, no con la tarjeta que lo contiene.

Detalle no obvio: la rama `steps` muta el `Paint` compartido a `PaintingStyle.fill` y lo restaura a `stroke` después; la rama `default` muta `strokeWidth` a 2 y **no lo restaura**. Es inocuo porque el `Paint` es local a cada llamada de `paint`, pero es el tipo de detalle que se vuelve un error al extraer el objeto a un campo.

`bool shouldRepaint(covariant MovementPainter oldDelegate)`

Repinta si cambió `pattern` o `darkMode`. Correcto y ajustado.

Herramientas · `tool/app_version.mjs`

`export function readAppVersion(pubspecPath = 'pubspec.yaml')`

Aspecto	Detalle
Retorno	<code>{ version, build, full }</code> . Para <code>version: 0.1.0+1</code> → <code>{ '0.1.0', '1', '0.1.0+1' }</code> .
Excepciones	Error si no encuentra una línea <code>version: X.Y.Z+N</code> .

Lo usan

`validate_repository.mjs`, `verify_apk.mjs`, `build_site.mjs` y `release.yml`.

También funciona como CLI: `node tool/app_version.mjs version|build|full`. La detección de invocación directa compara `import.meta.url` con `pathToFileURL(process.argv[1])`.

Es el símbolo con más consecuencias del repositorio: todo el sistema de versiones depende de que esta expresión regular siga coincidiendo.

Continuar por

- [06 · Explicación profunda](#) para ver estos símbolos en movimiento.
- [19 · Matriz de trazabilidad](#) para ir de una funcionalidad a su prueba en una sola fila.

Contenido de este documento

Flujo 1 · Arranque

Flujo 2 · Pintar el estado

Flujo 3 · Completar una clase

Flujo 4 · Reiniciar el avance

Flujo 5 · El metrónomo

Flujo 6 · Dibujar un esquema de piso

Flujo 7 · Publicar una versión

Continuar por

06 · Explicación profunda del código

Aquí se explica el sistema **flujo por flujo**, no archivo por archivo. Cada flujo empieza en un gesto de la persona usuaria y termina donde ese gesto deja de producir efectos.

Flujo 1 · Arranque

Empieza: el sistema operativo lanza el proceso. **Termina:** la primera imagen en pantalla.

`main()` es asíncrono y hace cuatro cosas en orden estricto:

```
WidgetsFlutterBinding.ensureInitialized();
final preferences = await SharedPreferences.getInstance();
final state = AppState(curriculumRepository: ..., progressRepository: ...);
await state.load();
runApp(PañueloAlVientoApp(state: state));
```

`ensureInitialized()` es obligatorio antes de tocar cualquier canal de plataforma; sin él, `SharedPreferences.getInstance()` fallaría porque el canal todavía no existe.

Dentro de `state.load()` ocurren dos operaciones distintas:

1. `CurriculumRepository.load()` pide a `rootBundle` los 43 165 bytes del JSON, los decodifica y construye el árbol de 8 niveles → 24 clases → 72 actividades. Cada `fromJson` hace casts explícitos: un campo ausente lanza `TypeError` aquí y no produce un objeto a medias que llegue a la interfaz.
2. Se lee el avance guardado y se **cruza** con los identificadores válidos.

El cruce es la línea que un lector nuevo pasaría por alto:

```
final validIds = curriculum.lessons.map((lesson) => lesson.id).toSet();
_completedLessonIds =
  _progressRepository.readCompletedLessonIds().intersection(validIds);
```

Sin él, retirar o renombrar una clase en una versión futura dejaría en el almacén un identificador que ya no existe, contado para siempre en un porcentaje que nunca podría llegar a 100 %. Con él, ese identificador simplemente desaparece del cálculo. Nótese que **no se reescribe el almacén**: el valor huérfano sigue en disco hasta la próxima escritura.

Toda esta secuencia está envuelta en un `try`. Si cualquier paso lanza:

```
FlutterError.reportError(FlutterErrorDetails(  
  exception: error, stack: stackTrace, library: 'inicio de Pañuelo al Viento');  
runApp(const _StartupErrorApp());
```

`_StartupErrorApp` es una pantalla completa —no un diálogo— con un icono, un título y una instrucción. No ofrece reintento, y es correcto: las causas plausibles (activo corrompido, almacenamiento inaccesible) no se resuelven repitiendo la misma operación en el mismo proceso. El texto evita jerga y evita culpar a quien lee.

Flujo 2 · Pintar el estado

Empieza: `AppState.notifyListeners()`. **Termina:** la pantalla redibujada.

`HomeShell.build` envuelve todo en un `AnimatedBuilder(animation: widget.state)`. Cualquier notificación reconstruye el subárbol completo de las cuatro pestañas.

Dentro, un `LayoutBuilder` decide la forma:

```
final wide = constraints.maxWidth >= 840;
```

Se mide el **ancho disponible**, no la plataforma. Una ventana de escritorio estrecha se comporta como un teléfono, y eso se prueba sin emulador.

Las cuatro pantallas se construyen en una lista y se meten en un `IndexedStack`. Aquí hay una decisión con consecuencias que conviene entender bien.

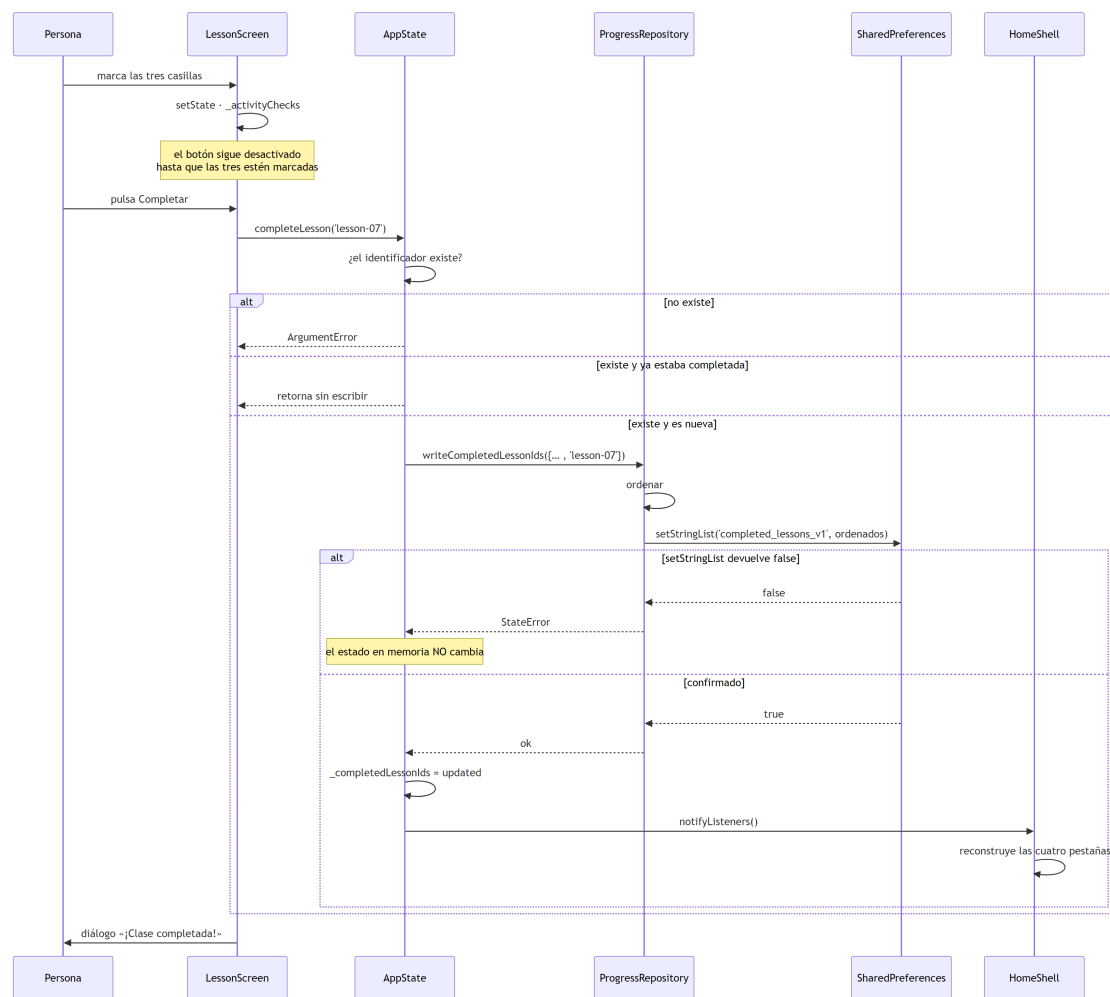
`IndexedStack` **construye y mantiene en el árbol a los cuatro hijos**; solo pinta el que corresponde al índice. La ventaja buscada: cambiar de pestaña conserva la posición de desplazamiento de cada lista y no rehace la Ruta de ocho niveles cada vez.

La consecuencia no buscada: el `State` de las pestañas invisibles sigue vivo. `_RhythmLabScreenState.dispose()` **no se ejecuta** al salir de Ritmo, así que su temporizador sigue programando pulsos y `_emitPulse()` sigue pidiendo clics al sistema mientras la persona está en Inicio, Ruta o Avance.

Esto se comprobó con una prueba temporal que arrancó el pulso, cambió a Inicio y contó las señales emitidas: **se emitieron señales estando en otra pestaña**, y al volver a Ritmo el botón seguía diciendo «Detener». Contradice el paso 6 de [../PERMISSIONS.md](#) («Detener, cambiar de pantalla o cerrar la app cancela el temporizador») y queda registrado en [15 · Riesgos](#) sin corregir.

Flujo 3 · Completar una clase

Empieza: la persona pulsa «Completar esta clase». **Termina:** el porcentaje actualizado en las cuatro pantallas.



El diagrama muestra las tres ramas posibles y, sobre todo, dónde se sitúa el cambio de estado respecto de la confirmación del disco. Lo que no muestra: el diálogo de celebración solo se alcanza si `completeLesson` no lanzó, porque el `await` está antes; y `LessonScreen` hace un `setState` vacío al cerrarlo, para que la barra superior pase a mostrar el chip «Completada».

Tres detalles del código que sostienen el flujo:

El botón sólo se habilita con las tres casillas marcadas.

```

onPressed: _allActivitiesChecked
? completed ? _finishRepeat : _complete
: null,

```

`onPressed: null` es lo que desactiva un botón en Flutter. La etiqueta cambia en paralelo entre cuatro textos según las dos condiciones, de modo que el botón desactivado explica **por qué** lo está («Marca los tres pasos para terminar») en vez de quedarse mudo.

Repetir no escribe. Si la clase ya estaba completada, `_finishRepeat` no toca la persistencia: limpia las casillas y muestra un `SnackBar` recordando que el avance anterior sigue guardado. Sin ese aviso, desmarcar tres casillas parece un retroceso.

El identificador se valida contra el currículum, no contra el almacén. Un `ArgumentError` aquí impide que una errata de contenido contamine el disco.

Flujo 4 · Reiniciar el avance

Empieza: «Reiniciar el avance» en Mi avance. **Termina:** el almacén vacío o el avance intacto.

`ProgressTab._confirmReset` abre un `AlertDialog` que devuelve `bool?`. Solo `confirmed == true` procede: cerrar el diálogo tocando fuera devuelve `null` y no borra nada.

`AppState.resetProgress()` sigue el mismo orden que `completeLesson`: primero `clear()`, que lanza si no se confirmó, y solo después vacía el conjunto en memoria. Si el borrado falla, el avance visible **se mantiene**. Es preferible a mostrar cero clases y que reaparezcan al reiniciar la aplicación.

El borrado es irreversible: no hay copia en la nube ni exportación en 0.1.0. La confirmación es el único freno que existe.

Flujo 5 · El metrónomo

Empieza: «Comenzar». **Termina:** «Detener», o el cierre de la aplicación.

Este es el flujo con más densidad técnica del proyecto, y la razón está en un problema que no se ve hasta que se practica varios minutos.

El problema que resuelve

`Timer.periodic(Duration(milliseconds: 714), ...)` parece la solución obvia para 84 pulsos por minuto. No lo es. Un temporizador periódico vuelve a contar desde el momento en que **se ejecutó** la devolución de llamada anterior. Si el hilo visual estaba ocupado y el tick llegó 30 ms tarde, el siguiente se programa 714 ms después de ese instante ya retrasado. Los errores se suman y no se recuperan nunca: al cabo de unos minutos el metrónomo va notoriamente atrasado respecto a la música con la que se está practicando.

La solución

Tres piezas:

```
final Stopwatch _clock = Stopwatch(); // referencia monotónica
int _scheduledTick = 0;               // cuántos pulsos van programados
Timer? _timer;                       // un disparo, no periódico
```

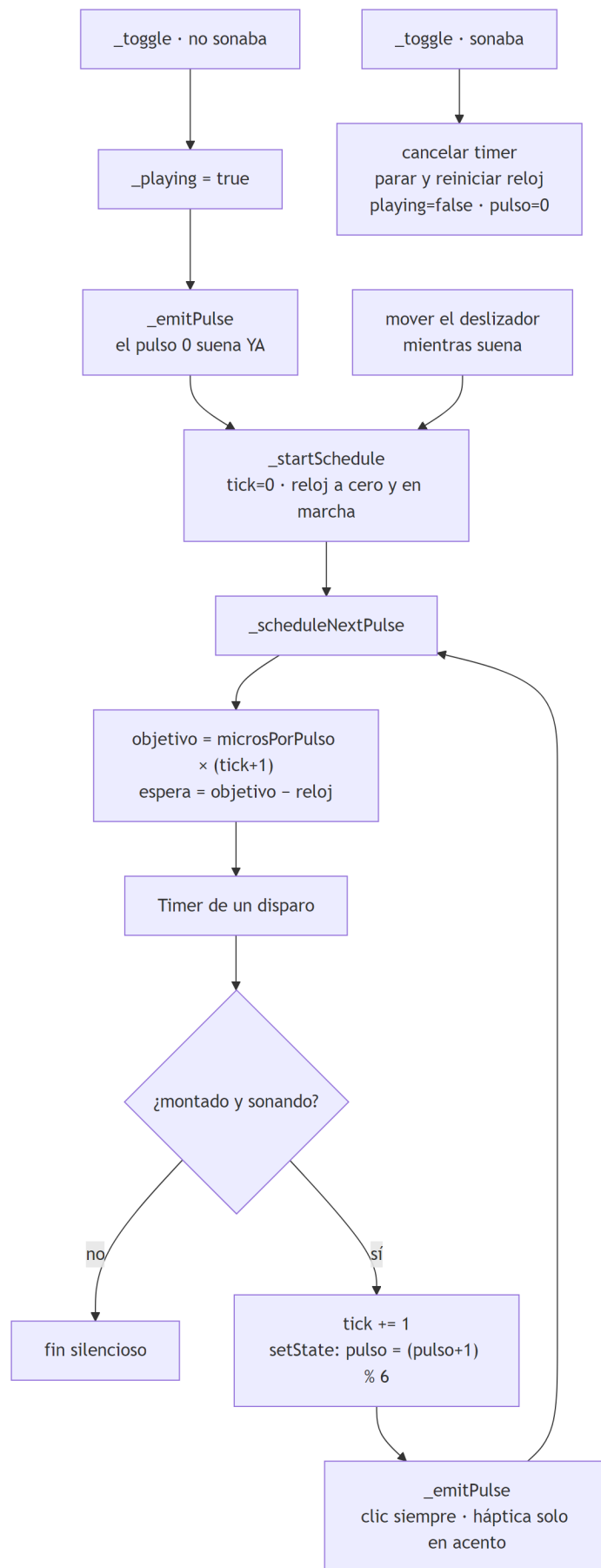
Y el cálculo:

```
final microsPerPulse = 60000000 / _bpm;
final targetMicros = (microsPerPulse * (_scheduledTick + 1)).round();
final remainingMicros = targetMicros - _clock.elapsedMicroseconds;
_timer = Timer(Duration(microseconds: remainingMicros > 0 ? remainingMicros : 0), ...);
```

El objetivo del pulso n es siempre $n \times \text{microsPerPulse}$ **desde el inicio**, no desde el pulso anterior. Si el pulso 10 se ejecutó tarde, `remainingMicros` para el 11 sale más pequeño y el intervalo se acorta solo. Si el retraso fue mayor que un pulso entero, `remainingMicros` sale negativo y el `> 0 ? ... : 0` dispara inmediatamente para recuperar terreno. La serie vuelve a la línea temporal en vez de arrastrar el error.

Se usan microsegundos y no milisegundos porque a 120 PPM el intervalo es 500 000 μs exactos, pero a 84 PPM son 714 285,71 μs : redondear a milisegundos introduciría un error sistemático de hasta medio milisegundo por pulso que, sobre cientos de pulsos, es precisamente la deriva que se intenta evitar.

El ciclo completo



El diagrama muestra el bucle y sus dos salidas. Lo que no muestra: `_emitPulse` se llama **después** de `setState`, de modo que la señal visual se dibuja antes de pedir sonido y vibración; y la rama `SLIDER` reinicia la línea temporal desde cero, para que un cambio de tempo se note de inmediato en vez de esperar a que venza el intervalo ya programado.

Las salidas

```
if (_sound) SystemSound.play(SystemSoundType.click);  
if (_vibration && accent) HapticFeedback.mediumImpact();
```

El clic en cada pulso; la háptica **solo en acentos**, para que la vibración marque la agrupación y no sea un zumbido continuo. Ninguna de las dos lee el entorno: se piden al sistema operativo, que puede ignorarlas. En Windows la háptica normalmente no existe y la ausencia no se trata como error.

Los acentos vienen de un `switch` de expresión exhaustivo:

```
_PulsePattern.sixEight => const [0, 3], // 3+3  
_PulsePattern.threeFour => const [0, 2, 4], // 2+2+2
```

Al ser exhaustivo, añadir un valor a `_PulsePattern` rompe la compilación aquí. Es el comportamiento deseado: obliga a decidir sus acentos.

Flujo 6 · Dibujar un esquema de piso

Empieza: `LessonScreen` construye `MovementDiagram(pattern: lesson.diagram)`. **Termina:** píxeles y una etiqueta semántica.

Dos caminos independientes desde la misma clave:

Camino	Función	Ramas
Visual	<code>_MovementPainter.paint</code>	<code>circle</code> , <code>eight</code> , <code>semicircle</code> , <code>steps</code> , <code>wave</code> , <code>default</code>
Semántico	<code>MovementDiagram._description</code>	<code>circle</code> , <code>eight</code> , <code>semicircle</code> , <code>steps</code> , <code>pair</code> , <code>wave</code> , <code>comodín</code>

Los dos caminos **no cubren el mismo conjunto**. El currículo usa siete valores; el pintor dibuja cinco trazos distintos y manda `pair` y `free` a la rama por defecto —una línea recta entre las dos figuras—, mientras la descripción textual sí distingue `pair` («dos personas dialogando con distancia») y manda `free` al comodín. Resultado: cuatro clases con `pair` y tres con `free` muestran el mismo dibujo, y quien use un lector de pantalla recibe descripciones distintas de un esquema idéntico. Registrado en [15 · Riesgos](#).

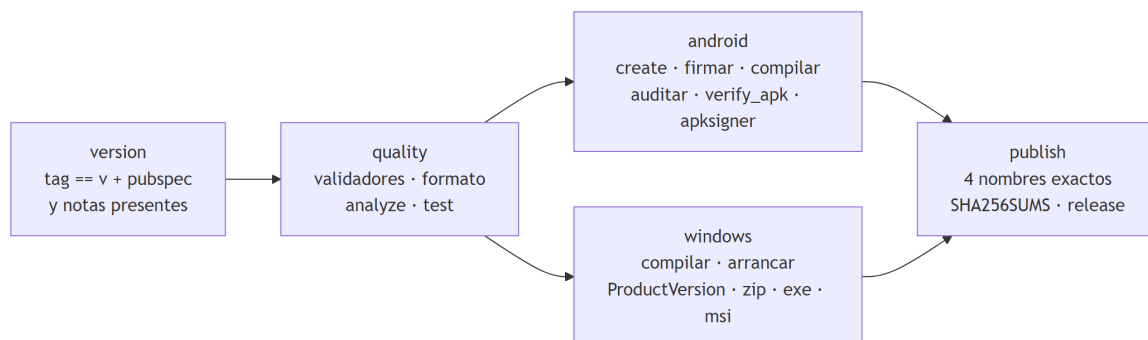
Todas las coordenadas son fracciones de `size`, nunca píxeles: el mismo esquema es legible en un teléfono estrecho y en una ventana de escritorio. `eight` se genera paramétricamente recorriendo t de 0 a 2π en pasos de 0,03 —unos 210 segmentos— con $\sin(t)$ en el eje horizontal y $\sin(2t)$ en el vertical, que es una curva de Lissajous en forma de ocho.

Los colores del lienzo se eligen a mano en vez de tomarse del `ColorScheme`, porque el dibujo pinta su propio fondo y debe contrastar con **él**, no con la tarjeta que lo contiene. Contrapartida: las dos figuras usan `AppColors.red` y `AppColors.blue` fijos en ambos modos, y ese par no lo mide `test/theme_contrast_test.dart`, que solo cubre colores del `ColorScheme`.

Flujo 7 · Publicar una versión

Empieza: un tag `v*`. **Termina:** cuatro archivos y sus hashes en GitHub Releases.

Cinco jobs encadenados en `.github/workflows/release.yml`:



El diagrama muestra el orden y el paralelismo. Lo que no muestra: cada job **rechaza** en vez de avisar. El job `version` aborta si el tag no es `v` + la versión del manifiesto; el job `publish` aborta si sobra o falta cualquiera de los cuatro nombres esperados. Esa severidad es lo que impide publicar archivos con la versión equivocada, un fallo que el CHANGELOG documenta haber tenido antes de existir `tool/app_version.mjs`.

La comprobación que más aporta es `node tool/verify_apk.mjs`: extrae `assets/flutter_assets/assets/content/curriculum.json` del APK, cuenta niveles y clases y compara su SHA-256 con el del archivo del repositorio. Detecta el fallo silencioso por excelencia: un paquete que compila, se firma y cuadra en checksum pero se instala sin contenido.

Continuar por

- [07 · Persistencia](#) para el detalle del almacenamiento.
- [08 · Flujo de datos](#) para el recorrido completo del dato.
- [15 · Riesgos](#) para los hallazgos citados aquí.

Contenido de este documento

No hay base de datos, y se puede demostrar

Los dos almacenes

Almacén 1 · El currículo

Almacén 2 · El avance

Continuar por

07 · Persistencia

No hay base de datos, y se puede demostrar

El sistema **no usa ningún motor de base de datos**: ni SQLite, ni Hive, ni Isar, ni Drift, ni ObjectBox, ni un servidor remoto. No hay migraciones ni esquema SQL.

Comprobación, no opinión:

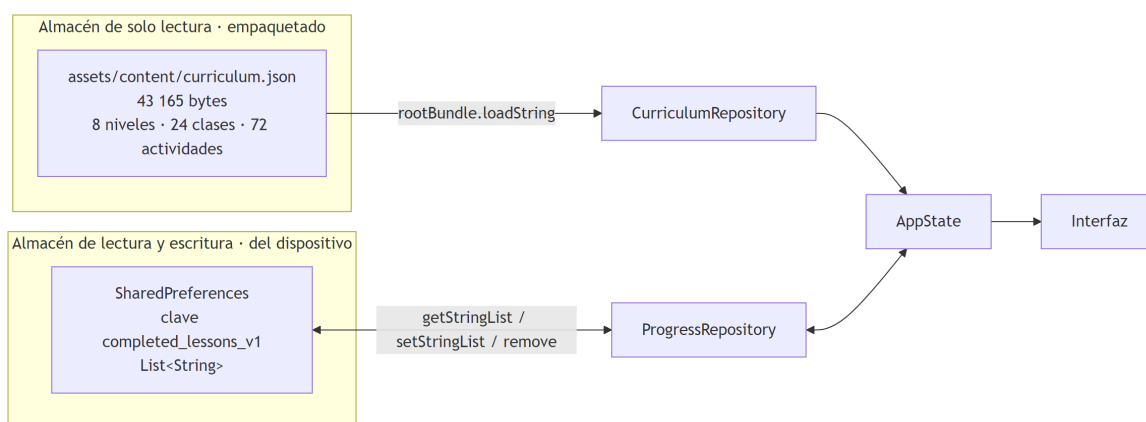
```
$ grep -rniE "sqlite|sqlflite|drift|moor|hive|isar|objectbox|realm|postgres|mysql|mongo" lib/ test/ tool/ pubspec.yaml
# sin resultados

$ grep -c "^ [a-z_]*$" pubspec.lock
52      # 52 paquetes resueltos, ninguno de base de datos
```

La única dependencia de ejecución declarada en pubspec.yaml es `shared_preferences: ^2.5.3`.

Este documento describe, entonces, **el mecanismo de persistencia que sí existe**, que son dos almacenes de naturaleza muy distinta.

Los dos almacenes



El diagrama muestra la separación fundamental: el contenido nunca se escribe y el avance nunca se lee de la red. Lo que no muestra: dónde vive físicamente cada uno, que es lo que se detalla más abajo, ni que el cruce entre ambos ocurre una sola vez, en `AppState.load()`.

Currículo	Avance
Actualiza el empaquetado en el binario	Preferencias del sistema operativo
Acceso a la lectura	Lectura y escritura
Formato UTF-8	Lista de cadenas
Tamaño de 16 bytes fijos	Como máximo 24 cadenas de 9 caracteres
Cada nueva instalación con	Cada clase completada o el reinicio
Sólo se desinstala el binario a	La actualización con la misma clave de firma; no a la desinstalación
MCurriculumRepository que lo usa	ProgressRepository

Almacén 1 · El currículo

Dónde está

En el repositorio, `assets/content/curriculum.json`. Dentro del APK, `assets/flutter_assets/assets/content/curriculum.json` —ruta que `tool/verify_apk.mjs` extrae literalmente para comprobar que el contenido llegó al paquete—. En Windows, dentro de la carpeta `data` que acompaña al ejecutable, razón por la cual el ZIP portable no puede extraerse a medias.

Diccionario de datos

Raíz

Campo	Tipo	Obligatorio	Valor en 0.1.0	Notas
<code>version</code>	string	Sí	"1.0.0"	Versión del currículo , independiente de <code>pubspec.yaml</code> . Ningún validador la comprueba.
<code>title</code>	string	Sí	"Pañuelo al Viento"	No se muestra en la interfaz.
<code>audience</code>	string	Sí	Descripción del público objetivo	No se muestra en la interfaz.
<code>levels</code>	array	Sí	8 elementos	Los validadores exigen exactamente 8.

`title` y `audience` se modelan en `Curriculum` pero ninguna pantalla los lee. NO DOCUMENTADO EN EL REPOSITORIO: por qué se conservan.

`levels[]` → `LearningLevel`

Campo	Tipo	Obligatorio	Restricción validada
id	string	Sí	Único. <code>level-01</code> ... <code>level-08</code>
order	int	Sí	1–8
title	string	Sí	—
subtitle	string	Sí	—
emoji	string	Sí	Se dibuja a 32 px en Ruta y 30 px en Avance
lessons	array	Sí	Exactamente 3

Valores reales:

	order	title	emoji	Clases
level-01	1	Descubro la cueca	■	1–3
level-02	2	Pañuelo y saludo	■	4–6
level-03	3	Vueltas y recorridos	■	7–9
level-04	4	Pasos que conversan	■	10–12
level-05	5	Diálogo en pareja	■	13–15
level-06	6	Zapateo y remate	■	16–18
level-07	7	Música y creación	■	19–21
level-08	8	Cuecas de Chile y presentación	■■	22–24

El título del nivel 8 es Cuecas de Chile y presentación. Tanto `../CURRICULUM.md` como el diagrama del README lo llaman «Diversidad y presentación». Registrado en [15 · Riesgos](#).

`levels[].lessons[]` → `Lesson`

Campo	Tipo	Obligatorio	Restricción validada	Dónde se muestra
id	string	Sí	Único entre las 24. <code>lesson-01</code> ... <code>lesson-24</code>	Nunca visible; es la clave que se persiste
order	int	Sí	1–24 sin saltos ni repetición	Número de la tarjeta y título «Clase N»
title	string	Sí	No vacío	Tarjeta y encabezado

durationMinutes	int	Sí	Igual a la suma de sus actividades; entre 10 y 15 según la prueba	Tarjeta y encabezado
objective	string	Sí	No vacío	Tarjeta y encabezado
why	string	Sí	No vacío	Tarjeta destacada de la clase
diagram	string	Sí	No vacío. Ningún validador comprueba que el valor tenga dibujo	MovementDiagram
activities	array	Sí	Exactamente 3	Las tres casillas
challenge	string	Sí	No vacío	Tarjeta «Reto de la clase»
safety	string	Sí	No vacío	Tarjeta «Baila con cuidado»
accessibility	string	Sí	No vacío	Tarjeta «Otra manera de hacerlo»
teacherTip	string	Sí	No vacío	Tarjeta «Para quien acompaña»

Las 24 clases duran **exactamente 12 minutos**. La prueba de integridad admite el rango 10–15 y varios documentos hablan de «clases de 10 a 15 minutos», pero en los datos no hay ninguna variación.

Valores de `diagram` y su frecuencia:

Valor	Clases	¿Tiene dibujo propio?	¿Tiene descripción propia?
circle	4	Sí	Sí
eight	4	Sí	Sí
steps	4	Sí	Sí
pair	4	No, cae en la rama por defecto	Sí
semicircle	3	Sí	Sí
free	3	No, cae en la rama por defecto	No, cae en el comodín
wave	2	Sí	Sí

`levels[].lessons[].activities[]` → [LearningActivity](#)

Campo	Tipo	Obligatorio	Valores reales
-------	------	-------------	----------------

type	string	Sí	discover 5 · observe 16 · move 22 · listen 4 · create 7 · reflect 18
title	string	Sí	—
instruction	string	Sí	—
minutes	int	Sí	3, 4 o 5

Un type desconocido **no rompe**: `activityTypeFromJson` degrada a `discover`. Los validadores tampoco lo comprueban, porque el valor no cambia lo que se muestra.

Quién lo valida

Herramienta	Comprueba
<code>tool/validate_curriculum.mjs</code>	8 niveles, 24 clases, 3 clases por nivel, identificadores únicos, órdenes 1–24 sin huecos, 3 actividades por clase, suma de minutos, ocho campos de texto no vacíos
<code>tool/validate_curriculum.dart</code>	Lo mismo, en Dart. Duplicado ; CI no lo ejecuta
<code>test/curriculum_integrity_test.dart</code>	Lo mismo más el rango 10–15 minutos, cargándolo por el modelo real
<code>tool/verify_apk.mjs</code>	Que el JSON dentro del APK tenga niveles y clases y su SHA-256 coincida con el del repositorio

Ninguno comprueba que `diagram` sea un valor con dibujo, ni que `type` sea un valor del enum, ni que `version` avance.

Almacén 2 · El avance

Qué se guarda, exactamente

Una única entrada:

Aspecto	Valor
Clave	<code>completed_lessons_v1</code>
Tipo	<code>List<String></code>
Contenido	Identificadores de clases completadas, ordenados alfabéticamente
Ejemplo	<code>["lesson-01", "lesson-02", "lesson-05"]</code>
Máximo	24 cadenas de 9 caracteres

Y nada más. No hay marcas de tiempo, ni puntuaciones, ni tiempo de práctica, ni casillas marcadas, ni velocidad preferida del metrónomo, ni preferencia de sonido o vibración. Las casillas de una clase a medias y los interruptores del laboratorio son estado en memoria y se pierden al cerrar.

Ese mínimo no es una carencia: es el inventario declarado en [../PRIVACY.md](#), que sigue siendo la fuente autorizada de qué datos infantiles trata el producto y por qué.

Dónde vive físicamente

`shared_preferences` delega en el mecanismo nativo de cada plataforma:

Plataforma	Mecanismo	Consecuencia práctica
Android	<code>SharedPreferences</code> del sistema, en el almacenamiento privado de la app	Se borra al desinstalar o al limpiar datos. Sobrevive a una actualización solo si la clave de firma es la misma
Windows	Perfil del usuario, fuera de la carpeta de instalación	Por eso la versión portable también recuerda el avance; reinstalar con EXE o MSI normalmente lo conserva

REQUIERE VALIDACIÓN: las rutas exactas no están declaradas en el repositorio y no se comprobaron en un dispositivo real. [../BUILD_WINDOWS.md](#) es la fuente autorizada del comportamiento en escritorio.

El versionado de la clave

El sufijo `_v1` reserva la posibilidad de cambiar el formato leyendo la clave antigua y escribiendo una nueva. **Ese mecanismo de migración no existe todavía**: hoy el sufijo es solo una convención previsor. Cambiar la clave sin escribir la migración dejaría huérfano el avance de todos los dispositivos instalados.

El contrato transaccional

La propiedad más importante de este almacén no es lo que guarda, sino **cuándo** lo publica.

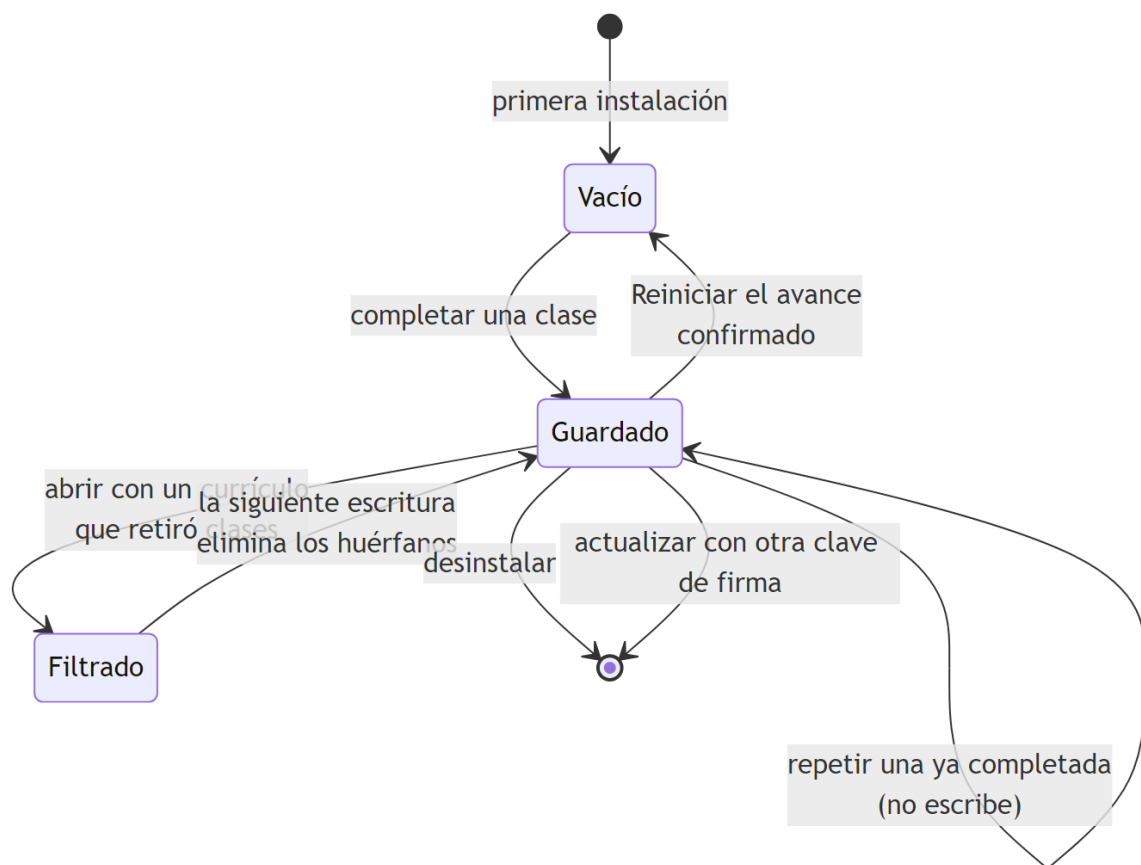
```
// AppState.completeLesson
final updated = {..._completedLessonIds, lessonId};
await _progressRepository.writeCompletedLessonIds(updated); // puede lanzar
_completedLessonIds = updated;
notifyListeners();
```

`ProgressRepository` lanza `StateError` si `setStringList` o `remove` devuelven `false`. Si lanza, el `await` propaga y las dos líneas siguientes no se ejecutan. La interfaz nunca muestra un avance que el disco no confirmó.

Tres pruebas de `test/app_state_test.dart` protegen esto inyectando un `ProgressStore` en memoria que falla a voluntad:

Prueba	Simula	Comprueba
«rechaza una clase inexistente sin alterar el avance»	Identificador falso	<code>ArgumentError</code> y almacén intacto
«solo publica el avance después de guardarlo correctamente»	Escritura fallida	<code>StateError</code> y <code>completedLessonIds</code> vacío
«solo borra el avance en memoria después de borrar el guardado»	Borrado fallido	<code>StateError</code> y avance anterior conservado

Ciclo de vida del dato



El diagrama muestra todas las transiciones posibles del avance. Lo que no muestra: el estado «Filtrado» es solo en memoria —el disco sigue conteniendo el identificador huérfano hasta la próxima escritura—, y las dos transiciones a [*] son irreversibles porque no hay exportación ni copia remota en 0.1.0.

Lo que se pierde, y cuándo

Conviene decirlo sin rodeos, porque es lo que sorprende a una familia:

- **Desinstalar borra el avance.** No hay recuperación.
- **Actualizar con una clave de firma distinta** obliga a desinstalar primero, y por tanto borra el avance. El APK 0.1.0 está firmado con una clave efímera generada durante la compilación, así que **esto ocurrirá con la primera actualización** salvo que antes se configure la firma permanente. Está advertido en el README, en la landing, en las notas de la versión y en [../PARENT_GUIDE.md](#), que recomienda anotar las clases en papel.
- **Limpiar los datos de la aplicación** desde el sistema lo borra.
- Cerrar la aplicación **no** borra nada.

Continuar por

- [08 · Flujo de datos](#) para el recorrido completo.
- [11 · Seguridad](#) para la superficie del almacenamiento local.
- [../PRIVACY.md](#) para el inventario de datos infantiles.

Contenido de este documento

La conclusión primero

Mapa completo

Flujo de lectura del currículo

Flujo de lectura del avance

Cálculos derivados

Flujo de escritura

Datos que nunca se guardan

Flujo de datos fuera de la aplicación

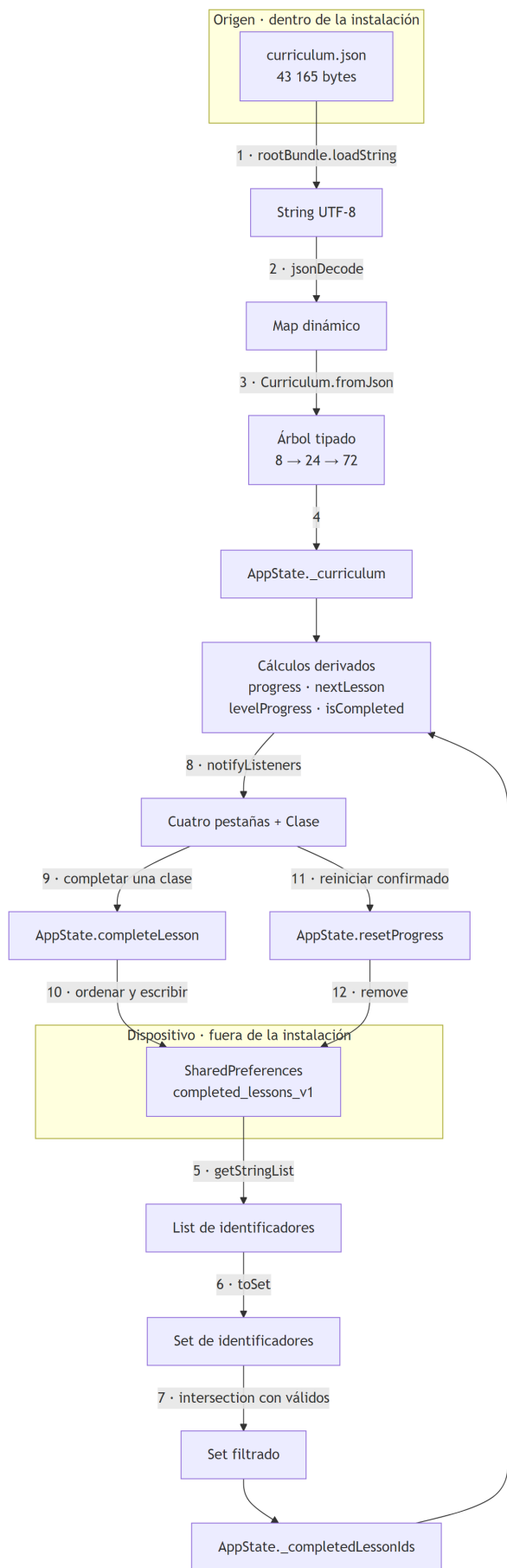
Continuar por

08 · Flujo de datos

La conclusión primero

Solo hay **dos** flujos de datos en todo el sistema, y ninguno cruza la red. Uno va del binario a la pantalla y no vuelve. El otro va de la pantalla al disco del dispositivo y vuelve al abrir. Ese es todo el movimiento de información que existe.

Mapa completo



El diagrama muestra las doce transformaciones que sufre un dato en este sistema, de principio a fin. Lo que **no** muestra, y conviene decir: no hay ninguna flecha que salga del recuadro del dispositivo. No existe serialización hacia red, ni exportación a archivo, ni telemetría, ni caché. Tampoco muestra el camino de error, que se detalla más abajo.

Flujo de lectura del currículo

Paso	Operación	Dónde	Puede fallar con
1	<code>rootBundle.loadString('assets/conCurriculumRepository.load</code>		<code>FlutterError</code> si el activo no está en el bundle
2	<code>jsonDecode(source)</code> as <code>Map<String, dynamic></code>	ídem	<code>FormatException</code> si el JSON es inválido
3	<code>Curriculum.fromJson(json)</code>	<code>lib/domain/curriculum.dart</code>	<code>TypeError</code> si falta un campo o el tipo no coincide
4	Asignación a <code>_curriculum</code>	<code>AppState.load</code>	—

Los tres fallos posibles suben sin capturarse hasta `main`, que los reporta con `FlutterError.reportError` y muestra `_StartupErrorApp`. El paso 3 es deliberadamente estricto: es preferible fallar al arrancar que producir una clase a medias que llegue a la interfaz con un campo vacío.

Este flujo ocurre una sola vez por ejecución. No hay recarga, no hay invalidación y el árbol resultante es inmutable.

Flujo de lectura del avance

Paso	Operación	Detalle
5	<code>getStringList('completed_lessons_v1')</code>	Devuelve <code>const <String>[]</code> si la clave no existe
6	<code>.toSet()</code>	Elimina duplicados que pudieran haber quedado
7	<code>.intersection(validIds)</code>	Descarta identificadores que el currículo actual no declara

El paso 7 es el que evita que un porcentaje quede atascado para siempre si una versión futura retira una clase. Detalle importante: el filtrado es **solo en memoria**. El identificador huérfano sigue en el disco hasta la próxima escritura, momento en el que desaparece porque `completeLesson` escribe el conjunto ya filtrado, no el original.

Cálculos derivados

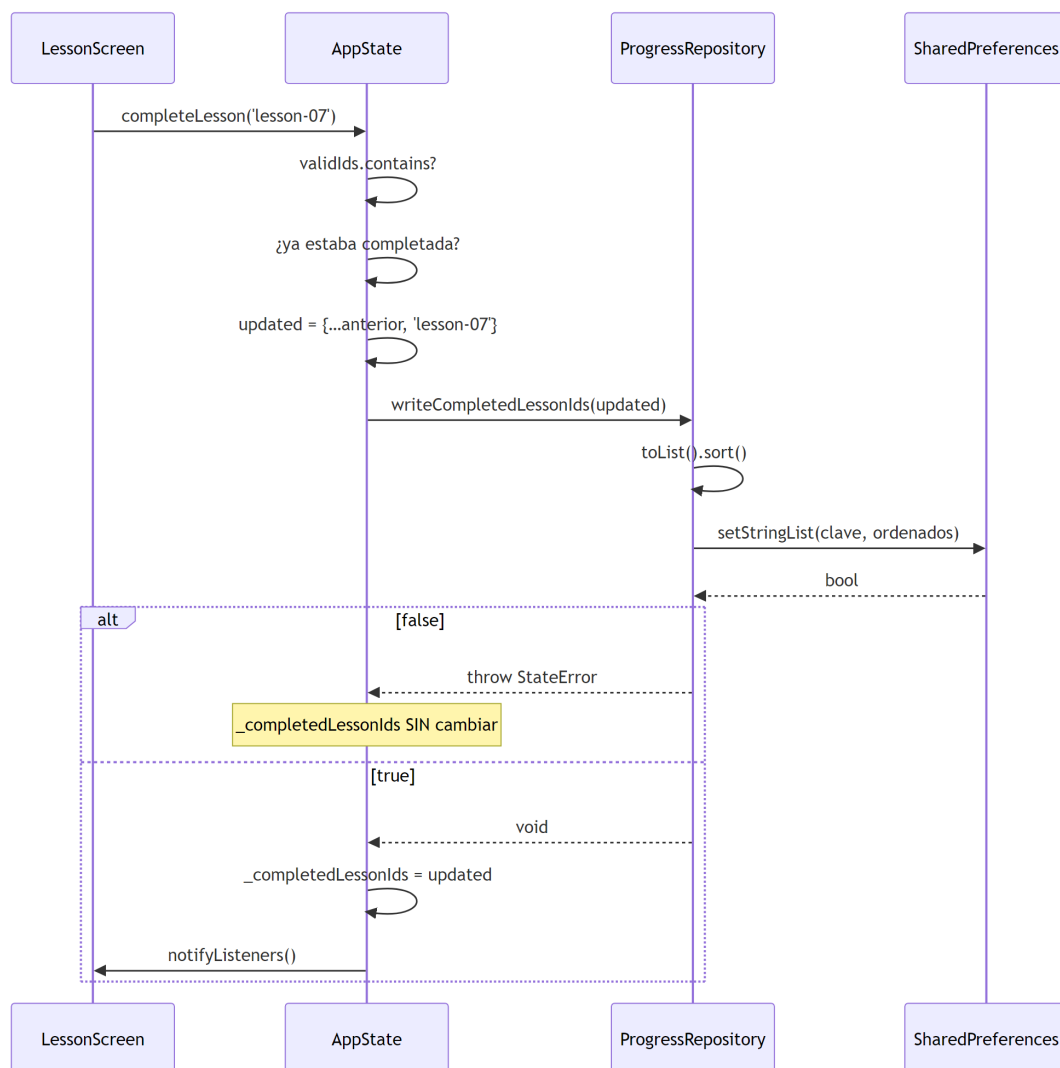
Nada de esto se almacena. Se recalcula en cada lectura desde las dos únicas fuentes de verdad.

Valor	Fórmula	Coste	Quién lo consume
<code>totalCount</code>	<code>curriculum.lessons.length</code>	Aplana y ordena 24 elementos	<code>_ProgressCard</code>
<code>completedCount</code>	<code>_completedLessonIds.length</code>	O(1)	<code>_ProgressCard</code> , <code>_HeroProgress</code>

<code>progress</code>	<code>completedCount / totalCount</code> , o 0	$O(n \log n)$ por <code>totalCount</code>	Barras y porcentajes
<code>nextLesson</code>	Primera clase sin completar en orden global	$O(n)$ sobre lista ya ordenada	<code>HomeTab</code>
<code>levelProgress(level)</code>	Completadas del nivel / 3	$O(3)$	Ruta y Avance
<code>isCompleted(id)</code>	Pertenencia al conjunto	$O(1)$	Cada <code>LessonCard</code>

`ProgressTab` invoca `levelProgress(level)` **dos veces** por nivel en el mismo build: una para la barra y otra para el porcentaje. Son 16 llamadas triviales; se anota porque es el tipo de detalle que crece mal si el currículum se amplía.

Flujo de escritura



Lo que el diagrama muestra: el punto exacto donde el estado en memoria se sincroniza con el disco, y que ese punto está **después** de la confirmación. Lo que no muestra: que `AppState` construye `updated` como un conjunto nuevo en vez de mutar el existente, lo que garantiza que el conjunto anterior siga íntegro si la escritura falla.

Se escribe **siempre el conjunto completo**, no un delta. Con 24 elementos como máximo no compensa complicarlo, y escribir el total elimina toda posibilidad de que un delta perdido deje el almacén inconsistente.

Datos que nunca se guardan

Esta lista importa tanto como el flujo:

Dato	Dónde vive	Se pierde al
Casillas marcadas de una clase a medias	<code>_LessonScreenState._activityChecks</code>	Salir de la clase
Velocidad del metrónomo (<code>_bpm</code>)	<code>_RhythmLabScreenState</code>	Cerrar la aplicación
Interruptores de sonido y vibración	ídem	Cerrar la aplicación
Agrupación rítmica elegida	ídem	Cerrar la aplicación
Pestaña seleccionada	<code>_HomeShellState._selectedIndex</code>	Cerrar la aplicación
Niveles desplegados en Ruta	ExpansionTile interno	Cerrar la aplicación

INFERENCIA: no persistir las preferencias del metrónomo es coherente con la decisión de guardar lo mínimo, pero significa que quien practica cada día vuelve a encontrar 84 PPM y ambas salidas encendidas. Ningún documento del repositorio declara si eso es intencionado.

Flujo de datos fuera de la aplicación

Los otros flujos del repositorio no tocan datos de personas usuarias, pero existen y conviene tenerlos mapeados:

Flujo	Origen	Destino	Herramienta
Versión	<code>pubspec.yaml</code>	Landing, nombres de artefacto, comprobación del APK, tag de la release	<code>tool/app_version.mjs</code>
Capturas	Aplicación web servida en localhost	<code>docs/screenshots/*.png</code>	<code>tool/capture_screenshots.mjs</code>
Landing	<code>site/</code> + capturas + <code>logo.svg</code>	<code>build/site/</code> → GitHub Pages	<code>tool/build_site.mjs</code>
Currículo empaquetado	APK	Comparación SHA-256 con el del repositorio	<code>tool/verify_apk.mjs</code>

El flujo de la versión es el más consecuente: un único punto de verdad alimentando cinco destinos. El CHANGELOG documenta que antes no era así y que un tag `v0.2.0` habría publicado archivos llamados `0.1.0`.

Continuar por

- [07 · Persistencia](#) para el detalle del almacenamiento.
- [09 · APIs e integraciones](#) para la demostración de que no hay red.

Contenido de este documento

No hay red. Demostración

Lo que sí existe: interfaces de plataforma

Contratos de datos

Servicios externos del repositorio

Continuar por

09 · APIs e integraciones

No hay red. Demostración

El producto **no realiza ninguna llamada de red**. No es una afirmación de intención: es comprobable con un comando.

```
$ grep -rniE "\bhttp\b|\bhttps\b|HttpClient|WebSocket\b|Socket\b|package:http|package:dio|Uri\|parse|InternetAddress" lib/
$ echo $?
1
```

Código de salida 1 significa «ninguna coincidencia». No hay cliente HTTP, no hay sockets, no hay URL construidas en el código de producto.

Dos comprobaciones complementarias:

```
$ grep -rn "dart:io" lib/      # sin resultados
$ grep -c "^ [a-z_]*:$" pubspec.lock
52                          # ningún paquete de red entre ellos
```

La única dependencia de ejecución declarada es `shared_preferences: ^2.5.3`.

Nota sobre falsos positivos. Un grep de `dio` sin límites de palabra coincide con la palabra «audio» en un texto de la interfaz. Por eso el comando de arriba usa `\b`. Vale la pena decirlo: una comprobación que se presenta como prueba tiene que ser reproducible tal cual está escrita.

Y no solo en el código: en el binario

La prueba más fuerte no está en el repositorio sino en el paquete compilado. `tool/verify_apk.mjs` incluye `android.permission.INTERNET` entre los permisos prohibidos y **falla la release** si aparece. Es la única comprobación que ve el manifiesto **fusionado**, es decir, los permisos que una dependencia podría añadir sin que aparezcan en el código del proyecto.

Según `../VALIDATION_RESULT.md`, sobre el APK publicado de 0.1.0 el resultado fue «sin permisos del sistema declarados». Esa medición no se reprodujo en este análisis por falta de las build-tools de Android; se cita como afirmación del repositorio.

`../PERMISSIONS.md` sigue siendo la fuente autorizada de la matriz completa de permisos y activaciones.

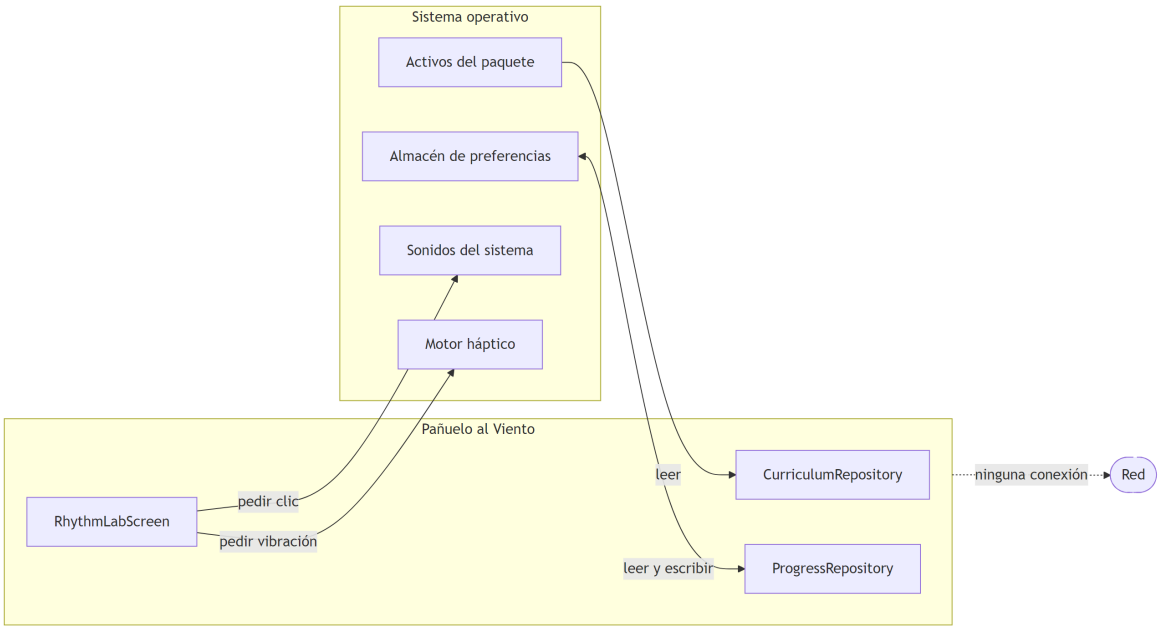
Lo que sí existe: interfaces de plataforma

No hay red, pero sí hay cuatro puntos donde la aplicación habla con el sistema operativo. Son las integraciones reales del producto.

Interfaz	Símbolo	Dónde	Dirección	Qué obtiene o pide
Activos empaquetados	<code>rootBundle.loadString</code>	<code>curriculum_repository.dart:24</code>	Escritura	Los 43 165 bytes del currículo
Preferencias del sistema	<code>SharedPreferences</code>	<code>progress_repository.dart</code>	Lectura y escritura	Hasta 24 identificadores
Sonido del sistema	<code>SystemSound.play(Systemrhythm_lab_screen.dart:35)</code>		Salida	Un clic breve
Retroalimentación háptica	<code>HapticFeedback.mediumInrhythm_lab_screen.dart:35)</code>		Salida	Una vibración corta

Las dos últimas son **salidas, no sensores**. No leen el entorno, no identifican a nadie y no conservan nada. El sistema operativo puede ignorarlas: en Windows la háptica normalmente no existe, y esa ausencia no se trata como error.

Ninguna de las cuatro requiere un permiso que el usuario deba conceder.



El diagrama muestra las cuatro únicas superficies de contacto con el exterior y marca la red como ausente. Lo que no muestra: las dos flechas de salida (AUDIO, HAPTIC) pueden no producir efecto y la aplicación no lo comprueba ni lo necesita.

Contratos de datos

Al no haber API, los contratos que hay que respetar son de formato de archivo.

Contrato 1 · `curriculum.json`

Consumido por cuatro consumidores independientes que **deben mantenerse de acuerdo**:

Consumidor	Lenguaje	Qué exige
------------	----------	-----------

lib/domain/curriculum.dart	Dart	Tipos exactos en 12 campos de clase, 6 de nivel y 4 de actividad
tool/validate_curriculum.mjs	Node	Cardinalidades, unicidad, órdenes y 8 campos no vacíos
tool/validate_curriculum.dart	Dart	Lo mismo, duplicado
tool/verify_apk.mjs	Node	Solo levels[], lessons[] para contar, y el hash completo

Añadir un campo obligatorio obliga a tocar los cuatro más el JSON de las 24 clases. Es el cambio con mayor radio del repositorio. El diccionario completo está en [07 · Persistencia](#).

Contrato 2 · pubspec.yaml → todo lo demás

tool/app_version.mjs exige literalmente una línea:

```
version: 0.1.0+1
```

y la expresión regular es `/^version:\s*(\d+\.\d+\.\d+)(\d+)\s*$/m`. Un sufijo de precompilación como 0.2.0-beta+3 **no coincide** y haría fallar el validador, la landing y la release. **NO DOCUMENTADO EN EL REPOSITORIO**: la restricción existe en el código pero ningún documento la enuncia.

Consumidores: validate_repository.mjs, verify_apk.mjs, build_site.mjs y el job version de release.yml.

Contrato 3 · __APP_VERSION__ en la landing

site/index.html debe contener el marcador `__APP_VERSION__` y **no debe** contener ningún nombre de artefacto con versión escrita a mano. validate_repository.mjs comprueba ambas cosas y falla si alguna se incumple:

```
if (!site.includes('__APP_VERSION__')) { ... }
const hardcoded = [...site.matchAll(/PanueloAlViento-(\d+\.\d+\.\d+)-/g)];
```

Motivo declarado en el código: «una descarga con el número congelado devuelve 404 tras publicar, y nadie se entera hasta que alguien la intenta».

Contrato 4 · Nombres de los artefactos

Cuatro nombres exactos, con la versión dentro, comprobados en tres sitios distintos:

```
PanueloAlViento-<versión>-Android.apk
PanueloAlViento-<versión>-Windows-Setup.exe
PanueloAlViento-<versión>-Windows.msi
PanueloAlViento-<versión>-Windows-portable.zip
```

Dónde se comprueba	Cómo
validate_repository.mjs	Que el README los nombre con la versión actual
release.yml, job windows	Que existan exactamente esos tres y ningún otro en <code>release-artifacts</code>

`release.yml`, `job publish`

Que los cuatro estén presentes y que no haya archivos con otra versión

La comprobación de «ningún otro» del `job windows` es la que explica el commit «Separa símbolos de los artefactos Windows»: el archivo `.wixpdb` que genera WiX se dirige a `build/wix/` precisamente para que no aparezca en la carpeta de artefactos y haga fallar el paso.

Servicios externos del repositorio

El **producto** no habla con nadie. El **repositorio** sí depende de servicios, y conviene tenerlos inventariados:

Servicio	Uso	Dónde	Si desaparece
GitHub Actions	CI, release y publicación de la landing	<code>.github/workflows/</code>	No hay verificación automática ni artefactos
GitHub Releases	Distribución de los cuatro archivos	<code>release.yml</code> , <code>job publish</code>	No hay canal oficial de descarga
GitHub Pages	Landing pública	<code>pages.yml</code>	La landing deja de estar accesible
<code>pub.dev</code>	52 paquetes resueltos	<code>pubspec.lock</code>	No se puede resolver desde cero; <code>pubspec.lock</code> fija las versiones pero no las almacena
<code>subosito/flutter-action@v2</code>	Instalar Flutter en CI	Los dos workflows que compilan	Los jobs de calidad y compilación fallan
<code>softprops/action-gh-release@v2</code>	Crear la release	<code>release.yml</code>	Habría que publicar a mano
Chocolatey	Inno Setup y WiX en el runner	<code>release.yml</code> , <code>job windows</code>	No se generan EXE ni MSI
<code>img.shields.io</code>	Insignias del README	<code>README.md</code>	Solo cosmético

Las acciones de terceros se fijan **por etiqueta mayor** (`@v4`, `@v2`), no por SHA. Registrado en [11 · Seguridad](#) como observación, no como fallo: es la práctica habitual y el propio SECURITY.md declara que los workflows usan permisos mínimos.

Continuar por

- [10 · Configuración](#) para los valores y secrets.
- [11 · Seguridad](#) para la superficie de ataque.
- `../PERMISSIONS.md` para la matriz completa de permisos.

Contenido de este documento

Los siete archivos

pubspec.yaml

analysis_options.yaml

.gitattributes

.gitignore

Variables de entorno

Secrets de GitHub

Valores fijados en el código

Continuar por

10 · Configuración

No hay archivos `.env`, ni `config.json`, ni parámetros en tiempo de ejecución. Toda la configuración es de **compilación y de repositorio**, y vive en siete archivos.

Los siete archivos

Archivo	Gobierna	Consumido por
pubspec.yaml	Identidad, versión, SDK, dependencias, activos e iconos	Flutter y las cuatro herramientas que leen la versión
pubspec.lock	Versiones exactas de los 52 paquetes	<code>flutter pub get</code>
analysis_options.yaml	Reglas de análisis estático	<code>flutter analyze</code>
.gitignore	Qué no se versiona	Git
.gitattributes	Finales de línea y archivos binarios	Git
.editorconfig	Formato en el editor	Editores compatibles
.github/workflows/*.yaml	Automatización	GitHub Actions

pubspec.yaml

```
name: panuelo_al_viento
publish_to: "none"
version: 0.1.0+1
```

```
environment:  
  sdk: ">=3.7.0 <4.0.0"  
  flutter: ">=3.29.0"
```

Clave	Valor	Por qué importa
name	panuelo_al_viento	Determina el nombre del ejecutable Windows y el prefijo package:panuelo_al_viento/ de los imports en las pruebas
publish_to	"none"	Impide publicar en pub.dev por accidente
version	0.1.0+1	Única fuente de verdad. Formato obligatorio X.Y.Z+N
environment.sdk	>=3.7.0 <4.0.0	Mínimo declarado; CI usa Dart 3.12.2
environment.flutter	>=3.29.0	Mínimo declarado; CI fija 3.44.6

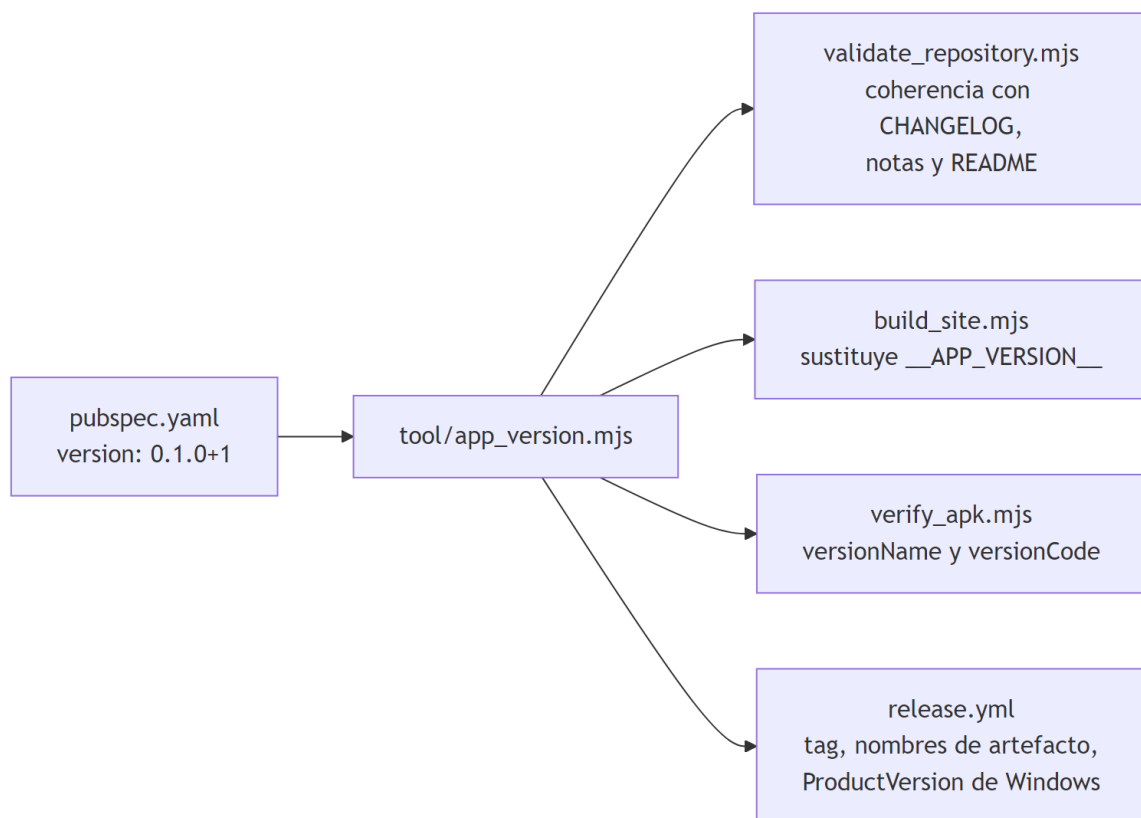
La versión, en detalle

El único punto donde el número existe. tool/app_version.mjs lo extrae con:

```
source.match(/^version:\s*(\d+\.\d+\.\d+)\s*(\d+)\s*$/m)
```

y lanza un error explícito si no coincide. **Consecuencia no documentada en el repositorio:** un sufijo de precompilación (0.2.0-beta+3) no coincide con esa expresión y rompería el validador, la landing y la release.

Quien consume esa versión:



El diagrama muestra el árbol de consumo de un único valor. Lo que no muestra: el `+N` (número de compilación) solo lo usan `verify_apk.mjs` —como `versionCode` esperado— y el propio Android; la landing y los nombres de archivo usan únicamente `X.Y.Z`.

Activos

```
assets:
  - assets/content/curriculum.json
  - assets/branding/
```

El primero es un archivo concreto; el segundo, un **directorio completo**. Cualquier archivo que se añada a `assets/branding/` entrará en el binario aunque nadie lo use.

Iconos

`flutter_launcher_icons` genera los iconos desde `assets/branding/app-icon.png`. `min_sdk_android: 24` y `adaptive_icon_background: "#F7EFE3"` —el crema de `AppColors.cream`—. `ios: false`: la plataforma no está soportada.

analysis_options.yaml

```
include: package:flutter_lints/flutter.yaml
linter:
  rules:
    avoid_print: true
    prefer_final_locals: true
    use_build_context_synchronously: true
```

Tres reglas añadidas sobre el conjunto de `flutter_lints`. La tercera es la que tiene consecuencias reales: obliga a comprobar `mounted` antes de usar un `BuildContext` después de un `await`, que es exactamente el patrón de `LessonScreen._complete()`.

`avoid_print` no afecta a `tool/*.mjs` —son JavaScript, no los ve el analizador— ni a `tool/validate_curriculum.dart`, que usa `stdout.writeln` y `stderr.writeln` en lugar de `print`. INFERENCIA: esa elección probablemente sea para cumplir la regla, aunque el archivo no lo dice.

Estado verificado: `flutter analyze` → `No issues found!`.

`.gitattributes`

```
* text=auto eol=lf
*.ps1 text eol=crlf
*.png binary
*.jpg binary
*.jpeg binary
*.ogg binary
*.mp3 binary
*.mp4 binary
```

LF en todo el repositorio, salvo PowerShell, que lleva CRLF. Respetarlo no es opcional: `tool/verify_apk.mjs` compara el SHA-256 del currículo del APK con el del repositorio, y un CRLF de más cambiaría el hash y haría fallar la release.

Cualquier herramienta que escriba archivos aquí debe forzar LF explícitamente. Las extensiones de audio y video declaradas como binarias no tienen archivos todavía: son previsión para los medios de la versión 0.3 descrita en el [../ROADMAP.md](#).

Este documento añade una línea a `.gitattributes` para los PDF generados, por el mismo motivo: una normalización de finales de línea corrompería un PDF.

`.gitignore`

Cuatro grupos, y el segundo es el que sorprende:

Grupo	Entradas	Nota
Flutter y Dart	<code>.dart_tool/</code> , <code>build/</code> , <code>coverage/</code> , <code>.packages</code> , <code>.pub-cache/</code> , <code>.flutter-plugins*</code>	Estándar
Plataformas generadas	<code>/android/</code> , <code>/windows/</code> , <code>/release-artifacts/</code>	Se regeneran con <code>tool/bootstrap.*</code> o en CI
Editores y sistema	<code>.idea/</code> , <code>.vscode/</code> , <code>*.iml</code> , <code>.DS_Store</code> , <code>Thumbs.db</code>	Estándar
Firma y secretos	<code>*.jks</code> , <code>*.keystore</code> , <code>key.properties</code> , <code>.env</code>	Impide confirmar material de firma
Documento local	<code>MASTER_PROMPT.md</code>	«Documento de trabajo local; no forma parte del producto publicado»

El grupo de firma es un control de seguridad real, no higiene: `key.properties` contiene contraseñas del almacén de claves y `release.yml` lo crea dentro del runner.

Variables de entorno

Ninguna la necesita el producto. Todas son de herramientas:

Variable	Lee	Por defecto	Para qué
AAPT2	verify_apk.mjs	Búsqueda automática	Ruta directa al binario aapt2
ANDROID_SDK_ROOT	verify_apk.mjs, release.yml	—	Localizar las build-tools
ANDROID_HOME	verify_apk.mjs	—	Alternativa a la anterior
LOCALAPPDATA	verify_apk.mjs, capture_screenshots.mjs	—	Rutas por defecto en Windows
SITE_OUT	build_site.mjs	build/site	Directorio de salida de la landing
SITE_PORT	build_site.mjs	8080	Puerto de --serve
SHOT_PORT	capture_screenshots.mjs	8731	Puerto del servidor de capturas
SHOT_DEBUG_PORT	capture_screenshots.mjs	9333	Puerto del protocolo DevTools
CHROME_BIN	capture_screenshots.mjs	Rutas habituales	Ejecutable de Chrome
RUNNER_TEMP	release.yml	Lo pone GitHub	Directorio del almacén de claves efímero

verify_apk.mjs busca aapt2 recorriendo build-tools y ordenando las versiones con localeCompare numérico para tomar la más reciente.

Secrets de GitHub

Cuatro, todos opcionales, todos para la firma de Android:

Secret	Contenido	Si falta
ANDROID_KEYSTORE_BASE64	Archivo JKS en base64	Se genera una clave efímera con keytool y se emite un ::warning::
ANDROID_KEY_ALIAS	Alias de la clave	ídem
ANDROID_KEY_PASSWORD	Contraseña de la clave	ídem
ANDROID_STORE_PASSWORD	Contraseña del almacén	ídem

Estado en 0.1.0: no configurados. ../VALIDATION_RESULT.md lo confirma midiendo la firma del APK publicado. La consecuencia —una actualización obligará a desinstalar y borrará el avance local— está declarada en el README, en la landing, en las notas de la versión, en SECURITY.md y en el ../ROADMAP.md, que la pone como prioridad inmediata.

La rama de respaldo de release.yml contiene un alias y una contraseña **literales** para ese almacén efímero. No son un secreto real —la clave se crea y se descarta en el runner—, pero son credenciales escritas en un repositorio público y

se registran como observación en [15 · Riesgos](#). Sus valores no se reproducen aquí.

`../BUILD_MOBILE.md` es la fuente autorizada del procedimiento para crear y proteger una clave permanente.

Valores fijados en el código

Constantes que no salen de ningún archivo de configuración y que hay que buscar para cambiar:

Valor	Dónde	Comentario
<code>cl.panueloalviento.panuelo_al_viento</code>	<code>verify_apk.mjs</code> , <code>bootstrap.*</code> , <code>release.yml</code>	Identificador del paquete Android
24 (nivel de API mínimo)	<code>verify_apk.mjs</code> , <code>configure_platforms.mjs</code> , <code>pubspec.yaml</code>	Android 7.0
840	<code>home_shell.dart</code>	Umbral entre barra inferior y lateral
60–120, 12 divisiones	<code>rhythm_lab_screen.dart</code>	Rango del deslizador; el inicial 84 no cae en división
6	<code>rhythm_lab_screen.dart</code>	Pulsos por ciclo
<code>completed_lessons_v1</code>	<code>progress_repository.dart</code>	Clave del almacén
4.5 y 1.12	<code>theme_contrast_test.dart</code>	Umbrales WCAG
8, 24, 3, 10–15	Ambos validadores y la prueba de integridad	Cardinalidades del currículo
'Explorar las 24 clases'	<code>home_tab.dart</code>	Cifra escrita a mano, existiendo <code>state.totalCount</code>
'Se desmarcarán las 24 clases...'	<code>progress_tab.dart</code>	Ídem
7C81FC5C-...-78019866B239	<code>installer.iss</code> y <code>product.wxs</code>	Identidad del producto Windows
3.44.6	<code>ci.yml</code> y <code>release.yml</code>	Versión de Flutter, repetida en dos archivos

Las dos cifras escritas a mano en la interfaz y la versión de Flutter repetida están registradas en [15 · Riesgos](#).

Continuar por

- [11 · Seguridad](#) para el tratamiento de secretos.
- [13 · Despliegue y operación](#) para el uso de estos valores en CI.

Contenido de este documento

Superficie de ataque, delimitada

Controles presentes

Controles ausentes

Hallazgos

Barrido de secretos sobre el árbol versionado

Lo que está bien y conviene proteger

Continuar por

11 · Seguridad

[../SECURITY.md](#) es la fuente autorizada para **reportar** una vulnerabilidad y describe la superficie de 0.1.0 en términos de producto. [../PRIVACY.md](#) y [../PERMISSIONS.md](#) gobiernan el tratamiento de datos infantiles y de permisos.

Este documento aporta lo que no está cubierto: el análisis técnico de la superficie, el inventario de controles **presentes** y **ausentes**, y los hallazgos descritos sin cadena explotable.

Este análisis fue **estático**. No se ejecutaron pruebas destructivas, no se atacó ningún sistema y no se instaló el producto en un dispositivo.

Superficie de ataque, delimitada

Lo que hace que este producto tenga poca superficie no es un control: es lo que no tiene.

Vector habitual	Estado aquí	Evidencia
Servidor o API propia	No existe	<code>grep</code> de APIs de red sin coincidencias — ver 09
Autenticación y sesiones	No existe	No hay cuentas ni identidad
Base de datos	No existe	Ningún motor entre los 52 paquetes — ver 07
Inyección SQL	Imposible	No hay SQL
Entrada de texto de la persona usuaria	No existe	Todos los controles son casillas, deslizadores y botones
Deserialización de datos externos	Limitada	Solo el JSON propio, empaquetado y de solo lectura
Carga de archivos	No existe	No hay importación
Contenido de terceros	No existe	Sin música, video ni web incrustada

Actualización automática	No existe	La instalación es manual
Permisos sensibles	Ausentes	Verificado sobre el APK publicado por <code>verify_apk.mjs</code>

Lo que queda es un binario local que lee un activo propio y escribe hasta 24 cadenas en las preferencias del sistema. Los riesgos reales, como enumera `SECURITY.md`, son **manipulación de artefactos, dependencias, persistencia local y cambios futuros de permisos**.

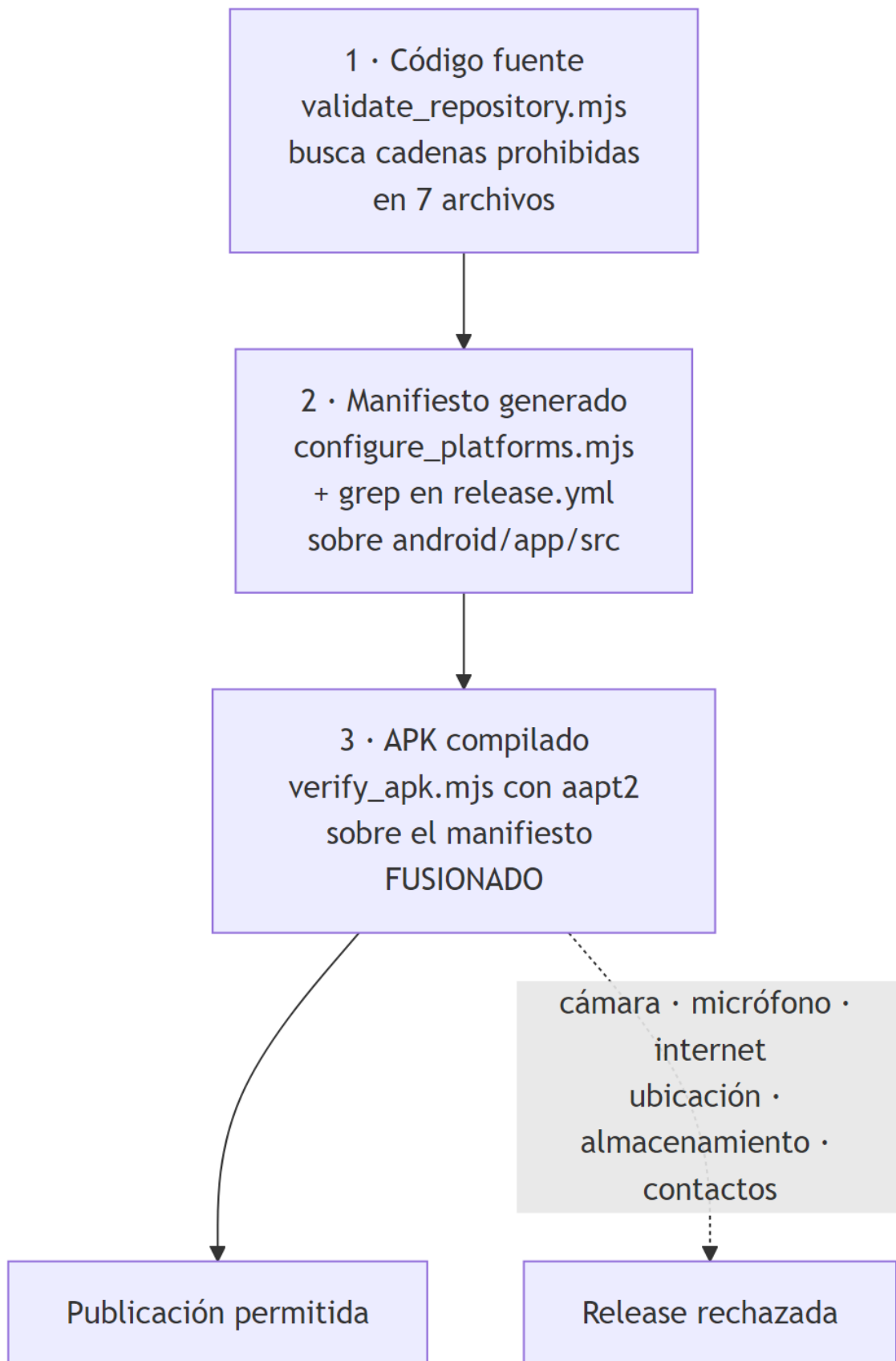
Controles presentes

En el código

Control	Dónde	Qué impide
Validación de identificador antes de persistir	<code>AppState.completeLesson</code>	Que una errata de contenido contamine el almacén
Escritura confirmada antes de publicar	<code>AppState + ProgressRepository</code>	Estado visible que el disco no respalda
Conjunto de avance inmutable hacia fuera	<code>Set.unmodifiable</code> en <code>completedLessonIds</code>	Mutación del avance sin pasar por la persistencia
Filtrado de identificadores huérfanos	<code>AppState.load</code>	Porcentaje inflado por clases retiradas
Confirmación antes de borrar	<code>ProgressTab._confirmReset</code>	Pérdida accidental e irreversible
Captura del arranque	<code>main</code>	Pantalla en blanco ante un activo corrompido
Modelos inmutables	<code>lib/domain</code>	Estado compartido mutable
<code>use_build_context_synchronously</code>	<code>analysis_options.yaml</code>	Uso de un contexto ya desmontado tras un <code>await</code>

En la cadena de publicación

Tres controles independientes sobre permisos, en tres momentos distintos, porque cada uno puede fallar sin que el anterior se entere:



El diagrama muestra la defensa en profundidad sobre permisos. Lo que no muestra, y es la razón de que el tercero exista: **solo el control 3 ve el manifiesto fusionado**, es decir, los permisos que una dependencia podría añadir sin aparecer nunca en el código del proyecto. Los controles 1 y 2 son ciegos a eso.

Control	Herramienta	Alcance
Cadenas prohibidas	<code>validate_repository.mjs</code>	<code>permission_handler</code> , dos claves de sensor y <code>dart:io</code> , en 7 archivos concretos
Manifiesto fuente	<code>configure_platforms.mjs</code>	Lanza excepción si encuentra <code>CAMERA</code> o <code>RECORD_AUDIO</code>
Manifiesto fuente, otra vez	<code>release.yml</code>	<code>grep -R</code> sobre <code>android/app/src</code>
Manifiesto fusionado	<code>verify_apk.mjs</code>	Siete permisos prohibidos, incluido <code>INTERNET</code>
Firma	<code>release.yml</code>	<code>apksigner verify --verbose</code>
Integridad del contenido	<code>verify_apk.mjs</code>	SHA-256 del currículo empaquetado contra el del repositorio
Integridad de la descarga	<code>release.yml</code>	<code>SHA256SUMS.txt</code> con los cuatro artefactos

En el repositorio

Control	Detalle
<code>.gitignore</code> de material de firma	<code>*.jks</code> , <code>*.keystore</code> , <code>key.properties</code> , <code>.env</code>
Permisos mínimos por workflow	<code>ci.yml</code> : <code>contents: read</code> . <code>pages.yml</code> : <code>contents: read</code> , <code>pages: write</code> , <code>id-token: write</code> . <code>release.yml</code> : <code>contents: write</code>
concurrency en release	<code>cancel-in-progress: false</code> : una publicación no se aborta a medias
<code>pubspec.lock</code> versionado	Las 52 versiones resueltas son las mismas en toda compilación
Una sola dependencia de ejecución	Superficie de suministro mínima
Coherencia documental automatizada	<code>validate_repository.mjs</code> impide que el README prometa lo que el manifiesto no declara

Controles ausentes

Enumerar solo lo presente daría una imagen falsa.

Control ausente	Impacto	Comentario
Firma Android permanente	Alto y ya materializado	El APK 0.1.0 usa clave efímera. La primera actualización obligará a desinstalar y borrará el avance local
Firma Authenticode de Windows	Medio	SmartScreen advertirá. Declarado en README y <code>BUILD_WINDOWS.md</code>

Acciones de terceros fijadas por SHA	Bajo-medio	Se usan @v4 y @v2, etiquetas móviles. Un compromiso del repositorio de la acción se propagaría
Escaneo automático de dependencias	Bajo	Sin Dependabot ni auditoría de vulnerabilidades en CI. La superficie es de 52 paquetes
Escaneo automático de secretos	Bajo	Ninguna comprobación impide confirmar un secreto nuevo
Cifrado del avance local	Ninguno, y es correcto	Los datos son identificadores como lesson-01. Cifrarlos añadiría gestión de claves sin proteger nada
Esquema JSON formal	Bajo	El currículo es propio y empaquetado; el riesgo es de corrección, no de seguridad
Comprobación de integridad en ejecución	Bajo	La aplicación no verifica su propio activo al arrancar. Un activo alterado tras la instalación implica que ya se comprometió el dispositivo

Hallazgos

Descritos sin cadena explotable y sin reproducir ningún valor.

H-1 · Credenciales literales en el workflow de release · Media

`.github/workflows/release.yml`, en la rama que se ejecuta cuando los secrets de firma **no** están configurados, contiene un alias y dos contraseñas escritas literalmente. Se usan para crear con `keytool` un almacén de claves que vive solo en el runner y se descarta al terminar el job.

- **No es la filtración de un secreto real.** La clave no existía antes ni persiste después.
- **Sí es material con forma de credencial en un repositorio público**, que puede disparar escáneres de secretos y, sobre todo, señala el estado que hay que resolver: mientras esa rama se ejecute, cada release produce un APK firmado con una clave distinta e irrepetible.
- El propio workflow emite un `::warning::` al usarla, y la limitación está declarada en cinco documentos del repositorio. No hay ocultación.
- **Recomendación:** configurar los cuatro secrets. Al hacerlo, la rama de respaldo deja de ejecutarse y las contraseñas literales dejan de tener función; podrían entonces sustituirse por un valor generado al vuelo.

H-2 · Cobertura parcial del control de cadenas prohibidas · Baja

`validate_repository.mjs` busca las cadenas prohibidas en una **lista fija de siete archivos**:

```
const sourceFiles = ['pubspec.yaml', 'lib/main.dart', 'lib/app.dart'];
const libSources = [
  'lib/screens/rhythm_lab_screen.dart',
  'lib/screens/lesson_screen.dart',
  'lib/screens/progress_tab.dart',
  'lib/data/progress_repository.dart',
];
```

lib/ tiene 15 archivos. Un archivo nuevo, o los ocho no listados, quedan fuera de este control. El impacto real es bajo porque los controles 2 y 3 —manifiesto fuente y manifiesto fusionado— sí son exhaustivos y son los que bloquean la release. Pero el primer control da una sensación de cobertura que no tiene.

Recomendación: recorrer lib/ completo en lugar de una lista.

H-3 · Acciones de terceros por etiqueta móvil · Baja

Se usan actions/checkout@v4, subosito/flutter-action@v2, softprops/action-gh-release@v2, entre otras. Las etiquetas mayores se mueven. Un compromiso del repositorio de una acción se propagaría a la siguiente ejecución.

Es la práctica habitual y el impacto está acotado por los permisos mínimos de cada workflow. Se registra por completitud, no como fallo.

H-4 · Sin escaneo de dependencias · Baja

52 paquetes resueltos, ninguna comprobación automática de vulnerabilidades conocidas. La superficie es pequeña y pubspec.lock está versionado —lo que significa que las versiones **sí son auditables**, a diferencia de un repositorio sin lock—, pero nadie las audita.

Recomendación: un job que ejecute la auditoría de dependencias del ecosistema es barato y encaja en ci.yml.

Barrido de secretos sobre el árbol versionado

Ejecutado durante este análisis sobre todos los archivos que Git rastrea:

```
$ git ls-files -z | xargs -0 grep -nEI \
"(sk-[A-Za-z0-9]{10,})ghp_|gho_|github_pat_|AKIA[0-9A-Z]{16}BEGIN [A-Z]*PRIVATE KEY|xox[baprs]-)"
# ninguna coincidencia
```

Un segundo barrido, insensible a mayúsculas, sobre patrones de contraseña y clave de API devolvió **cuatro líneas, todas en release.yml**: dos que asignan desde los secrets de GitHub —correctas— y dos de la rama de respaldo descritas en H-1. Ningún otro archivo del repositorio contiene material con forma de credencial.

No se encontraron datos personales, ni de niñas o niños, en el código, en la documentación ni en las nueve capturas.

Lo que está bien y conviene proteger

- **La defensa en profundidad sobre permisos.** Tres controles en tres momentos, y el tercero mira el manifiesto fusionado. Es un diseño mejor que el de la mayoría de las aplicaciones infantiles.
- **La verificación del artefacto, no solo del repositorio.** Abrir el APK y contar las clases que trae dentro detecta el fallo silencioso por excelencia.
- **La minimización de datos.** No se puede filtrar lo que no se recoge. El avance es tan pobre en información que cifrarlo sería teatro.
- **La honestidad documental.** El repositorio declara la clave efímera, la ausencia de firma Authenticode y la falta de validación humana en lugar de esconderlas. Es una propiedad de seguridad: quien lee sabe qué está aceptando.
- **La única dependencia de ejecución.** Cada plugin nuevo es una vía por la que un permiso puede entrar en el manifiesto fusionado sin aparecer en el código.

Continuar por

- [15 · Riesgos y deuda técnica](#) para el registro completo de hallazgos, incluidos los no relacionados con seguridad.
- [../SECURITY.md](#) para reportar.

Contenido de este documento

Las compuertas del repositorio

Resultado real de la ejecución

Las 25 pruebas, una por una

Cobertura: qué queda fuera

Análisis estático y formato

Los validadores independientes

Lo que ninguna compuerta cubre

Continuar por

12 · Pruebas y calidad

[../VALIDATION.md](#) es la fuente autorizada de **cómo** reproducir la verificación, y [../VALIDATION_RESULT.md](#) del resultado fechado de 0.1.0. Este documento describe qué protege cada compuerta, cuánto cubre realmente y **qué no cubre ninguna**.

Las compuertas del repositorio

Un verificador propio dice más sobre las prioridades de un equipo que su README. Aquí hay seis, y su orden en CI no es casual:



El diagrama muestra el orden de `ci.yml`. Lo que no muestra, y explica el orden: las tres primeras corren **sin instalar Flutter** —son las más baratas y las que más rápido detectan un error de mantenimiento—, y las tres últimas necesitan el toolchain completo. Un fallo temprano ahorra minutos de runner.

Resultado real de la ejecución

Ejecutado durante este análisis sobre el commit `6efae74`, con Flutter 3.44.6:

```
$ node tool/validate_repository.mjs
Repositorio válido: 32 archivos esenciales, 9 capturas y versión 0.1.0+1 coherente en pubspec, CHANGELOG, notas y README.

$ node tool/validate_curriculum.mjs
Currículo válido: 8 niveles, 24 clases y 72 actividades.

$ node tool/build_site.mjs
Landing ensamblada en ...\.build\site: versión 0.1.0 en 5 lugares, 9 capturas y 9 imágenes comprobadas.

$ dart format --output=none --set-exit-if-changed lib test tool
```

```
Formatted 22 files (0 changed) in 0.16 seconds.
```

```
$ flutter analyze
```

```
No issues found! (ran in 56.7s)
```

```
$ flutter test
```

```
All tests passed! (+25)
```

```
$ dart run tool/validate_curriculum.dart
```

```
Currículo válido: 8 niveles, 24 clases y 72 actividades.
```

Las siete en verde, antes y después de documentar el código.

Las 25 pruebas, una por una

A Prueba	Qué protege exactamente
« <code>curriculum</code> no es <code>JSON</code> en un modelo inmutable»	Que <code>Lesson.fromJson</code> mapee los 12 campos, resuelva <code>ActivityType</code> y que los minutos sumen
« <code>curriculum</code> tiene exactamente 24 clases y órdenes continuos»	Cardinalidades y <code>order</code> de 1 a 24 sin huecos
«cada clase tiene tres actividades que completan su duración»	3 actividades, suma exacta y rango 10–15 min
«todos los identificadores son únicos»	Sin <code>id</code> repetidos
« <code>progress</code> respeta y valida los identificadores locales»	El ciclo completo sobre <code>SharedPreferences</code> simulado, incluido el ordenamiento
« <code>progress</code> no se actualiza si no existe sin alterar el avance»	<code>ArgumentError</code> y que el almacén no se toque
«solo publica el avance después de guardarlo correctamente»	Que un fallo de escritura deje el avance vacío
«solo borra el avance en memoria después de borrar el guardado»	Que un fallo de borrado conserve el avance anterior
« <code>app</code> no muestra el <code>id</code> y el detalle de privacidad»	Las cuatro pestañas navegables y los textos de cámara y micrófono presentes
«16 pruebas en <code>test.dart</code> »	Contraste WCAG de cada par realmente dibujado, en modo claro y oscuro

Las tres pruebas más valiosas

Las de `app_state_test.dart`. Son las únicas que prueban una **regla de negocio con consecuencias para la persona usuaria**, no una estructura de datos. Inyectan un `_MemoryProgressStore` que falla a voluntad:

```
final store = _MemoryProgressStore(failWrites: true);
...
await expectLater(state.completeLesson('lesson-01'), throwsStateError);
expect(state.completedLessonIds, isEmpty);
```

Sin ellas, alguien podría invertir el orden de `completeLesson` —publicar antes de confirmar— y todas las demás pruebas seguirían pasando. Toda la interfaz `ProgressStore` existe para que estas tres puedan escribirse.

Las 16 pruebas de contraste

No miden lo obvio. Miden **el par de colores que la interfaz dibuja de verdad**, que no es el par que la teoría de Material sugiere. El propio archivo lo explica:

Las tarjetas de la aplicación pintan un color de contenedor y dejan que el texto herede `onSurface`, que es lo que hace Card de Material 3 cuando se le pasa un color propio. Por eso el par que hay que medir no es (`primaryContainer`, `onPrimaryContainer`) sino el que realmente se dibuja.

Se generan en bucle: 6 superficies × 2 modos = 12, más el pulso acentuado × 2, más la distinguibilidad de contenedores × 2 = **16**. Implementan la fórmula WCAG 2.1 de luminancia relativa a mano, sin dependencias, con umbral 4,5:1 para texto y 1,12:1 para que una tarjeta no se confunda con el fondo.

Son la razón de que exista `AppColors.redDeep`: el rojo de marca sobre blanco solo alcanzaba 4,2:1. El CHANGELOG documenta que la suite pasó de 9 a 25 pruebas al añadirlas.

Cobertura: qué queda fuera

Lo importante de una suite no es lo que cubre, sino saber lo que no.

Área	Cobertura	Consecuencia
<code>lib/domain</code>	Alta	Tres pruebas sobre el modelo y el currículo real
<code>lib/data</code>	Media-alta	Ciclo completo del progreso; el currículo se lee, pero no se prueba el fallo de lectura
<code>lib/state</code>	Alta en lo que importa	Las tres reglas transaccionales. <code>nextLesson</code> y <code>levelProgress</code> no tienen prueba dedicada
<code>lib/core</code>	Alta en color	16 pruebas de contraste. La tipografía no se prueba
<code>lib/screens</code>	Baja	Un solo smoke test. Ninguna prueba del metrónomo, del cierre de clase ni del reinicio
<code>lib/widgets</code>	Ninguna	<code>LessonCard</code> y <code>MovementDiagram</code> no tienen ninguna prueba
<code>tool/</code>	Ninguna	Las 11 herramientas no tienen pruebas propias
Compilación real	En CI, al publicar	No en <code>ci.yml</code>
Dispositivo real	Ninguna	Vibración, clic y persistencia entre reinicios sin verificar

Los cinco huecos que más importan

1. El metrónomo no tiene ninguna prueba. Es el componente con más lógica temporal del proyecto —la corrección de deriva sobre `Stopwatch`— y la única comprobación de que funciona es abrir la aplicación y escuchar. Una prueba de widget con reloj simulado podría verificar que el pulso n cae en $n \times 60/\text{bpm}$ incluso tras introducir un retraso artificial. INFERENCIA: es el hueco con mejor relación entre esfuerzo y valor.

2. Ninguna prueba cubre el cambio de pestaña. Por eso el comportamiento del temporizador que sobrevive al `IndexedStack` no lo detectó nadie. Se verificó en este análisis con una prueba temporal, que **no se dejó en el repositorio** porque documentar no es corregir.

3. MovementDiagram no tiene prueba. Ninguna comprobación relaciona los valores de `diagram` del currículo con las ramas del pintor. Una prueba de una línea que recorra las 24 clases y afirme que cada `diagram` tiene rama propia habría detectado que `pair` y `free` comparten dibujo.

4. LessonScreen no tiene prueba. La lógica de habilitación del botón y el comportamiento de repetición son reglas de producto sin red.

5. Las herramientas no se prueban a sí mismas. `app_version.mjs` es el símbolo del que dependen la landing, los nombres de artefacto y la release. Su expresión regular no tiene ninguna prueba.

Análisis estático y formato

Compuerta	Comando	Estado
Formato	<code>dart format --output=none</code> <code>--set-exit-if-changed lib test tool</code>	22 archivos, 0 cambios
Análisis	<code>flutter analyze</code>	Sin incidencias

`--set-exit-if-changed` es lo que convierte el formato en compuerta. El propio `VALIDATION.md` lo explica: sin esa opción, `dart format --output=none` imprime los archivos que cambiarían y **aun así devuelve éxito**, de modo que el paso pasa siempre y no comprueba nada. Fue un fallo real, corregido según el `CHANGELOG`.

Efecto secundario: el formateador cambia entre versiones de Dart. Por eso CI fija Flutter 3.44.6 y `CONTRIBUTING.md` pide formatear con esa versión.

Los validadores independientes

Su valor es que **corren sin instalar Flutter**. Cualquiera puede comprobar el repositorio con Node y en segundos.

[validate_curriculum.mjs](#)

8 niveles, 24 clases, 3 clases por nivel, identificadores únicos, órdenes 1–24 sin huecos, 3 actividades por clase, suma de minutos y 8 campos de texto no vacíos. No comprueba que `diagram` tenga dibujo ni que `type` sea un valor del enum.

`tool/validate_curriculum.dart` hace exactamente lo mismo en Dart. **CI no lo ejecuta**; solo lo mencionan `CONTRIBUTING.md` y el `MASTER_PROMPT.md` no versionado. Registrado como duplicación en [15 · Riesgos](#).

[validate_repository.mjs](#)

El más interesante, porque no comprueba código: comprueba que **la documentación diga la verdad**.

Comprobación	Fallo que previene
32 archivos esenciales presentes	Borrar un documento sin darse cuenta
<code>docs/releases/v<versión>.md</code> existe	Publicar sin notas

CHANGELOG.md encabeza la versión del manifiesto	Historial desincronizado
El README nombra los cuatro artefactos con la versión actual	README que promete descargas inexistentes
Las capturas que el README muestra existen	Imágenes rotas
Las notas de release no usan enlaces relativos	Enlaces muertos en GitHub Releases, donde no resuelven
site/index.html usa <code>__APP_VERSION__</code> y no versiones fijas	Landing que ofrece un 404 tras publicar
Cadenas prohibidas ausentes de 7 archivos	Introducción de dependencias de sensores

Cada una de esas comprobaciones existe porque el fallo correspondiente es fácil de cometer y difícil de notar. Es la compuerta con mejor relación entre líneas de código y errores evitados de todo el repositorio.

Limitación: la comprobación de cadenas prohibidas cubre 7 de los 15 archivos de `lib/`. Ver [11 · Seguridad](#), hallazgo H-2.

Lo que ninguna compuerta cubre

Y ningún documento debería sugerir lo contrario:

- **Validez pedagógica.** Que las 24 clases enseñen bien. Pendiente según `../TEACHER_REVIEW.md`.
- **Corrección cultural.** Que la secuencia represente honestamente la cueca. Requiere personas cultoras.
- **Seguridad corporal real.** Que las advertencias cubran los riesgos.
- **Accesibilidad con tecnologías de apoyo.** Hay etiquetas semánticas en el código; una auditoría con TalkBack y el Narrador es otra cosa.
- **Comportamiento en dispositivo.** Vibración, clic y persistencia entre reinicios.
- **Comprensión infantil.** Que un niño de diez años entienda las instrucciones.

`VALIDATION.md` lo dice con una frase que conviene no diluir: «una suite verde prueba consistencia técnica, no que el currículo sea cultural o corporalmente correcto».

Continuar por

- [13 · Despliegue y operación](#) para dónde se ejecutan estas compuertas.
- [15 · Riesgos](#) para los huecos registrados.

Contenido de este documento

Los tres workflows

ci.yml — la verificación continua

pages.yml — la landing

release.yml — la publicación

Firma de Android: el punto abierto

Operación de la landing

Regenerar las capturas

Regenerar los PDF de esta documentación

Continuar por

13 · Despliegue y operación

[../RELEASE_CHECKLIST.md](#) es la fuente autorizada de los **criterios** de publicación, y [../BUILD_MOBILE.md](#) y [../BUILD_WINDOWS.md](#) del procedimiento por plataforma. Este documento describe la **mecánica automatizada**: qué workflow hace qué, en qué orden, con qué compuertas, y cómo regenerar los PDF de esta carpeta.

Los tres workflows

Workflow	Se dispara con	Produce	Permisos
ci.yml	Push y PR a main; manual	Nada. Solo verifica	contents: read
pages.yml	Push a main que toque site/, docs/screenshots/, assets/branding/, pubspec.yaml, tool/build_site.mjs, tool/app_version.mjs o el propio workflow; manual	La landing en GitHub Pages	contents: read, pages: write, id-token: write
release.yml	Tag v*; manual con el tag como entrada	Cuatro artefactos y SHA256SUMS.txt en GitHub Releases	contents: write

Los tres declaran permisos explícitos y mínimos. Ninguno usa el token con permisos por defecto.

ci.yml — la verificación continua

Un solo job, `quality`, en `ubuntu-latest`, con Flutter 3.44.6 fijado y caché activada:

```

- run: flutter pub get
- run: |
  node tool/validate_repository.mjs
  node tool/validate_curriculum.mjs
  node tool/build_site.mjs
- run: dart format --output=none --set-exit-if-changed lib test tool
- run: flutter analyze
- run: flutter test --reporter expanded

```

Los tres validadores de Node corren **antes** que las compuertas de Flutter, y eso es intencionado: son las más baratas y las que más rápido detectan un error de mantenimiento. Un fallo temprano ahorra minutos de runner.

`build_site.mjs` figura aquí no para publicar, sino porque **falla si la landing no se puede ensamblar**: si falta una captura referenciada o si alguien escribió un número de versión a mano en el HTML. Es una compuerta disfrazada de compilación.

Lo que `ci.yml` **no** hace: no compila Android ni Windows. La compilación real solo ocurre al publicar. **INFERENCIA**: un error que solo se manifieste al compilar para una plataforma no se detecta hasta el momento del tag.

pages.yml — la landing

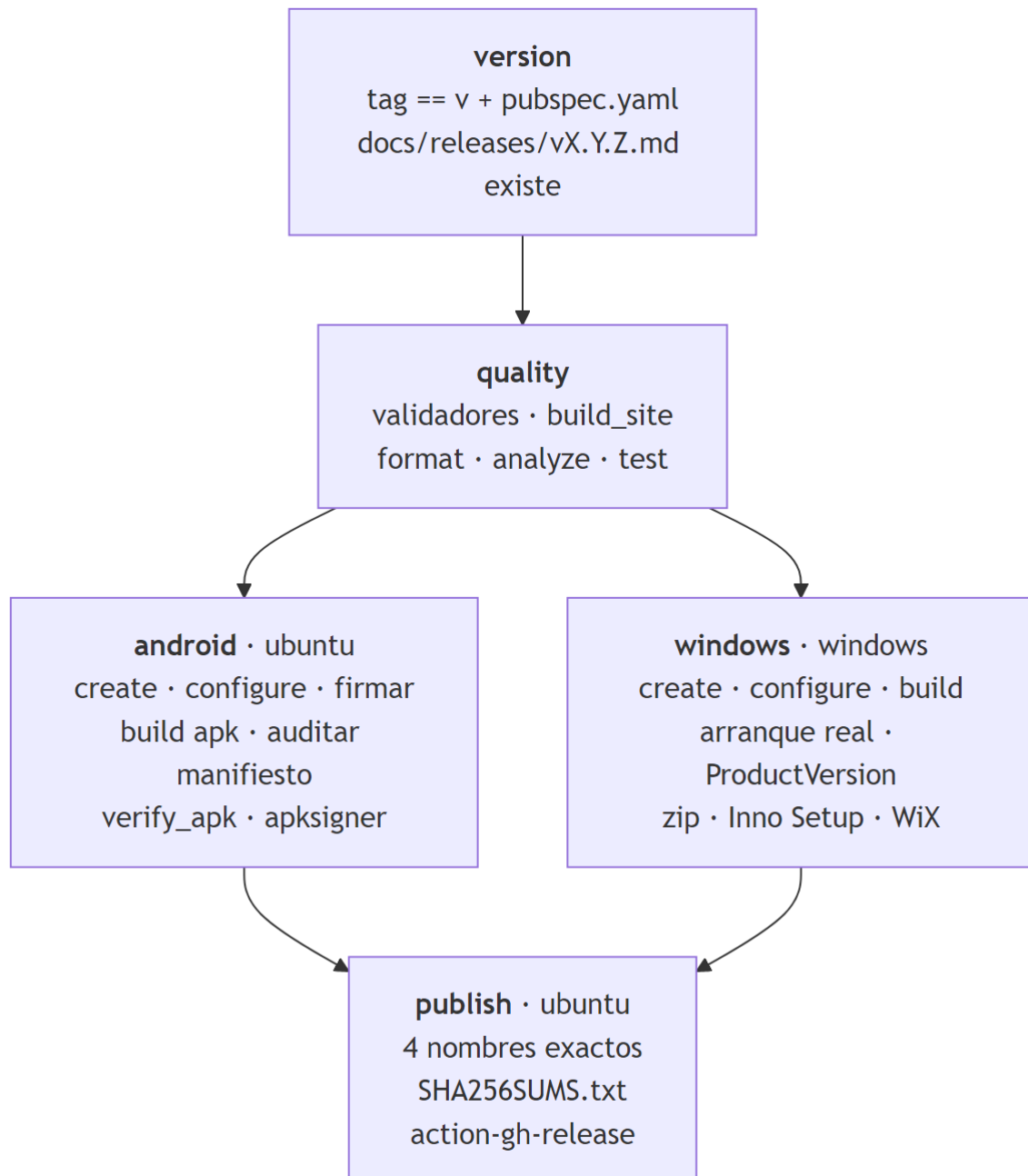


El diagrama muestra la cadena completa. Lo que no muestra: `build_site.mjs` **aborta con código 1** si ningún archivo de `site/` contiene `__APP_VERSION__`, si no hay capturas, o si el HTML referencia una imagen que no llegó al directorio de salida. La landing publicada nunca puede tener un hueco roto ni ofrecer la descarga de una versión anterior.

`concurrency: group: pages` con `cancel-in-progress: true`: dos publicaciones seguidas no se pisan y la última gana.

release.yml — la publicación

Cinco jobs. El grafo de dependencias importa tanto como su contenido.



El diagrama muestra el orden y el paralelismo entre Android y Windows. Lo que no muestra: **cada job rechaza en vez de avisar**. No hay pasos informativos.

Job `version` — la compuerta más barata

```
APP_VERSION="$(node tool/app_version.mjs version)"
if [ "$TAG_NAME" != "v$APP_VERSION" ]; then
  echo "::error::El tag $TAG_NAME no corresponde a la versión v$APP_VERSION..."
  exit 1
fi
```

Sin instalar Flutter, en segundos, impide el fallo que el CHANGELOG documenta haber tenido: un tag `v0.2.0` publicando archivos llamados `0.1.0` porque la versión estaba escrita a mano en el workflow.

También exige que existan las notas `docs/releases/<tag>.md` y las pasa como salida al job `publish`, que las usa como cuerpo de la release.

Job android

Paso	Qué comprueba o produce
<code>flutter create + configure_platforms.mjs</code>	Regenera android/ con la identidad correcta y <code>minSdk = 24</code>
Preparar firma	Usa los secrets si existen; si no, genera una clave efímera con <code>keytool</code> y emite un <code>::warning::</code>
<code>flutter build apk --release</code>	APK universal
<code>grep -R</code> sobre android/app/src	Aborta si aparece <code>CAMERA</code> o <code>RECORD_AUDIO</code> en el manifiesto fuente
<code>node tool/verify_apk.mjs</code>	Paquete, <code>versionName</code> , <code>versionCode</code> , <code>minSdkVersion</code> , permisos del manifiesto fusionado , ABIs y currículo empaquetado
<code>apksigner verify --verbose</code>	Que la firma sea válida
<code>cp</code> con el nombre final	PanueloAlViento-<versión>-Android.apk

La verificación del APK es la más valiosa de toda la cadena: extrae el currículo del paquete, cuenta niveles y clases y compara su SHA-256 con el del repositorio. Detecta el fallo que un build en verde no ve: un artefacto que compila, se firma, cuadra en checksum y se instala **sin contenido**.

Job windows

Paso	Qué comprueba o produce
<code>flutter build windows --release</code>	Ejecutable, DLL y carpeta <code>data</code>
Prueba de arranque	Lanza el <code>.exe</code> , espera 8 s y falla si terminó con código distinto de cero
<code>ProductVersion</code>	Aborta si el ejecutable no declara la versión del manifiesto
<code>Compress-Archive</code>	El ZIP portable, con la carpeta <code>Release</code> completa
<code>choco install innosetup wixtoolset</code>	Herramientas de empaquetado
<code>ISCC.exe</code>	El instalador <code>.exe</code> , con la versión inyectada por <code>/DMyAppVersion</code>
<code>heat + candle + light</code>	El <code>.msi</code>
Verificar artefactos	Exige los tres nombres exactos y rechaza cualquier archivo inesperado

La prueba de arranque es un detalle poco común y muy útil: comprueba que el binario **abre**, no solo que compila.

El rechazo de archivos inesperados explica el commit «Separa símbolos de los artefactos Windows»: el `.wixpdb` que produce WiX se dirige a `build/wix/` precisamente para no aparecer en `release-artifacts` y hacer fallar el paso.

Job publish

Descarga los artefactos de los dos jobs anteriores, comprueba que estén los cuatro nombres exactos, que no sobre ninguno con otra versión, calcula `sha256sum * > SHA256SUMS.txt` y publica con `softprops/action-gh-release@v2` usando las notas de `docs/releases/`.

`fail_on_unmatched_files: true`: si un patrón no encuentra archivos, falla.

Firma de Android: el punto abierto

Escenario	Qué ocurre
Los cuatro secrets configurados	Firma permanente. Las actualizaciones se instalan sobre la anterior y conservan el avance
Alguno falta	<code>keytool</code> genera una clave efímera, se emite un <code>::warning::</code> y el APK no se puede actualizar

Estado en 0.1.0: la segunda rama. `../VALIDATION_RESULT.md` lo confirma midiendo la firma del APK publicado. La consecuencia para las personas usuarias —desinstalar y perder el avance— está declarada en cinco documentos y en la propia landing. El `../ROADMAP.md` lo pone como prioridad inmediata.

Operación de la landing

```
node tool/build_site.mjs          # ensambla en build/site
node tool/build_site.mjs --serve  # además la sirve en 127.0.0.1:8080
```

Variables: `SITE_OUT` para el destino, `SITE_PORT` para el puerto. El servidor comprueba que la ruta resuelta esté dentro del directorio de salida antes de servir, lo que evita servir archivos de fuera.

Regenerar las capturas

```
node tool/capture_screenshots.mjs
node tool/capture_screenshots.mjs --keep # conserva el directorio temporal
```

Compila la aplicación para **web** —la única superficie que se puede renderizar sin Visual Studio ni un dispositivo Android—, la sirve en localhost y conduce un Chrome sin ventana por el protocolo DevTools. La interfaz, el currículo y el estado son los mismos que en Android y Windows, así que lo que se captura es lo que se ve allí. Requiere Flutter en el `PATH` y Chrome instalado, o `CHROME_BIN` apuntando al ejecutable.

`CONTRIBUTING.md` pide regenerarlas cuando un cambio afecte a la interfaz.

Regenerar los PDF de esta documentación

```
python tool/build_docs_pdf.py          # los 20 + el consolidado
python tool/build_docs_pdf.py --only 03-architecture.md # iterar sobre uno
python tool/build_docs_pdf.py --no-mermaid # degradar los diagramas
```

Qué hace

1. Ordena los `.md` de `docs/system-documentation/`: la portada primero, después los numéricos.
2. Por cada documento: extrae cada bloque ````mermaid`, lo sustituye por un marcador que el conversor de Markdown no toca, y lo rasteriza con `mmdc` **cacheando por hash de su código fuente** en `.mermaid-cache/`. La segunda ejecución solo rehace lo que cambió.
3. Convierte Markdown a HTML con tablas y bloques de código.
4. Restaura los marcadores como imágenes de ancho fijo.
5. Compone portada, índice de secciones, cuerpo y pie, con hoja de estilo propia.
6. Escribe el PDF y **aborta si sale de 0 bytes**.
7. Repite concatenando todos los cuerpos para el consolidado.
8. Informa cuántos diagramas se rasterizaron y cuántos degradaron a texto.

Requisitos

Requisito	Si falta
<code>pip install markdown xhtml2pdf</code>	El script no arranca
<code>npm i -g @mermaid-js/mermaid-cli</code>	Los diagramas se incluyen como código fuente con un aviso visible, y el resumen final lo dice. Nunca degrada en silencio
Fuentes DejaVu, Arial o Segoe UI	Se usan las fuentes base de PDF y se avisa de que signos como <code>→ ≥ ● ★</code> podrían no dibujarse

Decisiones del generador que conviene conocer

- **La versión y el commit se leen del repositorio.** `pubspec.yaml` con la misma expresión regular que `tool/app_version.mjs`, y `git rev-parse --short HEAD`. Una portada con números escritos a mano queda obsoleta en el primer commit.
- **Las fuentes se registran por API de reportlab**, no con `@font-face` en el CSS: `xhtml2pdf` trata el `src` de una regla `@font-face` como recurso remoto, lo descarga a un temporal y `reportlab` falla al abrirlo.
- **Las tablas no usan `-pdf-keep-in-frame-mode: shrink`.** Esa opción encoge la fuente de toda la tabla hasta que quepa, y una tabla de cinco columnas acaba ilegible. Es preferible que el texto salte de línea dentro de la celda.
- **El índice se compone con párrafos y no con listas**, y solo indexa los `##`. `xhtml2pdf` da a cada elemento de lista una caja con separación generosa, y un índice de treinta entradas ocupaba más que la sección que anunciaba.
- **En Windows `mmdc` es un `.cmd`** y `Node ≥ 20.12` se niega a lanzarlo sin shell; el script lo detecta y lo invoca a través de `cmd /c`.

`.mermaid-cache/` está en `.gitignore`. Los PDF sí se versionan, y `.gitattributes` los declara `binary` para que ninguna normalización de finales de línea los toque.

Continuar por

- [12 · Pruebas y calidad](#) para qué protege cada compuerta.
- [14 · Troubleshooting](#) si algo de esto falla.
- [./RELEASE-CHECKLIST.md](#) para los criterios humanos de publicación.

Contenido de este documento

Entorno y arranque

Currículo y contenido

Persistencia

Ritmo

Compuertas y CI

Documentación y PDF

Índice rápido

14 · Troubleshooting

Cada entrada: síntoma → causa → diagnóstico → solución → archivos → **riesgo de aplicar la solución.**

Entorno y arranque

T-01 · «No supported devices connected» o no aparece Windows ni Android

Causa. Las carpetas `android/` y `windows/` no existen. `.gitignore` las excluye porque se generan al preparar el proyecto.

Diagnóstico.

```
ls -d android windows 2>&1 # No such file or directory
grep -n "^/android/" .gitignore
```

Solución. Ejecutar el bootstrap correspondiente:

```
./tool/bootstrap.ps1 # Windows
```

```
./tool/bootstrap.sh # Linux y macOS
```

Archivos. `tool/bootstrap.ps1`, `tool/bootstrap.sh`, `tool/configure_platforms.mjs`, `.gitignore`

Riesgo. Ninguno. Es el flujo previsto y los scripts abortan al primer fallo. Se ejecutan `flutter analyze` y `flutter test` al final, así que tarda varios minutos.

T-02 · No se encontró ninguna plataforma generada

Causa. Se ejecutó `node tool/configure_platforms.mjs` sin haber ejecutado antes `flutter create`. El script lanza a propósito en vez de terminar en silencio.

Diagnóstico. El mensaje es explícito y viene del final de `configure_platforms.mjs`.

Solución. Ejecutar el bootstrap completo, no el configurador suelto.

Archivos. `tool/configure_platforms.mjs:68`

Riesgo. Ninguno.

T-03 · «No pudimos abrir la ruta de aprendizaje» al abrir la app

Causa. El arranque lanzó una excepción. Tres posibilidades: el activo del currículo no está en el paquete, el JSON es inválido, o las preferencias del sistema no se pudieron abrir.

Diagnóstico. En una ejecución de desarrollo, el error real se imprime porque `main` lo pasa por `FlutterError.reportError`. Sobre un artefacto instalado, la comprobación es externa:

```
node tool/verify_apk.mjs <apk>      # ¿llegó el currículo al paquete?
node tool/validate_curriculum.mjs    # ¿es válido el JSON del repositorio?
```

Solución. Si el JSON es inválido, corregirlo y volver a compilar. Si el activo falta, revisar la sección `assets` de `pubspec.yaml`. Si es el almacenamiento, reinstalar.

Archivos. `lib/main.dart:9-31`, `lib/data/curriculum_repository.dart`, `pubspec.yaml`

Riesgo. Bajo. Si se decidiera capturar la excepción más abajo para «que no falle», se destruiría la pantalla de recuperación y volvería la pantalla en blanco. No hacerlo.

Currículo y contenido

T-04 · Currículo inválido: <lista de errores>

Causa. El JSON incumple una de las reglas estructurales.

Diagnóstico. La salida enumera cada incumplimiento con el identificador de la clase afectada. Los mensajes más frecuentes:

Mensaje	Qué corregir
Se esperaban 8 niveles y hay N	Cardinalidad de <code>levels</code>
<code><id></code> debe tener 3 clases	Cardinalidad de <code>lessons</code> en ese nivel
<code><id></code> : actividades=N, clase=M	Los minutes no suman <code>durationMinutes</code>
Orden repetido: N	Dos clases con el mismo <code>order</code>
Falta el orden N	Hueco en la numeración 1–24
<code><id></code> no tiene <code><campo></code>	Campo de texto vacío o ausente

Solución. Corregir `assets/content/curriculum.json` y volver a validar.

Archivos. `assets/content/curriculum.json`, `tool/validate_curriculum.mjs`

Riesgo. Medio si se «arregla» relajando el validador en vez del dato: se perdería la garantía y las tres pruebas de integridad fallarían igualmente.

T-05 · Una clase muestra un esquema de piso que no corresponde

Causa. Su campo `diagram` tiene un valor sin rama propia en el pintor. En 0.1.0 ocurre con `pair` (4 clases) y `free` (3 clases), que caen ambos en la rama por defecto.

Diagnóstico.

```
node -e "const c=require('./assets/content/curriculum.json');
const m={}; for(const l of c.levels) for(const s of l.lessons)
m[s.diagram]=(m[s.diagram]||0)+1; console.log(m)"
grep -n "case " lib/widgets/movement_diagram.dart
```

Solución. Es un hallazgo abierto, **no corregido en esta documentación**. Hay dos caminos y requieren una decisión: añadir ramas de dibujo para `pair` y `free`, o cambiar el valor de esas clases a uno que sí tenga dibujo. Ver [15 · Riesgos](#), R-02.

Archivos. `lib/widgets/movement_diagram.dart`, `assets/content/curriculum.json`

Riesgo. Bajo en el código; **medio en lo pedagógico**: el dibujo comunica una intención de movimiento y cambiarlo altera lo que se enseña. Debería revisarlo quien redactó el currículo.

Persistencia

T-06 · «No fue posible guardar el avance en este dispositivo»

Causa. `SharedPreferences.setStringList` devolvió `false`. `ProgressRepository` lo convierte en `StateError` en lugar de simular éxito.

Diagnóstico. Comprobar espacio en disco y permisos del perfil de usuario. En Windows, que el perfil no sea de solo lectura.

Solución. Liberar espacio o reinstalar. El mensaje es correcto: la clase **no** quedó guardada, y la interfaz no la muestra como completada.

Archivos. `lib/data/progress_repository.dart:24-30`, `lib/state/app_state.dart`

Riesgo. Alto si se «resuelve» eliminando el `throw`. Sin él, `AppState` no tiene forma de saber que la escritura falló y la interfaz mostraría avance que el disco no respalda. Es exactamente lo que impiden las tres pruebas de `test/app_state_test.dart`.

T-07 · El avance desapareció tras actualizar en Android

Causa. La versión nueva se firmó con una clave distinta. Android exige desinstalar, y al desinstalar se borran las preferencias.

Diagnóstico.

```
apksigner verify --print-certs <apk-nuevo>
apksigner verify --print-certs <apk-anterior>
```

Si los certificados difieren, esa es la causa.

Solución. Preventiva, no correctiva: configurar los cuatro secrets de firma descritos en [../BUILD_MOBILE.md](#) antes de publicar la siguiente versión. Una vez perdido, el avance no se recupera: no hay copia en la nube ni exportación en 0.1.0.

Archivos. `.github/workflows/release.yml:99-130`, `tool/configure_android_signing.mjs`

Riesgo. Ninguno al configurar la firma. El riesgo está en **no** hacerlo: cada release sin secrets genera una clave irrepetible y repite el problema.

T-08 · El porcentaje no llega al 100 % aunque se completaron todas

Causa. `AppState.load()` cruza el avance guardado con los identificadores válidos. Si el currículo cambió de identificadores, los antiguos se descartan.

Diagnóstico. Comparar los `id` guardados con los del currículo actual. En desarrollo:

```
debugPrint(state.completedLessonIds.toString());
```

Solución. Comportamiento correcto y deliberado: evita que un porcentaje quede atascado por clases retiradas. Si el cambio de identificadores no era intencionado, revertirlo en el JSON.

Archivos. `lib/state/app_state.dart`, `assets/content/curriculum.json`

Riesgo. Ninguno. Nótese que el descarte es solo en memoria: los identificadores huérfanos siguen en disco hasta la próxima escritura.

Ritmo

T-09 · El clic sigue sonando después de salir de la pestaña Ritmo

Causa. `HomeShell` mantiene las cuatro pestañas en un `IndexedStack`, que no destruye las no visibles.

`_RhythmLabScreenState.dispose()` no se ejecuta al cambiar de pestaña y el temporizador sigue programando pulsos.

Diagnóstico. Verificado en pruebas durante este análisis: iniciar el pulso, cambiar a Inicio y contar las señales emitidas. Se emiten. Al volver a Ritmo, el botón sigue diciendo «Detener».

Solución. Para quien usa la aplicación: pulsar **Detener** antes de cambiar de pestaña. Como cambio de código es un hallazgo abierto y **no corregido**; ver [15 · Riesgos](#), R-01.

Archivos. `lib/screens/home_shell.dart:40-43`, `lib/screens/rhythm_lab_screen.dart`, `docs/PERMISSIONS.md`

Riesgo de corregirlo. Medio, y depende del camino. Sustituir el `IndexedStack` perdería la posición de desplazamiento de las cuatro pestañas. Detener el pulso al cambiar de pestaña requiere que `HomeShell` conozca el estado del laboratorio, lo que introduce acoplamiento donde hoy no lo hay. La opción menos invasiva es un `TickerMode` o una notificación de visibilidad. Sea cual sea, hay que actualizar `docs/PERMISSIONS.md` o el código, porque hoy se contradicen.

T-10 · No vibra

Causa. Tres posibles, en este orden de probabilidad: el equipo no tiene motor háptico (habitual en escritorio), el interruptor **Vibración** está apagado, o el pulso actual no es un acento —la háptica solo se emite en los acentos—.

Diagnóstico. Comprobar que el interruptor esté encendido y observar si la vibración coincide con los círculos destacados (1 y 4 en 3+3; 1, 3 y 5 en 2+2+2).

Solución. Ninguna en escritorio: es una limitación de hardware declarada en `../ACCESSIBILITY.md`. La aplicación funciona igual sin ella.

Archivos. `lib/screens/rhythm_lab_screen.dart`

Riesgo. Ninguno.

Compuertas y CI

T-11 · `dart format` falla en CI pero en local no

Causa. El formateador de Dart cambia entre versiones. CI fija Flutter 3.44.6 (Dart 3.12.2).

Diagnóstico.

```
flutter --version
grep -n "FLUTTER_VERSION" .github/workflows/ci.yml
```

Solución. Formatear con la misma versión que CI, tal como pide [../CONTRIBUTING.md](#):

```
dart format lib test tool
dart format --output=none --set-exit-if-changed lib test tool
```

Archivos. `.github/workflows/ci.yml:14`, `.github/workflows/release.yml:23`

Riesgo. Ninguno. Aviso: la versión de Flutter está escrita en **dos** archivos; cambiar uno sin el otro deja CI y release en desacuerdo.

T-12 · Repositorio inválido: ...

Causa. `validate_repository.mjs` detectó una incoherencia. Es la compuerta que más falla, porque comprueba la parte que se desincroniza sola.

Mensaje	Qué corregir
Falta docs/releases/vX.Y.Z.md	Escribir las notas de esa versión
CHANGELOG.md encabeza la versión A y pubspec.yaml declara B	Encabezar el CHANGELOG con la versión del manifiesto
README.md no nombra el artefacto ...	Actualizar los cuatro nombres del README
README.md muestra docs/... pero el archivo no existe	Regenerar las capturas o corregir la ruta
Las notas de la versión usan enlaces relativos...	Convertirlos en absolutos: en GitHub Releases los relativos no resuelven
site/index.html debe usar <code>__APP_VERSION__</code> ...	Restituir el marcador
site/index.html nombra artefactos con una versión fija	Sustituir el número por <code>__APP_VERSION__</code>
Esta versión no debe usar <code><cadena></code>	Se introdujo una dependencia o API prohibida

Archivos. `tool/validate_repository.mjs`, y el archivo que nombre el mensaje

Riesgo. Alto si se resuelve relajando el validador. Cada comprobación existe porque el fallo correspondiente es fácil de cometer y difícil de notar. El mensaje del código lo justifica en cada caso.

T-13 · Ningún archivo de site/ usa __APP_VERSION__

Causa. Alguien escribió el número de versión a mano en la landing.

Diagnóstico.

```
grep -c "__APP_VERSION__" site/index.html # debe dar 5
grep -n "PañueloAlViento-[0-9]" site/index.html # no debe dar nada
```

Solución. Restituir el marcador en los cinco lugares.

Archivos. site/index.html, tool/build_site.mjs:47-52

Riesgo. Ninguno al corregirlo. El riesgo es el que la comprobación evita: una landing que ofrece descargas con 404 tras publicar una versión nueva, y nadie se entera hasta que alguien la intenta.

T-14 · No se encontró aapt2

Causa. verify_apk.mjs no localizó las build-tools de Android.

Diagnóstico.

```
echo $ANDROID_SDK_ROOT
ls "$ANDROID_SDK_ROOT/build-tools"
```

Solución. Instalar las build-tools, o apuntar directamente al binario:

```
AAPT2=/ruta/a/aapt2 node tool/verify_apk.mjs <apk>
```

Archivos. tool/verify_apk.mjs:33-59

Riesgo. Ninguno. Sin aapt2 no se puede verificar un APK, pero el resto de las compuertas funciona.

T-15 · Artefactos inesperados en la release de Windows

Causa. El job windows exige exactamente tres archivos en release-artifacts y rechaza cualquier otro. El caso conocido es el .wixpdb que genera WiX.

Diagnóstico. El mensaje enumera los archivos sobrantes.

Solución. Dirigir los archivos auxiliares fuera de release-artifacts, como ya hace light.exe con -pdbout build/wix/....

Archivos. .github/workflows/release.yml:222-237

Riesgo. Alto si se resuelve relajando la comprobación: es lo que impide publicar símbolos de compilación junto a los binarios.

Documentación y PDF

T-16 · Los diagramas salen como código fuente en el PDF

Causa. mmdc no está instalado o no se pudo lanzar. El generador **avisa**; no degrada en silencio.

Diagnóstico. El resumen final dice N degradados a texto y añade un aviso explícito.

Solución.

```
npm install -g @mermaid-js/mermaid-cli
python tool/build_docs_pdf.py
```

Archivos. `tool/build_docs_pdf.py`

Riesgo. Ninguno. En Windows, `mmdc` es un `.cmd` y Node ≥ 20.12 se niega a lanzarlo sin shell; el script ya lo invoca a través de `cmd /c`.

T-17 · Una tabla del PDF sale ilegible o desbordada

Causa. Casi siempre, una tabla Markdown con la fila de cabecera vacía (`| |` seguido de `---|---`). Los motores de PDF colapsan los anchos de columna y el texto sale a una palabra por línea.

Diagnóstico. Buscar tablas sin encabezado real en el Markdown de origen.

Solución. Dar un encabezado real en el Markdown (`| Aspecto | Detalle |`). No pelearse con el CSS: el problema está en el dato, no en el estilo.

Archivos. El `.md` correspondiente, `tool/build_docs_pdf.py`

Riesgo. Ninguno. Nótese que el README principal del repositorio sí usa una tabla sin encabezado para la galería de capturas; es correcto en GitHub y sería un problema solo si ese archivo se convirtiera a PDF con este generador, cosa que no ocurre.

T-18 · Signos como $\rightarrow \geq \bullet \star$ no aparecen en el PDF

Causa. No se resolvió ninguna fuente TrueType con cobertura amplia y se usaron las fuentes base de PDF.

Diagnóstico. El script imprime la tipografía usada al empezar y avisa al final si cayó en el respaldo.

Solución. Instalar DejaVu, o ejecutar donde haya Arial y Consolas. El script también busca las fuentes que empaqueta matplotlib, que es el respaldo más fiable.

Archivos. `tool/build_docs_pdf.py`

Riesgo. Ninguno.

Índice rápido

Síntoma	Entrada
No hay dispositivos ni plataformas	T-01
No se encontró ninguna plataforma generada	T-02
Pantalla de error al abrir la app	T-03
Currículo inválido	T-04
Esquema de piso equivocado	T-05
No se guarda el avance	T-06

El avance desapareció al actualizar	T-07
El porcentaje no llega al 100 %	T-08
El clic sigue sonando fuera de Ritmo	T-09
No vibra	T-10
dart format falla solo en CI	T-11
Repositorio inválido	T-12
Versión escrita a mano en la landing	T-13
No se encontró aapt2	T-14
Artefactos inesperados en Windows	T-15
Diagramas como texto en el PDF	T-16
Tabla ilegible en el PDF	T-17
Signos ausentes en el PDF	T-18

Contenido de este documento

Resumen

- R-01 · El pulso sobrevive al cambio de pestaña · Alta
 - R-02 · Dos valores de diagram sin dibujo propio · Media
 - R-03 · El ROADMAP da por pendiente la landing ya publicada · Media
 - R-04 · El nivel 8 tiene dos nombres · Media
 - R-05 · Credenciales literales en el workflow de release · Media
 - R-06 · Cifras del currículum escritas a mano en la interfaz · Media
 - R-07 · «Clases de 10 a 15 minutos» cuando todas duran 12 · Baja
 - R-08 · Validador de currículum duplicado · Baja
 - R-09 · El control de cadenas prohibidas no cubre todo lib/ · Baja
 - R-10 · La versión de Flutter está en dos archivos · Baja
 - R-11 · El laboratorio de ritmo no tiene ninguna prueba · Media
 - R-12 · docs/CONTENT_PRODUCTION.md sin protección · Baja
 - R-13 · La velocidad inicial no cae en una división del deslizador · Baja
 - R-14 · Una captura versionada sin uso · Baja
 - R-15 · Los colores del esquema de piso no se miden · Baja
 - R-16 · Sin esquema JSON formal ni migración del currículum · Media
 - R-17 · Sin escaneo de dependencias ni de secretos en CI · Baja
- Lo que está bien y hay que proteger
- Continuar por

15 · Riesgos y deuda técnica

Este documento es informativo. Ningún hallazgo se ha corregido. El encargo de esta documentación fue documentar y registrar, no arreglar. La única excepción son los comentarios de documentación añadidos al código, que no cambian comportamiento.

Cada hallazgo lleva severidad, impacto, probabilidad, evidencia, ubicación, recomendación y prioridad. La evidencia es reproducible.

Resumen

Id	Hallazgo	Severidad	Prioridad
R-01	El pulso del laboratorio sobrevive al cambio de pestaña, contra lo que afirma la documentación	Alta	1
R-02	Dos valores de diagram no tienen dibujo propio; la documentación anuncia seis diagramas	Media	2
R-03	El ROADMAP sigue listando como pendiente la landing que ya está publicada	Media	3
R-04	El nombre del nivel 8 difiere entre el currículo y dos documentos	Media	4
R-05	Credenciales literales en el workflow de release	Media	5
R-06	Cifras del currículo escritas a mano en la interfaz	Media	6
R-07	«Clases de 10 a 15 minutos» cuando las 24 duran 12	Baja	7
R-08	Validador de currículo duplicado en dos lenguajes	Baja	8
R-09	El control de cadenas prohibidas cubre 7 de 15 archivos de lib/	Baja	9
R-10	La versión de Flutter está escrita en dos archivos	Baja	10
R-11	El laboratorio de ritmo no tiene ninguna prueba	Media	11
R-12	docs/CONTENT_PRODUCTION.md no está protegido por el validador	Baja	12
R-13	La velocidad inicial no cae en una división del deslizador	Baja	13
R-14	Una captura versionada no se usa en ninguna superficie	Baja	14
R-15	Los colores del esquema de piso no se miden en las pruebas de contraste	Baja	15

R-16	Sin esquema JSON formal ni migración del currículo	Media	16
R-17	Sin escaneo de dependencias ni de secretos en CI	Baja	17

R-01 · El pulso sobrevive al cambio de pestaña · Alta

Impacto. Una niña o un niño inicia el metrónomo, cambia a Ruta y la aplicación sigue emitiendo un clic del sistema en cada pulso, indefinidamente, sin ningún control visible en pantalla. Más grave que la molestia: **contradice una afirmación explícita de la documentación de permisos**, que es precisamente el documento sobre el que descansa la confianza de una familia.

docs/PERMISSIONS.md, «Secuencia de activación del laboratorio», paso 6:

Detener, cambiar de pantalla o cerrar la app cancela el temporizador.

Probabilidad. Alta. Cualquier persona que use el laboratorio y luego navegue sin pulsar Detener.

Causa. HomeShell mantiene las cuatro secciones en un IndexedStack, que construye y conserva a los cuatro hijos y solo pinta el seleccionado. El State de las pestañas invisibles sigue vivo, dispose() no se ejecuta y _scheduleNextPulse sigue encadenando temporizadores.

Evidencia. Verificado con una prueba de widget temporal durante este análisis: se inició el pulso, se cambió a Inicio y se contaron las llamadas a SystemSound.play capturadas en el canal de plataforma simulado.

SONDA A · clics inmediatamente tras Comenzar: 3
 SONDA B · clics tras 3 s en la pestaña Ritmo: 5
 SONDA C · pestaña Inicio visible: true
 SONDA D · clics emitidos MIENTRAS SE ESTÁ EN INICIO: 2
 SONDA F · el botón dice: Detener (seguía sonando)

La prueba **no se dejó en el repositorio**: documentar no es corregir, y añadir una prueba que falla habría cambiado el estado de las computas.

Ubicación. lib/screens/home_shell.dart:40-43 · lib/screens/rhythm_lab_screen.dart:325-360 · docs/PERMISSIONS.md

Recomendación. Primero decidir cuál de las dos afirmaciones es la correcta. Si el comportamiento debe cambiar, la opción menos invasiva es envolver la sección no visible en un TickerMode o notificar la visibilidad al laboratorio; sustituir el IndexedStack perdería la posición de desplazamiento de las cuatro pestañas, que es justamente lo que se buscaba con él. Si el comportamiento es deseado —permitir practicar con el pulso mientras se lee una clase—, hay que corregir el paso 6 de PERMISSIONS.md y explicarlo en la interfaz.

Esfuerzo. 1–2 horas el código; 15 minutos la corrección documental.

R-02 · Dos valores de diagram sin dibujo propio · Media

Impacto. Siete clases —cuatro con pair, tres con free— muestran el mismo esquema genérico. Quien usa un lector de pantalla recibe **dos descripciones distintas del mismo dibujo**, porque el camino semántico sí distingue pair. Varios documentos anuncian seis diagramas distintos.

Probabilidad. Certeza: ocurre en 7 de las 24 clases.

Evidencia.

```
$ node -e "const c=require('./assets/content/curriculum.json');
const m={}; for(const l of c.levels) for(const s of l.lessons)
m[s.diagram]=(m[s.diagram]||0)+1; console.log(m)"
{ pair: 4, steps: 4, circle: 4, free: 3, wave: 2, semicircle: 3, eight: 4 }
```

`MovementPainter.paint` tiene ramas para `circle`, `eight`, `semicircle`, `steps` y `wave`, más `default`. `MovementDiagram._description` tiene ramas para esos cinco más `pair`, más un comodín.

Afirmaciones afectadas:

Documento	Dice
README.md	«seis patrones dibujados en Flutter»; «vuelta, ocho, medialuna, pasos, diálogo y pañuelo»
CHANGELOG.md, 0.1.0	«Diagramas semánticos originales para vuelta, ocho, medialuna, pasos, pareja y movimiento del pañuelo»
docs/releases/v0.1.0.md	«Diagramas semánticos de vuelta, ocho, medialuna, pasos, diálogo y pañuelo»

Ninguno menciona `free`, que existe en tres clases.

Ubicación. `lib/widgets/movement_diagram.dart` · `assets/content/curriculum.json`

Recomendación. Añadir ramas de dibujo para `pair` y `free`, y una prueba que recorra las 24 clases y afirme que cada valor de `diagram` tiene rama propia. Esa prueba habría detectado el hallazgo. Alternativamente, decidir que ambos comparten esquema a propósito y decirlo en la documentación.

Esfuerzo. 2–3 horas, más revisión pedagógica de qué debe dibujar cada uno.

R-03 · El ROADMAP da por pendiente la landing ya publicada · Media

Impacto. El plan de la versión 0.2 promete algo que ya está hecho. Quien lea el roadmap para decidir prioridades trabajará con información falsa.

Evidencia.

```
$ grep -n "Landing pública en GitHub Pages" ROADMAP.md
39:- Landing pública en GitHub Pages, la única superficie que violín y guitarra
    tienen y esta aplicación todavía no.
```

La línea está bajo «### 0.2 — Piloto y portabilidad de datos». Pero `site/` existe, `.github/workflows/pages.yml` la publica, el README enlaza a la página, y el commit `6efae74` —el analizado— se titula «Publica la landing en GitHub Pages».

Ubicación. `ROADMAP.md:39`

Recomendación. Mover esa línea a la sección «Publicado» o eliminarla. Es el ejemplo de libro de la diferencia entre estado actual y referencia histórica: un CHANGELOG que menciona la landing como novedad es correcto; un roadmap que la lista como pendiente es un error.

Esfuerzo. 5 minutos.

R-04 · El nivel 8 tiene dos nombres · Media

Impacto. Quien lea `docs/CURRICULUM.md` y luego abra la aplicación verá un título distinto en el mismo nivel.

Evidencia.

Fuente	Nombre
<code>assets/content/curriculum.json, levels[7].title</code>	Cuecas de Chile y presentación
<code>README.md:126, diagrama de la ruta</code>	8 · Diversidad y presentación
<code>docs/CURRICULUM.md:20, tabla de ocho semanas</code>	Diversidad y presentación

El dato es la fuente de verdad: es lo que se dibuja en pantalla.

Ubicación. `assets/content/curriculum.json` · `README.md:126` · `docs/CURRICULUM.md:20`

Recomendación. Elegir uno y unificar. Ningún validador comprueba que los títulos de los niveles coincidan entre el dato y la documentación; podría añadirse a `validate_repository.mjs` con el mismo criterio con el que ya comprueba los nombres de artefacto del README.

Esfuerzo. 10 minutos la corrección; 30 minutos añadir la comprobación.

R-05 · Credenciales literales en el workflow de release · Media

Impacto. `.github/workflows/release.yml` contiene, en la rama que se ejecuta cuando los secrets de firma no están configurados, un alias y dos contraseñas escritas literalmente. Sirven para crear un almacén de claves con `keytool` que vive solo en el runner y se descarta al terminar.

No es la filtración de un secreto real: la clave no existía antes y no persiste después. Sí es material con forma de credencial en un repositorio público, que puede disparar escáneres de secretos, y sobre todo **señala el estado que hay que resolver**: mientras esa rama se ejecute, cada release produce un APK firmado con una clave distinta e irrepetible.

Probabilidad. Certeza mientras los secrets no estén configurados. `./VALIDATION_RESULT.md` confirma midiendo el APK publicado que se usó la rama efímera.

Evidencia. Barrido sobre el árbol versionado, con los valores enmascarados:

```
.github/workflows/release.yml:111: KEY_PASSWORD= <VALOR OMITIDO> ← desde secret
.github/workflows/release.yml:112: STORE_PASSWORD= <VALOR OMITIDO> ← desde secret
.github/workflows/release.yml:116: KEY_PASSWORD= <VALOR OMITIDO> ← literal
.github/workflows/release.yml:117: STORE_PASSWORD= <VALOR OMITIDO> ← literal
```

Ningún otro archivo del repositorio contiene material con forma de credencial. Un barrido de patrones de token (`sk-`, `ghp_`, `AKIA`, claves privadas PEM) no devolvió coincidencias.

Ubicación. `.github/workflows/release.yml:114-123`

Recomendación. Configurar los cuatro secrets, que es lo que el `./ROADMAP.md` ya marca como prioridad inmediata. Al hacerlo, la rama de respaldo deja de ejecutarse y los literales dejan de tener función; podrían sustituirse por un valor generado al vuelo.

Esfuerzo. 1 hora, más la custodia segura del almacén de claves.

R-06 · Cifras del currículo escritas a mano en la interfaz · Media

Impacto. Si el currículo cambia de tamaño, dos textos de la interfaz mienten sin que ninguna compuerta lo detecte. Uno de ellos aparece en un diálogo de confirmación de borrado, que es el peor sitio para una cifra incorrecta.

Evidencia.

```
// lib/screens/home_tab.dart
title: 'Explorar las 24 clases',

// lib/screens/progress_tab.dart
content: const Text('Se desmarcarán las 24 clases en este dispositivo.'),
```

`AppState.totalCount` existe y devuelve el valor real. En `home_tab.dart` el `AppState` ya está disponible como campo; en `progress_tab.dart` también.

Ubicación. `lib/screens/home_tab.dart:105` · `lib/screens/progress_tab.dart:150`

Recomendación. Interpolarse `state.totalCount`. La cifra dejaría de poder desincronizarse.

Esfuerzo. 15 minutos. Nota: deja de ser texto constante, así que hay que retirar el `const` de esos widgets.

R-07 · «Clases de 10 a 15 minutos» cuando todas duran 12 · Baja

Impacto. Varios documentos describen una variedad de duración que los datos no tienen. No engaña a nadie de forma grave, pero es el tipo de afirmación que envejece mal.

Evidencia.

```
$ node -e "const c=require('./assets/content/curriculum.json');
const d=new Set(c.levels.flatMap(l=>l.lessons).map(s=>s.durationMinutes));
console.log([...d])"
[ 12 ]
```

Las 24 clases duran exactamente 12 minutos; 288 minutos en total.

Documento	Dice
<code>README.md:88</code>	«cada clase dura de 10 a 15 minutos»
<code>docs/PRACTICE_PLAN_8_WEEKS.md:5</code>	«Tres clases de 10 a 15 minutos por semana»
<code>docs/releases/v0.1.0.md</code>	«Clases de 10–15 minutos»
<code>docs/CURRICULUM.md</code>	«24 clases de 12 minutos» — correcto

`test/curriculum_integrity_test.dart` valida el rango 10–15, que es el contrato real; el rango no es incorrecto, solo está infrutilizado.

Ubicación. `README.md:88` · `docs/PRACTICE_PLAN_8_WEEKS.md:5` · `docs/releases/v0.1.0.md`

Recomendación. Decir «12 minutos» donde se describe el producto actual y reservar el rango 10–15 para el contrato que el validador impone. `docs/CURRICULUM.md` ya lo hace bien. Las notas de una versión publicada son referencia histórica y no deberían editarse.

Esfuerzo. 10 minutos.

R-08 · Validador de currículo duplicado · Baja

Impacto. Dos implementaciones de las mismas reglas, en Node y en Dart, que pueden divergir. La de Dart **no la ejecuta ningún workflow ni bootstrap**: solo la mencionan CONTRIBUTING.md y el MASTER_PROMPT.md no versionado. Un cambio de regla aplicado solo a la de Node pasaría inadvertido.

Evidencia. `tool/validate_curriculum.dart` (106 líneas) y `tool/validate_curriculum.mjs` (62 líneas) comprueban lo mismo y producen la misma línea de salida. `grep -rn "validate_curriculum.dart"` solo encuentra CONTRIBUTING.md.

Ubicación. `tool/validate_curriculum.dart` · `tool/validate_curriculum.mjs`

Recomendación. Decidir. Si se conserva la de Dart —tiene sentido para quien ya tiene el toolchain y no quiere Node—, añadirla a `ci.yml`, que ya tiene Flutter instalado y le costaría segundos. Si no, retirarla y actualizar CONTRIBUTING.md.

Esfuerzo. 20 minutos.

R-09 · El control de cadenas prohibidas no cubre todo `lib/` · Baja

Impacto. `validate_repository.mjs` busca las cadenas prohibidas en una lista fija de siete archivos. `lib/` tiene quince. Un archivo nuevo queda fuera del primer control.

El impacto real es bajo: los controles del manifiesto fuente y del manifiesto fusionado sí son exhaustivos y son los que bloquean la release. Pero el primer control da una sensación de cobertura que no tiene.

Ubicación. `tool/validate_repository.mjs:143-158`

Recomendación. Recorrer `lib/` completo con un `readdir` recursivo.

Esfuerzo. 20 minutos. Atención: la lista actual incluye `dart:io` entre las cadenas prohibidas y `test/curriculum_integrity_test.dart` lo importa legítimamente, así que la ampliación debe seguir limitándose a `lib/`.

R-10 · La versión de Flutter está en dos archivos · Baja

Impacto. `FLUTTER_VERSION: "3.44.6"` aparece en `ci.yml:14` y en `release.yml:23`. Actualizar uno sin el otro deja la verificación continua y la publicación compilando con versiones distintas, y el formateador de Dart cambia entre versiones.

Ubicación. `.github/workflows/ci.yml:14` · `.github/workflows/release.yml:23`

Recomendación. El repositorio ya resolvió este patrón para la versión del producto con `tool/app_version.mjs`. Aquí no hay un mecanismo equivalente sencillo en GitHub Actions sin variables de organización; la alternativa barata es una comprobación en `validate_repository.mjs` que exija que ambos valores coincidan.

Esfuerzo. 30 minutos.

R-11 · El laboratorio de ritmo no tiene ninguna prueba · Media

Impacto. Es el componente con más lógica temporal del proyecto —la corrección de deriva sobre Stopwatch, la agrupación de acentos, la emisión condicional de sonido y háptica— y la única forma de comprobar que funciona es abrir la aplicación y escuchar. Una regresión en la planificación no se detectaría en CI.

Es también la razón directa de que R-01 pasara inadvertido.

Evidencia. `grep -rn "RhythmLab\|_scheduleNextPulse\|SystemSound" test/` solo encuentra la comprobación del texto de privacidad en el smoke test.

Ubicación. `test/ · lib/screens/rhythm_lab_screen.dart`

Recomendación. Una prueba de widget con reloj simulado que verifique que el pulso n cae en $n \times 60/\text{bpm}$ incluso tras introducir un retraso artificial, y otra que compruebe que la háptica solo se emite en los acentos. Es el hueco con mejor relación entre esfuerzo y valor de todo el repositorio.

Esfuerzo. 3–4 horas.

R-12 · `docs/CONTENT_PRODUCTION.md` sin protección · Baja

Impacto. Es el único de los 18 documentos de `docs/` que no figura en la lista de archivos esenciales de `validate_repository.mjs`. Borrarlo no rompería ninguna compuerta, aunque el README lo enlaza —y ese enlace quedaría roto sin que nadie lo detecte, porque el validador solo comprueba las **imágenes** que el README referencia, no sus enlaces—.

Ubicación. `tool/validate_repository.mjs:21-37`

Recomendación. Añadirlo a `requiredFiles`. Considerar además comprobar los enlaces relativos del README con el mismo criterio con el que ya se comprueban sus imágenes.

Esfuerzo. 5 minutos añadirlo; 45 minutos la comprobación de enlaces.

R-13 · La velocidad inicial no cae en una división del deslizador · Baja

Impacto. El `Slider` va de 60 a 120 con 12 divisiones, es decir pasos de 5: 60, 65, 70... El valor inicial es 84, que no es ninguno de ellos. Al primer arrastre, la velocidad salta a 85 sin que la persona haya pedido ese cambio.

Evidencia. `lib/screens/rhythm_lab_screen.dart: double _bpm = 84; con min: 60, max: 120, divisions: 12.`

Ubicación. `lib/screens/rhythm_lab_screen.dart:35, 154-159`

Recomendación. Usar 85, o cambiar `divisions` a 60 para pasos de 1 punto. NO DOCUMENTADO EN EL REPOSITORIO: por qué se eligió 84 como velocidad inicial; INFERENCIA: parece un tempo escogido por criterio musical, en cuyo caso la solución correcta es ajustar las divisiones y no el valor.

Esfuerzo. 5 minutos, más una decisión musical.

R-14 · Una captura versionada sin uso · Baja

Impacto. `docs/screenshots/09-escritorio.png` (68 KB) se muestra en el README y `tool/build_site.mjs` la copia a la landing, pero `site/index.html` no la referencia: la galería muestra las ocho primeras. Es peso muerto en el artefacto publicado.

Evidencia.

```
$ grep -c "09-escritorio" site/index.html
0
```

Ubicación. `site/index.html · tool/build_site.mjs:57-65`

Recomendación. Añadirla a la galería —es la única que muestra la barra lateral de escritorio, que es una característica del producto— o excluirla de la copia.

Esfuerzo. 10 minutos.

R-15 · Los colores del esquema de piso no se miden · Baja

Impacto. `test/theme_contrast_test.dart` mide 16 pares de color, todos del `ColorScheme`. `._MovementPainter` pinta su propio fondo y sus propias figuras con `AppColors.red` y `AppColors.blue` **fijos en ambos modos**. Ese par no lo mide nadie.

No es un fallo demostrado —el fondo del lienzo también cambia con el modo—, sino un hueco de cobertura en la única suite que protege la legibilidad.

Ubicación. `lib/widgets/movement_diagram.dart:154-155` · `test/theme_contrast_test.dart`

Recomendación. Extender la suite a los colores del lienzo, reutilizando la función `contrastRatio` que ya existe en el archivo de pruebas.

Esfuerzo. 1 hora.

R-16 · Sin esquema JSON formal ni migración del currículo · Media

Impacto. La clave del almacén de progreso lleva sufijo `_v1`, lo que reserva la posibilidad de migrar, pero **el mecanismo de migración no existe**. El currículo tampoco tiene esquema formal: la disciplina la ponen dos validadores y tres pruebas, todos con reglas escritas a mano.

Consecuencia concreta: cambiar el formato del avance sin escribir la migración dejaría huérfano el progreso de todos los dispositivos instalados.

Ubicación. `lib/data/progress_repository.dart:20` · `assets/content/curriculum.json`

Recomendación. Es un hueco que el propio repositorio ya reconoce: `docs/ARCHITECTURE.md` dice que «una versión futura debería añadir un esquema JSON formal y migraciones entre versiones», y el `../ROADMAP.md` lo pone en la versión 0.2 junto con la exportación de progreso. Se registra aquí para que no se pierda.

Esfuerzo. 1–2 días.

R-17 · Sin escaneo de dependencias ni de secretos en CI · Baja

Impacto. 52 paquetes resueltos y ninguna comprobación automática de vulnerabilidades conocidas. Ninguna comprobación impide confirmar un secreto nuevo. La superficie es pequeña y `pubspec.lock` está versionado —lo que hace las versiones auditable, a diferencia de un repositorio sin lock—, pero nadie las audita.

Ubicación. `.github/workflows/ci.yml`

Recomendación. Un job de auditoría de dependencias y un escáner de secretos son baratos y encajan en `ci.yml`. Ver [11 · Seguridad](#), H-3 y H-4, para las observaciones relacionadas sobre acciones fijadas por etiqueta móvil.

Esfuerzo. 2 horas.

Lo que está bien y hay que proteger

Un informe que solo enumera problemas da una imagen falsa. Estas propiedades son buenas, alguien podría romperlas sin darse cuenta, y merecen defensa explícita.

Propiedad	Dónde	Por qué importa
Persistencia transaccional	AppState + ProgressRepository + 3 pruebas	Impide el fallo más caro del producto: avance que se ve y desaparece. Las pruebas existen para que nadie invierta el orden
La interfaz ProgressStore	lib/data/progress_repository.dart	Sin ella, las tres pruebas anteriores serían imposibles de escribir
Dominio sin dependencias	lib/domain/curriculum.dart	Permite validar el currículo sin Flutter. Un import de Flutter aquí rompería tres pruebas y un validador
Corrección de deriva del pulso	_scheduleNextPulse	Volver a Timer.periodic reintroduce un error que solo se nota tras varios minutos, que es cuando importa
Versión con una sola fuente	tool/app_version.mjs	El CHANGELOG documenta el fallo que evita: un tag v0.2.0 publicando archivos 0.1.0
Verificar el binario, no solo el repositorio	tool/verify_apk.mjs	Detecta un artefacto que compila, se firma, cuadra en checksum y se instala vacío
Coherencia documental automatizada	tool/validate_repository.mjs	Impide que el README prometa lo que el manifiesto no declara. Mejor relación líneas/errores evitados del repositorio
Contraste medido sobre el par real	test/theme_contrast_test.dart	Mide lo que se dibuja, no lo que la teoría sugiere. Es la razón de que exista AppColors.redDeep
Defensa en profundidad sobre permisos	Tres controles en tres momentos	Solo el tercero ve el manifiesto fusionado. Retirar cualquiera deja un hueco distinto
Una sola dependencia de ejecución	pubspec.yaml	Cada plugin nuevo es una vía por la que un permiso entra sin aparecer en el código
.gitattributes con LF forzado	.gitattributes	Los hashes de verify_apk.mjs dependen de ello
Honestidad documental	README, VALIDATION_RESULT.md, TEACHER_REVIEW.md	Declaran la clave efímera, la falta de firma Authenticode y la validación humana pendiente en vez de esconderlas

Continuar por

- [17 · Resumen ejecutivo](#) para el orden de atención.
- [14 · Troubleshooting](#) para los síntomas asociados.

16 · Glosario

Definiciones pensadas para que alguien sin formación técnica pueda leer el resto de la documentación. Los términos marcados con ★ significan algo **específico en este sistema** y no lo que significarían en otro proyecto.

Términos del producto

Actividad ★ Una de las tres tareas cronometradas que componen una clase. Duran 3, 4 o 5 minutos y sus tiempos suman exactamente la duración de la clase. En el código es `LearningActivity`.

Avance ★ La lista de clases que alguien ha marcado como completadas. Es lo **único** que la aplicación guarda. No incluye puntuaciones, tiempos ni nada que identifique a una persona.

Clase ★ Una lección de 12 minutos. Contiene objetivo, motivo, esquema de piso, tres actividades, reto, advertencia de seguridad, alternativa accesible y consejo para la persona adulta. En el código es `Lesson`. Hay 24.

Equivalencia funcional ★ Otra forma de hacer un movimiento que conserva la intención pedagógica aunque no reproduzca la forma. Marcar el pulso con la voz en lugar de con los pies, por ejemplo. Cada clase incluye una. Ver [../ACCESSIBILITY.md](#).

Esquema de piso (o diagrama de movimiento) ★ El dibujo que muestra el recorrido que hacen los pies durante una clase. Se dibuja con código, no es una imagen guardada. Lleva siempre una descripción en texto para quien no puede verlo.

Laboratorio de ritmo ★ La pantalla con seis círculos que se encienden por turno, como un péndulo. Sirve para aprender a sentir el compás. Puede acompañarse de un clic y de una vibración, ambos opcionales.

Nivel ★ Un grupo de tres clases con un mismo foco temático. Hay 8, y cada uno tiene un emoji y un título.

Pulso Cada uno de los golpes regulares que marcan el tiempo de la música. El laboratorio muestra seis y los agrupa de dos maneras distintas.

Acento ★ Un pulso que se marca más fuerte que los demás. En la agrupación 3+3 son el 1 y el 4; en la 2+2+2 son el 1, el 3 y el 5. La vibración solo se emite en los acentos, para que se note la agrupación y no sea un zumbido continuo.

Ruta de aprendizaje ★ El recorrido completo de las 24 clases. Ninguna está bloqueada: se puede abrir cualquiera en cualquier orden. La aplicación sugiere la siguiente pendiente, pero no obliga.

Términos técnicos

Activo (asset) Un archivo que viaja **dentro** del programa instalado, no descargado. Aquí las 24 clases son un activo: por eso la aplicación funciona sin conexión.

APK El archivo de instalación de una aplicación de Android, equivalente a un `.exe` de Windows.

Artefacto Cada uno de los archivos que se publican en una versión: el APK, los dos instaladores de Windows, el ZIP portable y el archivo de comprobación.

Bootstrap ★ El script que prepara el proyecto desde cero. Aquí hace falta de verdad, porque las carpetas de Android y de Windows no están guardadas en el repositorio: se regeneran cada vez.

Build (compilación) Convertir el código escrito por personas en un programa que la máquina ejecuta. «Un build en verde» significa que la compilación terminó sin errores, lo que **no** garantiza que el resultado sea correcto.

CI (integración continua) El sistema que revisa automáticamente cada cambio: formato, análisis y pruebas. Aquí es GitHub Actions y tarda unos minutos.

Commit Un cambio guardado en la historia del proyecto, con su autor, su fecha y su explicación. Se identifica con un código corto como `6efae74`.

Compuerta (*gate*) ★ Una comprobación automática que **detiene** el proceso si falla, en vez de limitarse a avisar. Este proyecto tiene seis, y esa severidad es deliberada.

Currículo ★ Aquí, el archivo `curriculum.json` con las 24 clases. Ojo: **no** es el currículum académico oficial de un país, aunque el proyecto se apoye en objetivos del currículum nacional chileno.

Deriva (del metrónomo) ★ El retraso que se acumula cuando cada golpe se cuenta desde el anterior en lugar de desde el inicio. Sin corregirla, un metrónomo se va quedando atrás minuto a minuto. Este proyecto la corrige, y es una de sus decisiones técnicas más cuidadas.

Esquema (*schema*) La descripción formal de la forma que debe tener un archivo de datos: qué campos lleva, de qué tipo y cuáles son obligatorios. Este proyecto **no tiene uno formal**; en su lugar usa validadores con las reglas escritas a mano.

Firma (de una aplicación) Un sello criptográfico que identifica quién construyó el programa. Android solo instala una versión nueva sobre la anterior si ambas tienen el mismo sello.

Firma efímera ★ Un sello creado en el momento de compilar y descartado después. Sirve para instalar, pero **no para actualizar**: la siguiente versión tendrá otro sello y Android exigirá desinstalar. Es la situación de la versión 0.1.0.

Hash (SHA-256) Una huella digital de un archivo. Si cambia un solo byte, la huella cambia por completo. Sirve para comprobar que lo descargado es exactamente lo publicado.

Landing La página web pública del proyecto, con capturas y enlaces de descarga. Se publica en GitHub Pages.

Local-first ★ Un diseño en el que todo funciona en el dispositivo, sin servidor. Aquí es absoluto: no hay ningún caso en que la aplicación necesite conexión.

Manifiesto El archivo que describe una aplicación: su nombre, su versión y los permisos que pide. Aquí hay dos que importan: `pubspec.yaml`, del proyecto, y el manifiesto Android que se genera al compilar.

Manifiesto fusionado ★ El manifiesto **final** de un APK, después de mezclar el del proyecto con los que aportan sus dependencias. Importa porque un permiso puede aparecer ahí sin haber sido escrito por nadie del proyecto. Es lo único que ve `verify_apk.mjs`, y por eso ese control existe.

Permiso (del sistema) Una autorización que el sistema operativo pide antes de dejar que una aplicación use algo sensible: la cámara, el micrófono, la ubicación. Esta aplicación **no pide ninguno**.

Capacidad (del dispositivo) ★ Algo que el dispositivo puede hacer y que **no** requiere permiso, como emitir un sonido o vibrar. El proyecto distingue con cuidado permiso de capacidad: sonido y vibración son capacidades, no sensores. No escuchan nada.

Persistencia Guardar información para que sobreviva al cierre del programa.

Persistencia transaccional ★ La regla más protegida de este proyecto: el programa solo muestra un cambio **después** de que el disco confirme que lo guardó. Si el guardado falla, la pantalla no miente.

Portable (versión) Una carpeta que se descomprime y se ejecuta sin instalar. Hay que extraerla **completa**: el ejecutable solo no funciona.

Release Una versión publicada, con sus artefactos y sus notas.

Repositorio El lugar donde vive todo el código y su historia.

Semántica (etiqueta) ★ Una descripción en texto que se adjunta a un elemento visual para que un lector de pantalla pueda anunciarlo. Los esquemas de piso llevan una.

Smoke test Una prueba rápida que comprueba que lo esencial funciona: que la aplicación abre y que se puede navegar. No comprueba detalles.

Validador ★ Un programa pequeño que revisa que algo cumpla sus reglas. Este proyecto tiene dos para el contenido y uno para el repositorio, y los tres funcionan **sin instalar Flutter**, lo que permite comprobarlo casi desde cualquier máquina.

Verificador del artefacto ★ `verify_apk.mjs`: abre el archivo de instalación ya construido y cuenta lo que lleva dentro. Existe porque un programa puede compilarse, firmarse y publicarse correctamente y aun así instalarse **sin contenido**.

Workflow Una receta automatizada que GitHub ejecuta al ocurrir algo: un cambio, una etiqueta de versión. Este proyecto tiene tres.

Términos de las herramientas

Dart El lenguaje de programación en que está escrita la aplicación.

Flutter La tecnología que permite escribir la aplicación una sola vez y que funcione en Android y en Windows.

Mermaid Una forma de escribir diagramas como texto. GitHub los dibuja solo; para el PDF hay que convertirlos a imagen antes.

Node.js El programa que ejecuta las herramientas de verificación del repositorio. Deliberadamente, esas herramientas no necesitan Flutter.

SharedPreferences El almacén sencillo que ofrece cada sistema operativo para guardar ajustes pequeños. Aquí guarda la lista de clases completadas.

WCAG Las pautas internacionales de accesibilidad. La regla que este proyecto mide es que el texto contraste al menos 4,5 a 1 con su fondo.

Términos culturales

Cueca Baile tradicional chileno, declarado baile nacional en 1979. **No tiene una sola forma correcta**: cambia entre territorios y comunidades, y este proyecto insiste en no presentar una variante como la única válida.

Escobillado Paso de cueca en el que los pies rozan el suelo suavemente. Aparece en la clase 11.

Medialuna Recorrido en forma de arco que hacen las dos personas que bailan, una frente a otra.

Pañuelo El pañuelo que se lleva en la mano y se mueve mientras se baila. Da nombre a la aplicación.

Persona cultora Alguien que conoce y transmite una tradición desde dentro de su comunidad. El proyecto declara que necesita la revisión de al menos dos, de contextos distintos, antes de considerar validado su contenido cultural.

Remate El cierre de una secuencia de baile.

Zapateo Golpear el suelo con los pies siguiendo el ritmo. El currículo lo plantea expresamente **sin impacto fuerte**, por seguridad.

3+3 y 2+2+2 ★ Las dos maneras de agrupar seis pulsos que el laboratorio contrasta. En la cueca pueden sentirse a la vez; separarlas es una simplificación para entrenar la escucha, y la propia aplicación lo advierte.

Para el detalle pedagógico y cultural, [../CURRICULUM.md](#), [../PEDAGOGY_AND_SAFETY.md](#) y [../SOURCES.md](#) siguen siendo la fuente autorizada.

Contenido de este documento

En tres frases

Estado del producto

Lo que hace bien, y que no es habitual

Lo que hay que decidir

Trabajo sin decisión: se puede hacer ya

Costes y dependencias

Riesgo de continuidad

Recomendación

Continuar por

17 · Resumen ejecutivo

Para decisión. Sin detalle técnico innecesario y con el esfuerzo de cada recomendación cuantificado.

En tres frases

Pañuelo al Viento es una aplicación educativa que enseña cueca a niñas y niños de unos diez años mediante 24 clases de 12 minutos, funciona sin conexión y no recoge ningún dato personal. Técnicamente está **verificada**: las pruebas pasan, la coherencia entre documentación y código se comprueba sola, y el artefacto publicado se abrió y se midió en vez de darlo por bueno. Pedagógica y culturalmente **no está validada**, y el propio proyecto lo declara sin maquillarlo.

Estado del producto

Dimensión	Estado	Comentario
Funciona e instala	Sí	Android 7+ y Windows 10/11 en cuatro formatos
Calidad técnica verificada	Sí	25 pruebas, análisis estático limpio, 3 validadores propios
Privacidad infantil	Sí, y comprobada tres veces	El APK publicado no declara ningún permiso del sistema
Distribución automatizada	Sí	Tag → cuatro artefactos con hashes, con compuertas en cada paso
Validación pedagógica	No	Falta revisión por docente de Educación Física

Validación cultural	No	Faltan al menos dos personas cultoras de contextos distintos
Prueba con niñas y niños	No	Pendiente, con autorización adulta
Auditoría de accesibilidad	No	Hay etiquetas semánticas; falta probar con TalkBack y Narrador
Actualizable sin perder datos	No	Clave de firma efímera. La primera actualización obligará a desinstalar

Lo que hace bien, y que no es habitual

Cuatro propiedades que distinguen este proyecto de una aplicación educativa corriente y que conviene no perder:

Verifica el artefacto, no solo el repositorio. Antes de publicar, abre el APK compilado, cuenta las clases que trae dentro y compara su huella con la del repositorio. Un programa puede compilar, firmarse, cuadrar en checksum e instalarse **vacío**; un build en verde no lo detecta.

La documentación no puede mentir sin que salte una alarma. Un validador comprueba que la versión del manifiesto, el historial, las notas de la versión, los nombres de los archivos del README y la página web digan todos lo mismo, y que las capturas que el README muestra existan de verdad.

La privacidad se comprueba en tres momentos independientes: en el código, en el archivo de configuración generado y en el paquete ya compilado. Solo el tercero puede ver los permisos que una dependencia añadiría sin aparecer en el código, y por eso existe.

Declara lo que no sabe. Que la firma es temporal, que los instaladores de Windows no tienen certificado comercial, que ningún docente ha revisado el contenido. Eso es una propiedad de calidad, no una debilidad: quien instala sabe qué está aceptando.

Lo que hay que decidir

Diecisiete hallazgos registrados en [15 · Riesgos](#), ninguno corregido. Estos son los que requieren una decisión, no solo trabajo:

1 · La firma permanente de Android · prioridad máxima

Mientras no se configure, **cada versión nueva obliga a desinstalar la anterior y borra el avance de quien la usaba**. Y como no hay exportación ni copia en la nube, ese avance se pierde para siempre; la guía para familias llega a recomendar anotar las clases en papel.

El problema crece con el tiempo: cuantas más personas instalen 0.1.0, más usuarios afectará la primera actualización.

- **Esfuerzo:** 1 hora de configuración, más la custodia segura del archivo de claves.
- **Decisión requerida:** quién custodia esa clave y dónde. Perderla impide actualizar para siempre.

2 · El pulso que sigue sonando al cambiar de pantalla

La documentación de permisos afirma que cambiar de pantalla detiene el metrónomo. **No lo hace:** verificado en pruebas durante este análisis. El clic sigue emitiéndose mientras se navega por el resto de la aplicación.

Es el hallazgo de mayor severidad porque toca el documento sobre el que descansa la confianza de una familia.

- **Esfuerzo:** 1–2 horas si se cambia el código, 15 minutos si se corrige el documento.

- **Decisión requerida:** ¿es un fallo o una función? Permitir que el pulso siga sonando mientras se lee una clase podría ser útil. Lo que no puede seguir es que el código y el documento se contradigan.

3 · La validación humana pendiente

Es lo que separa «una aplicación bien construida» de «un método pedagógico». Ninguna cantidad de pruebas automáticas la sustituye, y el proyecto ya tiene escrito el protocolo en [../TEACHER_REVIEW.md](#).

- **Esfuerzo:** semanas de calendario, no de programación. Requiere convocar a personas: una docente de Educación Física, dos personas cultoras de contextos distintos, y un grupo de 5–8 niñas y niños con autorización.
- **Decisión requerida:** presupuesto, o red de contactos, y quién coordina.

4 · Los siete esquemas de piso que son cinco

Siete clases de las 24 muestran un dibujo genérico porque su tipo de recorrido no tiene ilustración propia. Además, quien usa un lector de pantalla recibe dos descripciones distintas del mismo dibujo.

- **Esfuerzo:** 2–3 horas de programación.
- **Decisión requerida:** qué debe dibujar cada uno. Es una decisión pedagógica, no técnica.

Trabajo sin decisión: se puede hacer ya

Tarea	Esfuerzo	Valor
Corregir el roadmap, que da por pendiente la página web ya publicada	5 min	Evita planificar sobre información falsa
Unificar el nombre del nivel 8, distinto en el dato y en dos documentos	10 min	Coherencia
Sustituir «24 clases» escrito a mano por el valor real, en dos pantallas	15 min	Evita que la interfaz mienta si el currículo cambia
Decir «12 minutos» donde hoy se dice «10 a 15»	10 min	Precisión
Añadir el documento desprotegido a la lista de archivos esenciales	5 min	Cierra un hueco del validador
Añadir la novena captura a la página web, o dejar de copiarla	10 min	Peso muerto en el artefacto publicado
Pruebas del laboratorio de ritmo	3–4 h	El hueco de cobertura con mejor relación esfuerzo/valor
Auditoría de dependencias en la verificación continua	2 h	52 paquetes hoy sin vigilancia

Las seis primeras suman menos de una hora y eliminan seis de los diecisiete hallazgos.

Costes y dependencias

Concepto	Situación
Infraestructura	Cero. No hay servidor, base de datos ni servicio de pago. Todo funciona en el dispositivo
Dependencias de terceros	Una de ejecución, 52 paquetes resueltos en total. Superficie muy pequeña
Servicios usados	GitHub Actions, Releases y Pages. Todos en el plan gratuito para repositorios públicos
Licencia	MIT. Sin música, video ni tipografías externas que licenciar
Mantenimiento recurrente	Actualizar la versión fijada de Flutter y regenerar las capturas cuando cambie la interfaz

El coste de operación es efectivamente cero. El coste real del proyecto es el **tiempo de las personas que deben revisar el contenido**.

Riesgo de continuidad

Riesgo	Nivel	Mitigación existente
Pérdida de la clave de firma	Alto	Ninguna: la clave todavía no existe. Al crearla hay que custodiarla
Dependencia de una sola persona mantenedora	Medio	El bootstrap regenera todo desde cero y la documentación es extensa
Obsolescencia del toolchain	Bajo	Las plataformas se regeneran con la versión instalada de Flutter, no con plantillas guardadas. Fue una decisión deliberada
Dependencias vulnerables	Bajo	Superficie mínima y versiones fijadas, pero sin vigilancia automática
Contenido cultural cuestionado	Medio	El proyecto ya declara que es una base introductoria y no una coreografía única

Recomendación

Publicar 0.1.0 fue razonable. El producto es instalable, seguro y honesto sobre sus límites.

Antes de 0.2.0, por orden:

1. Configurar la firma permanente de Android. Sin eso, cada persona que instale hoy perderá su avance mañana.
2. Resolver la contradicción del metrónomo: código o documento, pero no ambos.
3. Aplicar las seis correcciones documentales de menos de una hora.
4. Añadir pruebas al laboratorio de ritmo.
5. Convocar la revisión humana. Es lo más lento y lo que más cambia el valor del producto.

Lo que **no** conviene hacer antes de esa revisión: producir video, música o animaciones. El propio [../ROADMAP.md](#) lo dice, y [../PEDAGOGY_AND_SAFETY.md](#) es explícito: «ajuste del currículo antes de producir videos costosos». Grabar contenido sobre una secuencia que después hay que cambiar es la forma más cara de equivocarse en este proyecto.

Continuar por

- [15 · Riesgos](#) para el detalle de los diecisiete hallazgos.
- [01 · Visión general](#) para entender el producto.
- [../ROADMAP.md](#) para el plan del propio proyecto.

Contenido de este documento

Antes de nada: tres cosas que sorprenden

La primera hora

El primer día

La primera semana

Preguntas frecuentes de quien llega

Continuar por

18 · Guía de incorporación

Itinerario para quien llega hoy al proyecto. Está pensado para leerse en orden y en tres tramos: la primera hora, el primer día y la primera semana.

Antes de nada: tres cosas que sorprenden

Las tres cuestan tiempo si nadie las dice.

android/ y windows/ no existen. Están en `.gitignore` a propósito. Se generan con `tool/bootstrap.ps1` o `tool/bootstrap.sh`. Si ejecutas `flutter run` sin eso, no hay dispositivos.

La versión de Flutter importa. CI fija 3.44.6 y ejecuta `dart format --set-exit-if-changed`. El formateador de Dart cambia entre versiones: con otra, el formato falla aunque tu código sea correcto.

Buena parte de la verificación no necesita Flutter. Tres validadores en Node comprueban currículum, coherencia del repositorio y landing en segundos. Empieza siempre por ahí.

La primera hora

1 · Poner el proyecto en marcha (20 min)

```
.tool/bootstrap.ps1 # Windows
```

```
./tool/bootstrap.sh # Linux y macOS
```

El script genera las plataformas, aplica la identidad del producto, descarga dependencias y ejecuta las cuatro compuertas. Aborta al primer fallo, así que si termina, el proyecto está sano.

```
flutter run -d windows
```

2 · Usar la aplicación (15 min)

No leas código todavía. Recorre las cuatro secciones, abre una clase, marca las tres casillas, complétala y comprueba que el porcentaje cambia. Abre el laboratorio de ritmo y cambia entre 3+3 y 2+2+2. Reinicia el avance.

En quince minutos tendrás el modelo mental que hace falta para que el código se lea solo.

3 · Leer dos documentos (20 min)

- [../README.md](#) — qué es el producto y qué promete.
- [01 · Visión general](#) — qué hace, qué no hace y con qué decisiones.

4 · Ejecutar las compuertas (5 min)

```
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
flutter test
```

Verás Repositorio válido: 32 archivos esenciales..., Currículo válido: 8 niveles, 24 clases y 72 actividades. y All tests passed! con 25 pruebas. Ese es el estado normal; si algo sale distinto, algo cambió.

El primer día

Leer el código en este orden

Son 2 417 líneas. Se leen en una mañana, y el orden importa porque cada archivo se apoya en el anterior.

#	Archivo	Líneas	Por qué aquí
1	lib/domain/curriculum.dart	193	Los cuatro modelos. Sin dependencias. Todo lo demás habla de estos objetos
2	assets/content/curriculum.json	—	Abre una clase entera y compárala con el modelo. Con eso entiendes el contenido
3	lib/data/curriculum_repository.dart	27	Trivial, pero cierra el círculo
4	lib/data/progress_repository.dart	68	Lee los comentarios. Explican por qué la interfaz existe
5	lib/state/app_state.dart	126	El archivo más importante. Todas las reglas de negocio
6	lib/main.dart	89	El arranque y la pantalla de recuperación
7	lib/screens/home_shell.dart	143	La navegación y el <code>IndexedStack</code> , que tiene una consecuencia que hay que conocer

8	<code>lib/screens/lesson_screen.dart</code>	299	El flujo con más reglas de producto
9	<code>lib/screens/rhythm_lab_screen.dart</code>	356	El único con lógica temporal. Lee <code>_scheduleNextPulse</code> despacio
10	El resto	—	Se leen solos

Después: [06 · Explicación profunda](#), que recorre esos mismos archivos por flujos en vez de por archivo.

Las cuatro reglas que no se rompen

Si te quedas con cuatro cosas de este día, que sean estas.

1. La persistencia se confirma antes de publicarse.

```
await _progressRepository.writeCompletedLessonIds(updated); // puede lanzar
_completedLessonIds = updated; // solo si no lanzó
notifyListeners();
```

Invertir ese orden produce el fallo más caro del producto: avance que se ve y desaparece al reabrir. Hay tres pruebas dedicadas a impedirlo.

2. `lib/domain` no importa nada. Ni Flutter. Eso es lo que permite validar el currículo sin levantar un entorno de widgets. Un import `'package:flutter/...'` ahí rompe tres pruebas y un validador.

3. La versión sale de `pubspec.yaml` y de ningún otro sitio. Todo lo demás la consume a través de `tool/app_version.mjs`. El CHANGELOG documenta el fallo que esa regla evita.

4. Cámara, micrófono e internet están fuera del producto. No por descuido: hay tres controles automáticos que bloquean la publicación si aparecen. Añadir una dependencia que los arrastre hará fallar la release.

Los comandos del día a día

```
flutter test # 25 pruebas, ~2 s
flutter analyze # ~3 s en caliente
dart format lib test tool # antes de commitear
dart format --output=none --set-exit-if-changed lib test tool # como lo ve CI
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
node tool/build_site.mjs --serve # landing en :8080
```

La primera semana

Documentos por orden de utilidad

Cuándo	Documento
Antes de tocar contenido	../CURRICULUM.md y ../PEDAGOGY_AND_SAFETY.md
Antes de tocar la interfaz	../ACCESSIBILITY.md

Antes de tocar datos o dependencias	../PRIVACY.md y ../PERMISSIONS.md
Antes de tocar la publicación	13 · Despliegue y ../RELEASE_CHECKLIST.md
Para saber qué está roto	15 · Riesgos
Cuando algo falle	14 · Troubleshooting
Cuando busques un símbolo	05 · Referencia técnica
Cuando busques dónde vive algo	04 · Mapa del código y 19 · Trazabilidad

Primeras tareas recomendadas

Elegidas para que toques una parte distinta del sistema en cada una. Todas están registradas como hallazgos en [15](#), así que sabrás si acertaste.

Tarea	Toca	Dificultad
Corregir el ROADMAP , que da por pendiente la landing ya publicada (R-03)	Documentación	Trivial
Unificar el nombre del nivel 8 entre el dato y dos documentos (R-04)	Contenido y documentación	Trivial
Sustituir «24 clases» escrito a mano por <code>state.totalCount</code> (R-06)	Interfaz y estado	Fácil
Añadir docs/CONTENT_PRODUCTION.md a los archivos esenciales (R-12)	Herramientas	Fácil
Añadir ramas de dibujo para <code>pair</code> y <code>free</code> (R-02)	Widgets y pintura	Media
Escribir la primera prueba del laboratorio de ritmo (R-11)	Pruebas y tiempo	Media-alta

La última es la más formativa: te obliga a entender la corrección de deriva, que es la pieza técnica más cuidada del proyecto.

Cómo se contribuye aquí

[../CONTRIBUTING.md](#) es la fuente autorizada. Lo esencial:

```
flutter pub get
node tool/validate_repository.mjs
node tool/validate_curriculum.mjs
dart format lib test tool
flutter analyze
flutter test
dart run tool/validate_curriculum.dart
```

Y siete requisitos de contenido que **no** son negociables: lenguaje comprensible a los diez años, no presentar una variante local como la única correcta, incluir adaptación cuando un movimiento pueda excluir a alguien, no añadir

medios sin licencia documentada, nada de publicidad ni rastreo, y actualizar las pruebas cuando cambie la estructura del currículum.

Si tu cambio toca la interfaz, regenera las capturas (`node tool/capture_screenshots.mjs`). Si toca el empaquetado de Android, comprueba el binario (`node tool/verify_apk.mjs <apk>`).

Preguntas frecuentes de quien llega

¿Dónde está el estado global? No hay. `main` crea un `AppState` y lo pasa a mano por los constructores. Con quince archivos es la opción correcta; ver [03](#) · [Arquitectura](#).

¿Por qué el laboratorio de ritmo no recibe `AppState`? Porque no comparte nada. Es la única pantalla con constructor `const`.

¿Por qué hay dos validadores de currículum? Uno en Node y otro en Dart, con las mismas reglas. Es una duplicación registrada como R-08; CI solo ejecuta el de Node.

¿Por qué el currículum está en JSON y no en Dart? Para poder editarlo y validarlo sin recompilar, y para poder traducirlo o generar guías impresas en el futuro. Está razonado en [./ARCHITECTURE.md](#).

¿Puedo añadir una dependencia? Con cuidado. Cada plugin es una vía por la que un permiso puede entrar en el manifiesto fusionado sin aparecer en el código, y hay tres controles que bloquearán la publicación si eso ocurre. Hoy hay una dependencia de ejecución.

¿Por qué el metrónomo no usa `Timer.periodic`? Porque acumula el retraso de cada tick y se queda atrás sin recuperarse. Explicado en [06](#), flujo 5.

¿Puedo probar en Android sin dispositivo? Con un emulador, tras ejecutar el bootstrap. Para verificar un APK necesitas además las build-tools con `aapt2`.

¿Qué NO debo hacer nunca? Publicar el estado antes de confirmar la persistencia. Importar Flutter en `lib/domain`. Escribir la versión a mano en cualquier sitio. Añadir cámara, micrófono o red. Relajar un validador para que deje de fallar en vez de corregir lo que señala.

Continuar por

- [04](#) · [Mapa del código](#) cuando busques dónde vive algo.
- [19](#) · [Matriz de trazabilidad](#) para ir de una funcionalidad a su prueba en una fila.

19 · Matriz de trazabilidad

Cada fila permite seguir una funcionalidad desde la interfaz hasta la persistencia y sus pruebas **sin salir de la fila**.

Cuando una celda dice **ninguna**, es un hueco de cobertura real, no una omisión de este documento.

Convención de nombres abreviados. Para que cada fila quepa sin partirse, la columna de módulo da solo el nombre del archivo, sin `lib/...` ni `.dart`, y la de prueba omite `test/` y `_test.dart`. Así, `lesson_screen` es `lib/screens/lesson_screen.dart` y `app_state` en la última columna es `test/app_state_test.dart`. El directorio de cada archivo está en [04 · Mapa del código](#).

Funcionalidades del producto

Funcionalidad	Dónde se ve	Módulo	Símbolo	Dato	Prueba
Ver el porcentaje de avance	Inicio · «Tu recorrido»	<code>home_tab</code>	<code>_ProgressCard</code> → <code>progress</code>	avance (lectura)	indirecta en <code>app_state</code>
Ver la próxima clase	Inicio	<code>app_state</code>	<code>nextLesson</code>	currículo + avance	ninguna
Ver el cierre de la ruta	Inicio	<code>home_tab</code>	<code>_FinishedCard</code>	avance	ninguna
Ir a la Ruta desde Inicio	Inicio · tarjeta	<code>home_shell</code>	<code>onOpenRoute</code>	ninguno	<code>app_smoke</code> (parcial)
Ir al Ritmo desde Inicio	Inicio · tarjeta	<code>home_shell</code>	<code>onOpenRhythm</code>	ninguno	<code>app_smoke</code> (parcial)
Recorrer los ocho niveles	Ruta	<code>route_tab</code>	<code>RouteTab.build</code>	currículo	<code>app_smoke</code> · <code>curriculum_integrity</code>
Ver el avance de un nivel	Ruta · anillo	<code>app_state</code>	<code>levelProgress</code>	currículo + avance	ninguna
Abrir el nivel a medias	Ruta · desplegable	<code>route_tab</code>	<code>initiallyExpanded</code>	avance	ninguna
Abrir una clase	Inicio y Ruta	<code>route_tab</code>	<code>Navigator.push</code>	currículo	<code>app_smoke</code> (parcial)
Leer el contexto	Clase · tarjeta	<code>lesson_screen</code>	<code>lesson.why</code>	currículo	<code>curriculum_integrity</code>
Ver el esquema de piso	Clase	<code>movement_diagram</code>	<code>_MovementPainter</code>	<code>lesson.diagram</code>	ninguna
Oír su descripción	Clase · lector	<code>movement_diagram</code>	<code>_description</code>	<code>lesson.diagram</code>	ninguna
Marcar las actividades	Clase · casillas	<code>lesson_screen</code>	<code>_activityChecks</code>	nada persistido	ninguna
Habilitar el botón	Clase · botón	<code>lesson_screen</code>	<code>_allActivitiesChecked</code>	estado local	ninguna

Completar una clase	Clase · botón	app_state → progress_repository	completeLesson → writeCompletedLessonIds	avance (escritura)	app_state ×2 · progress_repository
Rechazar clase inexistente	defensa interna	app_state	completeLesson → ArgumentError	protege el almacén	app_state
No publicar sin confirmar	defensa interna	app_state · progress_repository	orden await → asignación	avance	app_state
Repetir una clase	Clase · botón	lesson_screen	_finishRepeat	nada: no escribe	ninguna
Ver el avance por nivel	Avance	progress_tab	_HeroProgress · levelProgress	currículo + avance	app_smoke (navegación)
Ver cámara y micrófono	Avance · tarjeta	progress_tab	_CapabilityLine	ninguno; texto fijo	app_smoke (afirma ambos)
Reiniciar el avance	Avance · botón	app_state → progress_repository	resetProgress → clear	avance (borrado)	app_state · progress_repository
Confirmar antes de borrar	Avance · diálogo	progress_tab	_confirmReset	—	ninguna
Elegir la agrupación	Ritmo · segmentos	rhythm_lab_screen	_pattern → _accents	nada persistido	ninguna
Regular la velocidad	Ritmo · deslizador	rhythm_lab_screen	_bpm → _startSchedule	nada persistido	ninguna
Iniciar y detener el pulso	Ritmo · botón	rhythm_lab_screen	_toggle	ninguno	ninguna
Corregir la deriva	Ritmo · invisible	rhythm_lab_screen	_scheduleNextPulse	ninguno	ninguna
Emitir el clic	Ritmo · «Sonido»	rhythm_lab_screen	_emitPulse	ninguno	ninguna
Emitir la háptica	Ritmo · «Vibración»	rhythm_lab_screen	_emitPulse, solo acentos	ninguno	ninguna
Adaptar al ancho	Todas	home_shell	LayoutBuilder, umbral 840	ninguno	ninguna
Seguir el modo del sistema	Todas	app · app_theme	ThemeMode.system	ninguno	theme_contrast ×16
Recuperarse de un fallo	Pantalla de error	main	try → _StartupErrorApp	—	ninguna
Descartar avance huérfano	invisible	app_state	load → intersection	avance (lectura)	ninguna

Lectura de la matriz

Diecinueve de las treinta y tres filas dicen ninguna. Concentradas en dos zonas: **la interfaz** —un solo smoke test para cinco pantallas— y **el laboratorio de ritmo**, que no tiene ninguna prueba pese a ser el componente con más lógica del

proyecto.

Las filas mejor cubiertas son exactamente las que más importan: las tres reglas transaccionales del avance, con dos o tres pruebas cada una.

Compuertas del repositorio

Regla que se protege	Herramienta	Dónde se ejecuta	Qué comprueba	Fallo que evita
El currículo tiene 8 niveles y 24 clases	<code>tool/validate_curriculum.mjs</code>	CI, release, bootstrap	Cardinalidades, unicidad, órdenes, minutos, campos	Contenido incompleto en el binario
Ídem, en Dart	<code>tool/validate_curriculum.dart</code>	Manual	Lo mismo	— (duplicado, R-08)
Ídem, por el modelo real	<code>test/curriculum_integrity_test</code> <code>flutter test</code>		Lo mismo más el rango 10–15 min	Que el modelo y el validador diverjan
La versión es una sola	<code>tool/app_version.mjs</code>	Consumido por 4 herramientas	Formato <code>X.Y.Z+N</code> en <code>pubspec.yaml</code>	Publicar con dos versiones distintas
La documentación no miente	<code>tool/validate_repository.mjs</code>	CI, release, bootstrap	32 archivos, coherencia de versión, capturas, enlaces de notas, marcador de la landing	README que promete descargas inexistentes
La landing se puede ensamblar	<code>tool/build_site.mjs</code>	CI, release, <code>pages.yml</code>	Marcador presente, capturas presentes, imágenes referenciadas existentes	Landing con huecos rotos o descargas 404
No hay sensores en el código	<code>tool/validate_repository.mjs</code>	CI, release	Cadenas prohibidas en 7 archivos	Dependencia que arrastre un permiso
No hay sensores en el manifiesto fuente	<code>tool/configure_platforms.mjs</code> <code>· release.yml</code>	Bootstrap y release	<code>CAMERA</code> y <code>RECORD_AUDIO</code> en <code>android/app/src</code>	Permiso añadido a mano
No hay sensores en el manifiesto fusionado	<code>tool/verify_apk.mjs</code>	<code>release.yml</code>	Siete permisos prohibidos, incluido <code>INTERNET</code>	Permiso que aporta una dependencia
El APK lleva el contenido dentro	<code>tool/verify_apk.mjs</code>	<code>release.yml</code>	Currículo extraído, contado y comparado por SHA-256	Artefacto que se instala vacío
El APK declara la versión correcta	<code>tool/verify_apk.mjs</code>	<code>release.yml</code>	<code>versionName</code> , <code>versionCode</code> , <code>minSdkVersion</code> , ABIs	Publicar el binario equivocado
El APK está firmado	<code>apksigner</code>	<code>release.yml</code>	Validez de la firma	Artefacto no instalable

El ejecutable Windows arranca	release.yml	Release	Lo lanza 8 s y comprueba el código de salida	Binario que compila y no abre
El ejecutable Windows declara la versión	release.yml	Release	ProductVersion	Ídem que el APK
Los cuatro artefactos tienen el nombre exacto	release.yml	Release	Presencia y ausencia de sobrantes	Publicar símbolos o versiones mezcladas
El tag coincide con el manifiesto	release.yml, job version	Release	\$TAG_NAME == v\$APP_VERSION	Un tag v0.2.0 publicando archivos 0.1.0
Existen las notas de la versión	release.yml · validate_repository.mjs	Release y CI	docs/releases/v<versión>.md	Publicar sin explicación
El formato es reproducible	dart format --set-exit-if-changed	CI, release	22 archivos de lib, test y tool	Un paso que pasa siempre sin comprobar nada
El análisis estático está limpio	flutter analyze	CI, release	flutter_lints + 3 reglas propias	Uso de contexto tras await, entre otros
El texto se lee en ambos modos	test/theme_contrast_test.dart flutter test		16 pares WCAG, umbral 4,5:1	Una tarjeta ilegible en modo oscuro

Requisitos declarados y su evidencia

Trazabilidad inversa: cada promesa pública del proyecto y dónde se sostiene.

Promesa	Declarada en	Sostenida por	Estado
«24 clases, 8 niveles, 72 actividades»	README, landing, CHANGELOG	Dato + 2 validadores + 3 pruebas	Verificado
«Funciona sin conexión»	README, landing	Currículo empaquetado; sin APIs de red	Verificado
«No usa cámara ni micrófono»	README, landing, PRIVACY.md, interfaz	3 controles independientes	Verificado en el código; en el APK, según VALIDATION_RESULT.md
«El APK no declara ningún permiso»	README, PERMISSIONS.md	verify_apk.mjs	REQUIERE VALIDACIÓN aquí: no se reprodujo la medición por falta de aapt2
«Solo guarda identificadores de clases»	PRIVACY.md, interfaz	ProgressRepository, clave única	Verificado
«Guardado transaccional»	README, ARCHITECTURE.md	3 pruebas dedicadas	Verificado

«25 pruebas y 2 validadores»	README	flutter test → +25; dos .mjs en CI	Verificado
«Contraste WCAG AA en ambos modos»	README	16 pruebas	Verificado para los pares del ColorScheme; no cubre el lienzo (R-15)
«Seis patrones de diagrama»	README, CHANGELOG, notas	5 ramas de dibujo, 7 valores en el dato	Contradicho — ver R-02
«Clases de 10 a 15 minutos»	README, plan de 8 semanas, notas	Las 24 duran 12	Impreciso — ver R-07
«Cambiar de pantalla cancela el temporizador»	PERMISSIONS.md, paso 6	El IndexedStack no lo destruye	Contradicho — ver R-01
«Landing pendiente para 0.2»	ROADMAP.md	Ya publicada en el commit analizado	Contradicho — ver R-03
«Nivel 8: Diversidad y presentación»	README, CURRICULUM.md	El dato dice «Cuecas de Chile y presentación»	Contradicho — ver R-04
«Validación pedagógica pendiente»	README, VALIDATION_RESULT.md, TEACHER_REVIEW.md	Declarado, no automatizable	Correcto y honesto

Continuar por

- [15 · Riesgos](#) para los cinco contradichos.
- [12 · Pruebas y calidad](#) para los huecos de cobertura.